GENERACIÓN DINÁMICA Y CÓDIGO INCRUSTADO

Introducción

RESUMEN HTML



HTML BÁSICO – TABLA RESUMEN



Guía rápida de elementos esenciales

Elemento	Etiqueta	Ejemplo	Uso
Encabezados	`<h1> ... <h6>`	`<h1>Título</h1>`	Títulos y subtitulos
Párrafo	`<p>`	`<p>Texto del párrafo</p>`	Texto normal
Listas	`<ul>, <ol>, <li>`	`<ul><li>Item</li></ul>`	Elementos en lista
Tabla	`<table>, <tr>, <td>, <th>`	`<table><tr><td>Dato</td> </tr></table>`	Mostrar datos en filas y columnas
Enlace	`<a>`	`<a href="https://...">Enlace</a>`	Ir a otra página o recurso
Imagen	`<img>`	`<img src="foto.jpg" alt="Descripción">`	Insertar imágenes



Recuerda

- Las etiquetas suelen abrirse y cerrarse.
- Los atributos van dentro de la etiqueta de apertura.
- `<img>` no necesita etiqueta de cierre.
- La indentación ayuda a leer el código.



APLICACIONES WEB

- En esta unidad vamos **a aprender a construir aplicaciones web** que se ejecutan en el servidor utilizando las herramientas básicas del lenguaje y del servidor web, **sin recurrir todavía a frameworks** como Django(Python), Flask(Python) o Laravel(PHP).
- Existen **dos modalidades** principalmente que son las que abordaremos en la unidad:
  - **Generación dinámica de páginas web**: un programa ejecutado en el servidor construye el documento HTML que recibirá el navegador.
  - **Programación con código incrustado**: el documento HTML contiene instrucciones de programación que se ejecutan en el servidor antes de enviar la respuesta al navegador. No podemos usar este método con Python pero sí con Java y con PHP.
- **Formularios**: son un **elemento transversal** a ambas modalidades, porque permiten pasar de páginas que simplemente presentan información a aplicaciones que reciben datos del usuario, los procesan y generan respuestas.

PÁGINAS WEB ESTÁTICAS Y DINÁMICAS

- Cuando un navegador solicita una página web, envía una petición HTTP a un servidor web, que se encarga de procesarla y devolver una respuesta.
- Habitualmente**, esta respuesta es un documento HTML** que el navegador interpreta para presentar la página al usuario. Además, el documento HTML puede ser una página estática o una página dinámica

PAGINA WEB ESTÁTICA

- La pagina se entrega tal cual fue diseñada sin modificaciones

PAGINA WEB DINÁMICA

- Página generada en el servidor que se construye total o parcialmente durante el procesamiento de una petición HTTP, mediante la ejecución de un programa en el servidor.
- El contenido de la página variará dependiendo de los resultados de la ejecución

PAGINA WEB ESTÁTICA

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Saludo</title>
</head>
<body>
    <h1>Bienvenido a nuestra web</h1>
    <p>Hoy tenemos ofertas especiales.</p>
</body>
</html>
```

- Cada vez que un usuario solicite este archivo, **el servidor devolverá el mismo documento HTML**, independientemente de quién realice la petición o de cuándo se produzca.
- Para modificar el mensaje será necesario cambiar el contenido del archivo almacenado en el servidor.

PAGINA WEB DINÁMICA



- Una página web dinámica es aquella **cuyo contenido se genera total o parcialmente mediante un programa**. El resultado puede variar según la hora, los datos almacenados o la información proporcionada por el usuario.
- Fíjate como el programa en Python determina el contenido de la página, dependiendo de la hora

```python
# Obtener la hora actual del servidor
hora = datetime.now().hour

# Determinar el contenido de la página
if hora < 14:
    saludo = "¡Buenos días!"
    oferta = "Café"
    precio = 1.50
else:
    saludo = "¡Buenas tardes!"
    oferta = "Tarta de chocolate"
    precio = 2.50

# Cabecera HTTP (obligatoria en CGI)
print("Content-Type: text/html; charset=utf-8")
print()

# Generación dinámica del documento HTML
print(f"""
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Tienda online</title>
</head>
<body>
    <h1>{saludo}</h1>
    <h2>Oferta del momento</h2>
    <p>Producto: {oferta}</p>
    <p>Precio: {precio:.2f} €</p>
</body>
</html>
""")
```

GENERACIÓN DINÁMICA

- Python construye el documento HTML utilizando instrucciones del propio lenguaje.
- el HTML no existe previamente. Se construye ejecutando instrucciones Python.
- Se puede utilizar tanto CGI como WSGI, aunque la mecánica final es ligeramente distinta

```
#!/usr/bin/env python3


print("Content-Type: text/html")
print()


nombre = "Fulano"


print("<html>")
print("<body>")
print(f"<h1>Hola, {nombre}</h1>")
print("</body>")
print("</html>")
```

CÓDIGO INCRUSTADO

- El código incrustado plantea una estrategia diferente respecto a la generación dinámica: **partimos de un documento** fundamentalmente **HTML** y **colocamos determinadas instrucciones de servidor** en los puntos donde necesitamos introducir contenido dinámico.
- No es posible incrustar código Python, pero si PHP y Java

Ejemplo PHP incrustado

```
<h1>Catálogo de productos</h1>


<ul>
    <?php
        $productos = ["Libro", "Ratón", "Teclado"];
       foreach ($productos as $producto) {
            echo "<li>" . htmlspecialchars($producto, ENT_QUOTES, 'UTF-8') . "</li>";
        }
    ?>
</ul>
```

Metodos de HTTP: Get y post

HTTP

- **HTTP (*Hypertext Transfer Protocol*, protocolo de transferencia de hipertexto) es un protocolo de aplicación que establece cómo un cliente y un servidor intercambian peticiones y respuestas.**
- Aunque su nombre menciona el hipertexto, permite transferir HTML, imágenes, vídeos, JSON, archivos y otros contenidos.
- En una interacción habitual:
  - El cliente envía una **petición** con un método, una dirección del recurso, cabeceras y, cuando corresponde, un cuerpo.
  - El servidor devuelve una **respuesta** con un código de estado —como 200 o 404—, cabeceras y, cuando corresponde, un cuerpo.
- **HTTPS es HTTP protegido mediante TLS**, que cifra la comunicación y permite autenticar al servidor.
- HTTP utiliza **mensajes HTTP.** Un mensaje HTTP es un paquete con la petición que se encapsula dentro de un segmento tcp, el cual se encapsula dentro de una trama IP la cual viaja por internet hasta el ordenador destino

METODOS HTTP

- **En el contexto de esta unidad vamos a utilizar los métodos que usan los formularios HTML tradicionales GET y POST para enviar datos al servidor**. Pero hay otros muchos:
  - **HEAD:** Solicitar una respuesta como la de GET, pero **sin el cuerpo**. Se usa por ejemplo para consultar metadatos de un archivo sin descargar su contenido.
  - **PUT:** Crear o sustituir el estado del recurso situado en una dirección concreta. Se usa por ejemplo para sustituir los datos completos del producto; PUT /productos/25
  - **PATCH** Aplicar una modificación parcial a un recurso. Cambiar únicamente el precio del producto /productos/25.
  - **DELETE** Solicitar la eliminación del recurso identificado. Eliminar el producto /productos/25.
  - **OPTIONS** Consultar las opciones de comunicación disponibles para un recurso o servidor. Preguntar qué métodos admite una dirección. También se utiliza en comprobaciones CORS.
  - **CONNECT** Establecer un túnel a través de un intermediario. Pedir a un proxy un túnel para una conexión HTTPS.
  - **TRACE** Solicitar que se devuelva la petición recibida, con fines de diagnóstico. Examinar cómo llega una petición al servidor. Suele estar deshabilitado por seguridad.
- Un recurso en este contexto es un elemento que puede ser identificado mediante su URL: un producto, un usuario, un pedido, una imagen o un documento.
- Estos métodos (que no son get y post) se usan con API's y clientes que permiten construir peticiones HTTP específicos

CAMPOS DE UN MENSAJE HTTP

- Un mensaje http tiene tres campos:
  - **Línea de petición:** indica el método (POST/GET/OTROS), el recurso solicitado y la versión HTTP.
  - **Cabeceras**: proporcionan información adicional sobre la petición: host, Content-Type, Content-Length
  - **Cuerpo (body):** contiene los datos que enviamos al servidor.

```
#LINEA DE PETICION
POST /pedidos HTTP/1.1
#CABECERA
Host: www.libreria.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 27

#CUERPO
producto=libro&cantidad=2
```

EJEMPLO DE MENSAJE PUT

```
http://localhost:8080/productos/25
```

```
PUT /productos/25
Content-Type: application/json

{
"nombre": "Teclado inalámbrico",
 "precio": 45,
"stock": 8
}
```

Es importante destacar, que la petición la recibe una aplicación que hará con ella lo que considere conveniente. El put no implica necesariamente que se vaya a modificar el dato. Eso depende de la aplicación que procesa la petición

- El navegador envía un mensaje put a la url http://localhost:8080/productos/25. Esta URL indica cual es el recurso sobre el que se hace el put
- En este ejemplo http://localhost:8080/ es una aplicación o un sgbd en esa dirección IP y puerto
- La URL indica una tabla (o recurso asimilable) /productos/ y la fila (25) sobre la que se debe realizar el cambio de datos
- En el cuerpo del mensaje HTTP se escribe lo que se indica en la segunda imagen
  - Se indica es una operación put, sobre la tabla productos línea 25
  - Se indica el tipo del contenido en el cuerpo del mensaje (json)
  - Se indican un diccionario, con los campos y datos que se deben introducir en el registro

DIFERENCIA ENTRE LA ENTRADA CON GET Y POST

- GET y POST son dos métodos del protocolo HTTP usados generalmente cuando se hace clic en el botón enviar de un formulario.
- Aunque ambos permiten enviar peticiones al servidor, tienen finalidades diferentes:
  - GET: se utiliza para **solicitar al servidor un recurso**, por ejemplo, una página web, una búsqueda o una lista de productos.
  - POST: se utiliza para enviar **datos al servidor solicitándole que los procese**, por ejemplo, registrar un usuario, publicar un comentario o tramitar un pedido.

GET

- GET: se utiliza para solicitar al servidor un recurso, por ejemplo, una página web, una búsqueda o una lista de productos. Por ejemplo, imagina que el navegador del cliente realiza este get para la web www.libreria.com

```
GET /libros?autor=Cervantes HTTP/1.1
```

El resultado será la generación de una URL que contiene por un lado host+ruta y por otro usa el separador ? Para el query string con los parámetros

```
Get https://www.libreria.com/libros?autor=Cervantes
```

- **Una petición GET normalmente no tiene cuerpo.** Si necesitamos enviar parámetros, los incluimos en la URL, después del signo ? En la imagen se muestra un mensaje http de tipo get

```
GET /calculadora.py?op1=10&op2=5 HTTP/1.1
Host: localhost:8080
```

- El servidor recibe la petición, consulta la información y devuelve una representación del recurso, por ejemplo, una página HTML con los libros de Cervantes.
- Cada vez que en el navegador escribimos una URL por defecto se lanza un get

GET

- **Get NO es adecuado para transmitir contraseñas u otra información sensible en la URL**, pues esta puede quedar registrada en historiales, registros del servidor y otros sistemas.
- Get NO DEBE USARSE cuando estemos solicitando una operación que **puede modificar el estado de la aplicación**. Por ejemplo, si para realizar un pedido de dos unidades en el stock de un producto libro uso el mensaje HTTP de la petición get quedaría así:

```
GET /pedidos?producto=libro&cantidad=2 HTTP/1.1
Host: www.libreria.com
```

- Cada vez que la aplicación recibe un get como este: la aplicación crea un pedido de dos unidades de libro, rebaja en dos el stock etc etc
- **Problema: los navegadores, los robots de búsqueda o los sistemas de precarga de enlaces pueden realizar peticiones GET automáticamente. ¡Podrían terminar creando pedidos sin que esa fuese la intención del usuario!**

POST

- Si en el ejemplo anterior el usuario desea realizar un pedido de dos libros, no estaría consultando el estado del sistema, estaría solicitando cambiarlo. Por ejemplo:
  - Crear un nuevo pedido en la base de datos.
  - Actualizar existencias.
  - Registrar datos de compra.
  - Generar una confirmación.
- En este caso utilizaremos POST porque **estamos solicitando una operación que puede modificar el estado de la aplicación**. La información del Post **no se transporta en la URL,, sino en el cuerpo del mensaje HTTP**
- Un mensaje HTTP de tipo Post podría tener la siguiente estructura:

```
POST /pedidos HTTP/1.1
Host: www.libreria.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 27

producto=libro&cantidad=2
```

POST

- Los datos producto=libro&cantidad=2 no aparecen dentro de la URL: se encuentran en el cuerpo del mensaje HTTP (body).. Por ejemplo, la estructura típica de un mensaje post es:

```
#LINEA DE PETICION
POST /pedidos HTTP/1.1
#CABECERA
Host: www.libreria.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 23

#CUERPO
producto=libro&cantidad=2
```

- **Línea de petición:** indica el método (POST), el recurso solicitado y la versión HTTP.
- **Cabeceras**: proporcionan información adicional sobre la petición: host, Content-Type, Content-Length. En concreto, en formulrios empleraremos siempre: *Content-Type: application/x-www-form-urlencoded* esto indica que el formato del cuerpo es pares nombre=valor, separados por &.
- **Cuerpo (body):** contiene los datos que enviamos al servidor.

Una cuestión relacionada con CGI: utilizar POST no impide que la URL contenga una QUERY_STRING. Una petición POST también puede incluir parámetros en su URL y, simultáneamente, otros datos en su cuerpo.

TIPOS DE CONTENT-TYPE

Tipo de contenido	Uso habitual
application/x-www-form-urlencoded	Formularios HTML convencionales.
multipart/form-data	Formularios que incluyen archivos.
application/json	Envío/devolver de objetos JSON, habitual en APIs.
text/plain	Envío/devolver de texto plano.
text/html	Envío/devolver de texto HTML
text/css	Envío/devolver hoja de estilos CSS.
text/javascript	Envío/devolver código Javascript
image/jpeg	Envío/devolver imagen JPG
image/png	Envío/devolver imagen PNG
application/pdf	Envío/devolver documento PDF

EJEMPLO: CUANDO GET Y CUANDO POST

Operación de una aplicación web	Método recomendado
Consultar un catálogo de productos	GET
Realizar una búsqueda	GET
Filtrar resultados por categoría	GET
Consultar una ficha de usuario	GET
Enviar un formulario de registro	POST (se envían datos personales)
Iniciar sesión	POST (se envía una contraseña)
Publicar un comentario	POST (mucho texto para ponerlo en una URL)
Realizar un pedido	POST (se cambia el estado del sistema)
Subir una fotografía	POST (no se puede poner una imagen en la URL)
Solicitar un cálculo que requiere enviar muchos datos	POST (si se usa get la URL será demasiado larga)

RESPUESTAS HTTP

- Hemos hablado hasta ahora de los mensajes http que llevan peticiones get/post y otras al servidor
- Pero la respuesta del servidor también se envía al cliente (el navegador) en la forma de un mensaje http
- Una respuesta HTTP tiene tres componentes
  1. **Estado: e**s un código numérico que informa al cliente del éxito, fracaso o las incidencias que se produjeron atendiendo la petición. Ejemplos: 200 OK, 404 Not Found, 500 Internal Error Server
  2. Cabeceras: Indican al navegador la naturaleza de la información que recibirá en el cuerpo y su tamaño. En general serán dos campos Content-Type y Content Length Ejemplo: Content-Type: text/html; charset=utf-8 Content-Length: 4
  3. Cuerpo: información que el servidor desea hacer llegar al cliente: puede ser un fichero, o el texto HTML generado de una página web dinámica



Codificación de texto

CODIFICACIÓN DE TEXTOS

- En html no hay ninguna codificación de texto por defecto. Si no se indica un charset espécifico:
  - En una página html el navegador trata de detectarla, si no asume la codificación de textos de la máquina local que en nuestro entorno cultural sería **Windows-1252**,
  - En un formulario, se asume la codificación de la página que lo contiene
  - En Python se usa Unicode de 16bits
- Esto es un problema: pueden aparecer textos como EspaÃ±a si se han enviado como bytes UTF-8 pero se han intepretado con otra codificación
- Además, en cada país la visualización de la página puede variar totalmente dependiendo de la codificación configuradaen el host

CODIFICACIÓN DE TEXTOS

- Pero además de la presentación, en diversos territorios, tenemos que tener en cuenta que la comunicación entre:
  - Cliente y servidor
  - Servidor web-Servidor aplicaciones-interprete del lenguaje
- **…Se transmite mediante secuencias de bytes**
- Por tanto, **necesitamos convertir el texto en bytes, para poder transmitirlo,** por eso la regla va a ser:
- **Usar UTF-8 en todos los procesos: HTML, formularios, y Python** UTF-8 codifica el texto en bytes, que es exactamente como lo necesitamos para convertirlo en una secuencia de bytes que se pueda transmitir. Además:
  - Todos los caracteres Unicode tienen un equivalente UTF-8
  - Es la codificación habitual de la web y facilita que todas las partes interpreten el texto igual.

CODIFICACIÓN DE TEXTOS HTML Y FORMULARIOS

- En HTML no debes asumir una codificación (Unicode, utf-8 o utf-16) para los textos.
- En la cabecera del mensaje http se puede definir la codificación del texto en el cuerpo del formulario añadiendo charset=UTF-8 (u otro código). Por ejemplo, para un mensaje que transporta parámetros de un formulario la cabecera del mensaje HTTP contendrá

```
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
```

- En el formulario se puede indicar la codificación

```
<form action="/pedidos" method="post" accept-charset="UTF-8">
```

- Si el mensaje HTTP transporta texto HTML, y queremos indicar la codificación entonces el content-type debe ser

```
Content-Type: text/html; charset=utf-8
```

- En el Head del fichero html se debería poner también el charset

```
<head>
    <meta charset="utf-8">
</head>
```

CODIFICACIÓN UTF-8 EN PYTHON

- En todos los procesos que vamos a ver, el resultado que genere nuestro programa Python debe convertirse a UTF-8, mediante el método encode(codificación) de la clase texto

```
texto = "España"                # str
datos = texto.encode("utf-8")  # bytes
print(datos)         # b'Espa\xc3\xb1a'
print(type(datos))  # <class 'bytes’>
texto2 = datos.decode("utf-8")  # str: "España“
print(texto2)         # "España“
```

- Es importante entender que el resultado no es otra cadena str codificada en utf-8. Porque en Python no existe el tipo de datos byte. Lo más parecido es el tipo int, y se representa con 4 caracteres . Y un str, está compuesto por caracteres de 16bits
- Encode genera un objeto de tipo BYTES. Podemos visualizar un objeto de este tipo como una lista de bytes, o una secuencia de bytes. Por ejemplo, la ñ se convierte en dos bytes C3 y C1. En cualquier caso, una secuencia de bytes es exactamente lo que necesitamos para empaquetar e intercambiar datos entre aplicaciones
- Existe el método decode(codificación) que dado un objeto bytes lo convierte en un str con la codificación indicada

CODIFICACIÓN DE TEXTO

- En nuestros programas Python WSGI vamos a utilizar continuamente esta conversión str←→Bytes
- En CGI en cambio la conversión es automática porque generamos la salida de datos, en forma de bytes. Pero el comando print() hace una recodificación hacia el código establecido por defecto para sys.stdout
- Nuestro setup, tiene utf-8 por defecto como codificación de stdout. Por tanto print hace decodificación automática de la secuencia de bytes. Pero si no fuera así se podría establecer ejecutando

sys.stdout.reconfigure(encoding="utf-8")

```
# Convertir la respuesta a bytes en WSGI
return [html.encode("utf-8")]


# Indicar al navegador cómo interpretar esos bytes
start_response(
    "200 OK",
    [("Content-Type", "text/html; charset=utf-8")]
```

```
# para comprobar la codificación de stdout
import sys

print("Content-Type: text/plain; charset=utf-8")
print()
print(sys.stdout.encoding)
```

Comunicación síncrona y asíncrona entre servidor y cliente

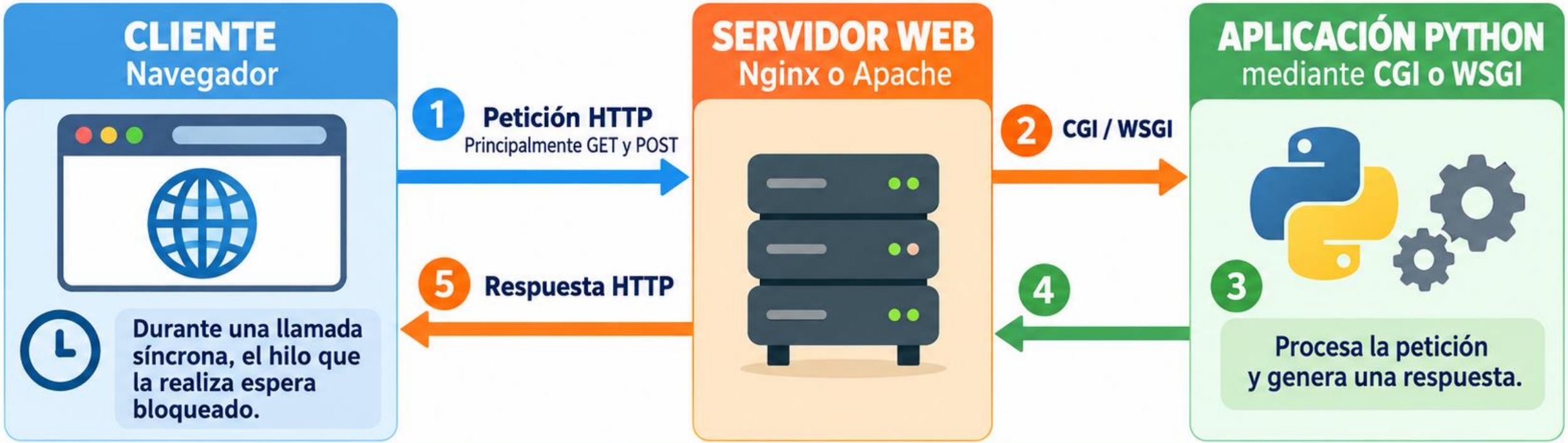
COMUNICACIÓN SINCRONA

- La diferencia entre comunicación síncrona y asíncrona está en **qué puede hacer el programa que realiza una petición mientras espera la respuesta**. En el caso de la **comunicación síncrona** lo que hace es **quedar a la espera d**e que la otra parte de una respuesta a su petición
  - En la comunicación síncrona y en el escenario que estudiamos en este tema se va a usar siempre HTTP
  - Las peticiones se envían siempre con métodos HTTP (HEAD, PUT, DELETE, PATCH, OPTIONS…pero sobre todo GET y POST)
  - Si la comunicación es síncrona: en el navegador del **cliente el hilo que hace la petición queda bloqueado** esperando la respuesta.
  - En el lado del servidor: el servidor de aplicaciones **ejecuta la aplicación que procesa la petición**, y **genera una respuesta** que el servidor web hace llegar al cliente por medio de http
  - La **aplicación en el servidor finaliza su ejecución** (esto admite matices, en CGI si, pero en WSGI lo que termina es la atención a la petición, aunque el proceso de fonde permanece activo)

PETICIÓN HTTP SÍNCRONA

Escenario de esta unidad: CGI y WSGI

El cliente espera la respuesta antes de continuar con la operación solicitada



Otros métodos HTTP: HEAD • PUT • DELETE • PATCH • OPTIONS

CGI: termina el proceso que ejecuta el script.

WSGI: termina la atención de la petición; el proceso permanece activo para atender nuevas peticiones.

COMUNICACIÓN ASÍNCRONA

- La diferencia entre comunicación síncrona y asíncrona está en **qué puede hacer el programa que realiza una petición mientras espera la respuesta**. En el caso de la **comunicación asíncrona** lo que hace es continuar realizando otras tareas mientras llega la respuesta. Cuando esta llega, se ejecuta el código encargado de procesarla. **Trataremos este tipo de comunicación en UD8: Aplicaciones interactivas, servicios web y APIs**.
    - El cliente (navegador) efectúa una petición al servidor pero no queda bloqueado, y permite seguir la interacción con el usuario mientras se recibe la respuesta
    - El servidor recibe una consulta y la procesa pero no queda bloqueado esperando que finalice, por ejemplo, esperando recibir respuesta de un SGBD , sino que puede vanzar en otras tareas o peticiones mientras esa operación termina.
  - **Importante:** una petición asíncrona desde el navegador no exige un servidor ASGI. Puede enviarse a una aplicación WSGI: la asincronía del cliente y la del servidor son aspectos diferentes.
  - Tanto HTTP como los métodos **GET y POST pueden emplearse tanto en operaciones síncronas como asíncronas**.
  - Pero en la comunicación asíncrona **una vez establecida la conexión por HTTP**, está **puede continuar usando WebSockets**, usando peticiones que ya no son HTTP

Comunicación síncrona y asíncrona

¿Qué hace el cliente mientras espera la respuesta?

SÍNCRONA



Si bloquéa el hilo principal, la página puede dejar de responder.

ASÍNCRONA

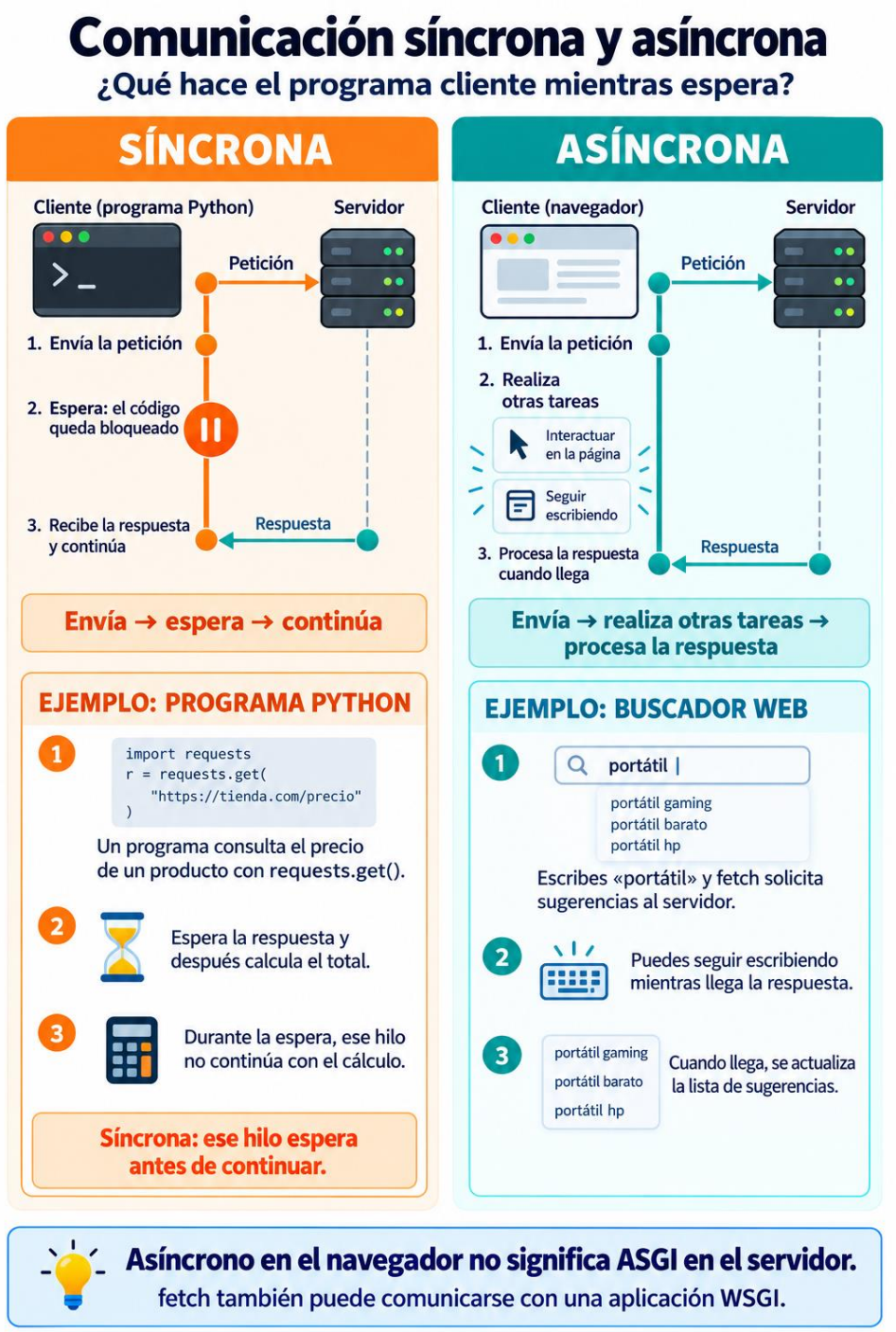


fetch permite peticiones asíncronas desde el navegador.

No exige ASGI: el servidor también puede utilizar WSGI.

COMUNICACIÓN SÍNCRONA Y ASINCRONA

- En la infografía se muestran ejemplos de comunicación síncrona y asíncrona
  - Un programa Python permite comunicación síncrona. El programa queda detenido a la espera de interacción con el usuario o respuesta a la petición.
  - Un ejemplo que podemos ver de comunicación asíncrona es el Fetch de Javascript. La aplicación sigue corriendo, y el usuario sigue interaccionando con ella a la espera de que se produzcan eventos externos. Otros ejemplos:
    - Un juego multijugador
    - Chat
    - Aplicación de mensajería

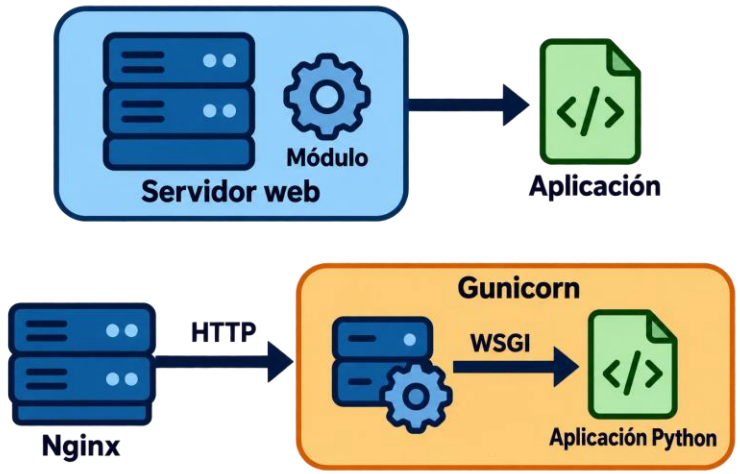


Interfaces entre el servidor y la aplicacion

INTRODUCCION

- Una página web dinámica puede construirse mediante un programa que se ejecuta en el servidor. Sin embargo, esto plantea una cuestión importante: ¿cómo consigue una petición realizada desde un navegador provocar la ejecución de un programa Python, PHP o de cualquier otro lenguaje en el servidor?
- Un servidor web, como Apache o Nginx, tiene entre sus funciones recibir peticiones HTTP y devolver las respuestas correspondientes. Cuando se solicita un archivo estático, puede limitarse a localizarlo y enviarlo al navegador.
- Sin embargo, **cuando una petición requiere ejecutar código, debe existir algún mecanismo que permita al servidor web comunicarse con el programa encargado de procesarla.**
- **En informática: Siempre que hay un proceso de comunicación entre dos partes, se tiene que implementar un protocolo. Es decir, unas reglas que establezcan como se debe establecer la comunicaión**

INTERFACES ENTRE EL SERVIDOR Y LA APLICACION



- Una **interfaz entre un servidor de aplicaciones y una aplicación web** es un conjunto de reglas (un protocolo de comunicaciones) que establece **cómo el servidor de aplicaciones entrega los datos de una petición a la aplicación web y cómo este devuelve la respuesta generada**.
- Estas reglas permiten repartir el trabajo: el **servidor de web gestiona la comunicación HTTP con el cliente**, el servidor de **aplicaciones pone en ejecución la aplicación entregándolo datos de entrada** mientras que la **aplicación procesa la petición y genera la respuesta**, incluidos su estado, sus cabeceras y, cuando corresponde, su contenido.
- Existen distintas formas de organizar esta ejecución. En algunos casos, el **servidor web incorpora un módulo que ejecuta la aplicación o lanza un programa externo, haciendo las funciones de servidor web**. En otros, **reenvía la petición a un servidor de aplicaciones**, como Gunicorn o Tomcat, que ejecuta el código correspondiente.

INTERFACES ENTRE EL SERVIDOR Y LA APLICACIÓN CGI – WSGI - ASGI

- En la industria existen múltiples interfaces entre servidor y aplicación, con múltiples configuraciones y acoplamientos. Es algo parecido a lo que en su momento contamos de los conjuntos o stacks tecnológicos. Cada lenguaje tiene un stack de interfaces:
- En el contexto de nuestro módulo van a ser:



Selección de interfaces para nuestra asignatura; existen otras alternativas.

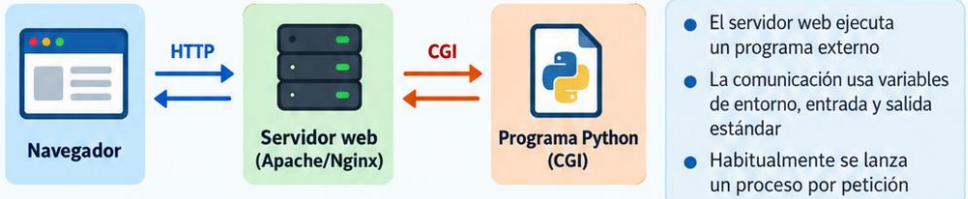
Ejecución de aplicaciones web en servidor sin frameworks

CGI y WSGI como mecanismos de comunicación

1 Conceptos básicos

Servidor web	Recibe peticiones HTTP y sirve contenido o las reenvía	**Ejemplos:** Apache, Nginx
Servidor de aplicaciones	Ejecuta la aplicación y genera la respuesta	**Ejemplo:** Gunicorn
Interfaz de comunicación	Define cómo se comunican servidor y aplicación	**Ejemplos:** CGI, WSGI

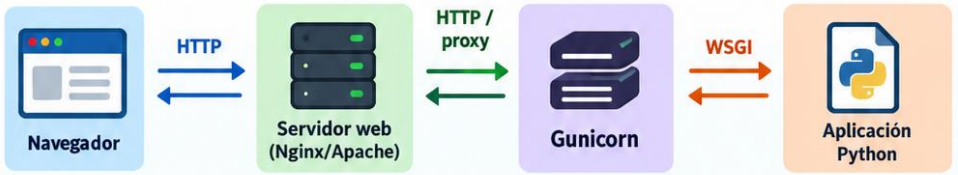
2 Modelo 1: CGI



- El servidor web ejecuta un programa externo
- La comunicación usa variables de entorno, entrada y salida estándar
- Habitualmente se lanza un proceso por petición

CGI no es un servidor de aplicaciones; es una interfaz.

3 Modelo 2: WSGI + Gunicorn



- Gunicorn es un servidor de aplicaciones
- WSGI define cómo Gunicorn invoca la aplicación Python
- La aplicación puede permanecer cargada y atender múltiples peticiones

WSGI no es un servidor de aplicaciones; es una interfaz.

Idea clave

CGI y WSGI no son servidores de aplicaciones. Son mecanismos de comunicación entre el servidor y la aplicación. Gunicorn sí es un servidor de aplicaciones WSGI.

- Vamos a estudiar dos modelos
  - Sin servidor de aplicaciones: uso de CGI
  - Con servidor de aplicaciones: WSGI Gunicorn
- Vamos aplicar cada uno de estos dos modelos a:
  - Generación dinámica de código
  - Codigo incrustado
- Para cada una de esa combinaciones estudiaremos la mecánica empleada para peticiones get y peticiones post

COMPATIBILIDAD CON LA INTERFACE

- No todos los programas escritos en Python son directamente compatibles con cualquier mecanismo de ejecución web.
- Para que un servidor web pueda utilizar un programa con la finalidad de generar una respuesta HTTP, ambos deben compartir las reglas de comunicación establecidas por el interface.
- Eso es precisamente lo que establecen CGI, WSGI, ASGI, etc.
- Un programa CGI, por ejemplo, es un programa en Python, PHP, Perl o de cualquier otro lenguaje que están preparados para comunicarse siguiendo las reglas de CGI con el servidor de aplicaciones o con un modulo CGI del servidor web.

INTERFACES ENTRE EL SERVIDOR Y LA APLICACIÓN
CGI

- CGI proporciona un interfaz estandarizado entre el servidor y la aplicación.
- En CGI **cuando hablamos del servidor** en general **no estamos hablando realmente de un servidor de aplicaciones como Gunicorn**, sino de un **servidor web con componentes adicionales** que le permiten ejecutar programas CGI. En general el servidor será:
  - Un servidor web como Apache, con un modulo CGI como mod_cgi
  - Una combinación servidor web+FastCGI+fcgiwrap (nuestro entorno de desarrollo)
- CGI es una de **las interfaces más antiguas y es independiente del lenguaje** lo que permite que pueda ser usadas en programas hechos en múltiples lenguajes de programación: Python, Rust, Kotlin, PHP, C, Perl, y Java.
- Pero eso **no significa que sea compatible con todas las tecnologías web**: Java puede usarse con CGI pero sus tecnologías web servlets y JSP no pueden usar CGI, ya que se ejecutan dentro de un contenedor tomcat, que es un entorno completamente distinto

INTERFACES ENTRE EL SERVIDOR Y LA APLICACIÓN
CGI

- CGI es hoy en día **una de las interfaces menos usadas**, ya que tiene un **problema grave de eficiencia**: crea un proceso por cada petición, lo que puede generar una situación de proliferación de procesos y tener como consecuencia una sobrecarga del servidor
- CGI, **utiliza mecanismos generales del sistema operativo para intercambiar información con el programa**. Esta es la razón por la que CGI puede ser utilizado por programas escritos en muchos lenguajes.
  - Entrada: variables de entorno con información de la petición y entrada estándar (stdin) para su cuerpo, cuando lo hay.
  - Salida: el programa escribe las cabeceras y el cuerpo de la respuesta en la salida estándar (stdout), que el componente encargado de CGI recoge, habitualmente mediante una tubería. tuberías para la salida estándar stdout
- **Al finalizar la ejecución del programa CGI, termina el proceso creado para esa petición.**

WSGI

- WSGI: WSGI: **es una interfaz diseñada específicamente para conectar en modo síncrono servidores de aplicaciones, (en nuestro despliegue Gunicorn), con aplicaciones web escritas en Python.** Remarquemos esta última parte: WSGI **usa objetos Python**, **no de otros lenguajes**, por eso **sólo es compatible con Python**. Cumple una función parecida a CGI, pero establece otra forma de ejecutar la aplicación y de intercambiar los datos de la petición y la respuesta.
- En nuestro set up cuando en WSGI nos referimos **al servidor hablamos de Gunicorn**, un servidor de aplicaciones (no un servidor web con módulos cgi, como en el caso de CGI). No obstante, en otros contextos **WSGI se puede implementar con otro conjunto de herramientas** que no son un servidor de aplicaciones (FastCGI+Flup+fcgiwrap)

WSGI

- **En nuestro despliegue** cuando **Gunicorn**, ejecuta una aplicación, **no crea un proceso independiente** para ella como en CGI.
  - **La aplicación se construye como funciones** o métodos de un **proceso persistente que se reutiliza** entre peticiones.
  - **Dentro de ese proceso se carga el interprete y la aplicación**, cuyo punto de entrada es una funcion Python invocable (una función o método).
- El servidor de aplicaciones (Gunicorn) **invoca el punto de entrada WSGI** de la aplicación como respuesta a las peticiones del cliente (que es como se ha indicado una función), evitando arrancar el interprete de Python repetidamente.

WSGI

- **No hay proliferación de procesos**. Gunicorn puede utilizar varios procesos trabajadores; pero no un nuevo proceso por petición. Esto permite **menor latencia, mayor capacidad para atender peticiones y reutilización de recursos ya inicializados como configuraciones o conexiones ya establecidas**
- **No usa únicamente recursos del sistema operativo para intercambiar información entre el servidor y la aplicación.** WSGI **define un contrato mediante llamadas y objetos Python** para entregar los datos de entrada y recoger la respuesta.
  - Entrada: **environ**: un diccionario con los datos de la petición que incluye wsgi.input para leer el cuerpo
  - Respuesta: **start_response**: una función proporcionada por el servidor que la aplicación llama para indicar el estado y las cabeceras. La aplicación devuelve un **iterable de bytes** con el cuerpo de la respuesta. Al finalizar del procesamiento termina la atención de la petición; pero el proceso principal sigue activo y puede atender otras.
- Al finalizar la atención de la petición, **el proceso trabajador permanece disponible para atender otras peticiones**.
- Django usa como base de la comunicación servidor/aplicación WSGI en procesos síncronos y ASGI en procesos asíncronos

ASGI

- **ASGI (Asynchronous Server Gateway Interface)**
- Es una **interfaz entre un servidor de aplicaciones y una aplicación web Python**.
- Permite la **ejecución asíncrona**: mientras una tarea espera una operación de entrada o salida, el servidor puede avanzar con otras tareas.
- Admite peticiones **HTTP** y conexiones **WebSocket**, útiles para chats, notificaciones o actualizaciones en tiempo real.
- Puede utilizarse como servidor de aplicaciones **Uvicorn o Daphne**.
- **ASGI se estudiará en el tema 8**, dedicado a la comunicación asíncrona. En esta unidad nos centraremos en **CGI y WSGI**.



OTRAS INTERFACES SERVIDOR -APLICACIÓN

- PHP:
  - Puede usar CGI tradicional mediante un módulo que extiende el servidor web (Apache)
  - Puede lanzar procesos PHP mediante su propio servidor de aplicaciones PHP-FPM, que se comunica con el servidor web mediante FastCGI
  - El interprete se puede integrar totalmente en Apache mediante el módulo mod_php
- JSP: Utiliza un servidor de aplicaciones independiente: Tomcat
  - Tomcat puede funcionar también como servidor web
  - Mantiene una JVM (maquina virtual Java)
  - Puede integrarse y trabajar con Nginx y apache
  - En ningún caso usará CGI
- Rust: Se puede incorporar a la aplicación programada en RUST su propio servidor web, mediante una biblioteca
  - No necesita un servidor web, ni un servidor de aplicaciones lo incorpora el lenguaje
  - Aunque puede incorporar FastCGI **para recibir peticiones reenviadas por Nginx o Apache**.
  - FastCGI no significa que ejecute programas CGI, significa que puede conectarse a otros servidores web.
  - Pero Sí se pueden crear aplicaciones web CGI escritas en Rust y hacer que un módulo externo como Fcgiwrap las ejecute.
- Kotlin: Como JSP usa Tomcat como servidor de aplicaciones
  - Pero como Rust, también puede incorporar un servidor web propio, mediante bibliotecas de la JVM
  - Puede usar CGI por el mismo mecanismo de Rust: Nginx + FastCGI + fcgiwrap

Páginas web con Python: lectura de parámetros

Aplicación web en Python

Generación dinámica
Python construye el HTML

CGI

GET

Leer
`os.environ.get("QUERY_STRING", "")`
↓
Interpretar
`urllib.parse.parse_qs()`

POST

Leer
`sys.stdin.buffer.read(n)`
↓ decodificar si son bytes
Interpretar
`urllib.parse.parse_qs()`

WSGI

GET

Leer
`environ.get("QUERY_STRING", "")`
↓
Interpretar
`urllib.parse.parse_qs()`

POST

Leer
`environ["wsgi.input"].read(n)`
↓ decodificar si son bytes
Interpretar
`urllib.parse.parse_qs()`

Código incrustado · Mako
HTML con expresiones y bloques Python

CGI

GET

Leer
`os.environ.get("QUERY_STRING", "")`
↓
Interpretar
`urllib.parse.parse_qs()`

POST

Leer
`sys.stdin.buffer.read(n)`
↓ decodificar si son bytes
Interpretar
`urllib.parse.parse_qs()`

WSGI

GET

Leer
`environ.get("QUERY_STRING", "")`
↓
Interpretar
`urllib.parse.parse_qs()`

POST

Leer
`environ["wsgi.input"].read(n)`
↓ decodificar si son bytes
Interpretar
`urllib.parse.parse_qs()`

GET: parámetros de la URL · POST: cuerpo del formulario application/x-www-form-urlencoded.
n = longitud del cuerpo indicada por CONTENT_LENGTH; el cuerpo leído se decodifica antes de parse_qs().
En WSGI, environ es el argumento de la aplicación. Mako no cambia cómo se leen los parámetros.

QUERY_STRING solo contiene parámetros de la URL, incluso en POST. JSON y archivos requieren otro tratamiento.

parse_qs() es una función, no una clase.

Formularios

FORMULARIOS

- Un formulario tiene este aspecto. Action establece que programa Python recibirá un post o un get con los datos

```
<form action="/cgi-bin/saludo.py" method="get">
    <label>Nombre:</label>
    <input type="text" name="nombre">


    <input type="submit" value="Enviar">
</form>
```

- Si introducimos “Fulano”en el cuadro de texto y pulsamos el botón enviar se genera un get

```
GET /cgi-bin/saludo.py?nombre=Fulano HTTP/1.1
```

- Si cambiamos el método de get a post, se genera una petición http post
- Se puede indicar la codificación de los datos con accept-charset=“código "

```
<form action="/cgi-bin/saludo.py" method="post“ accept-charset="UTF-8">
    <label>Nombre:</label>
    <input type="text" name="nombre">
    <input type="submit" value="Enviar">
</form>
```

```
POST /cgi-bin/saludo.py HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 13
Char-set:”utf-8”

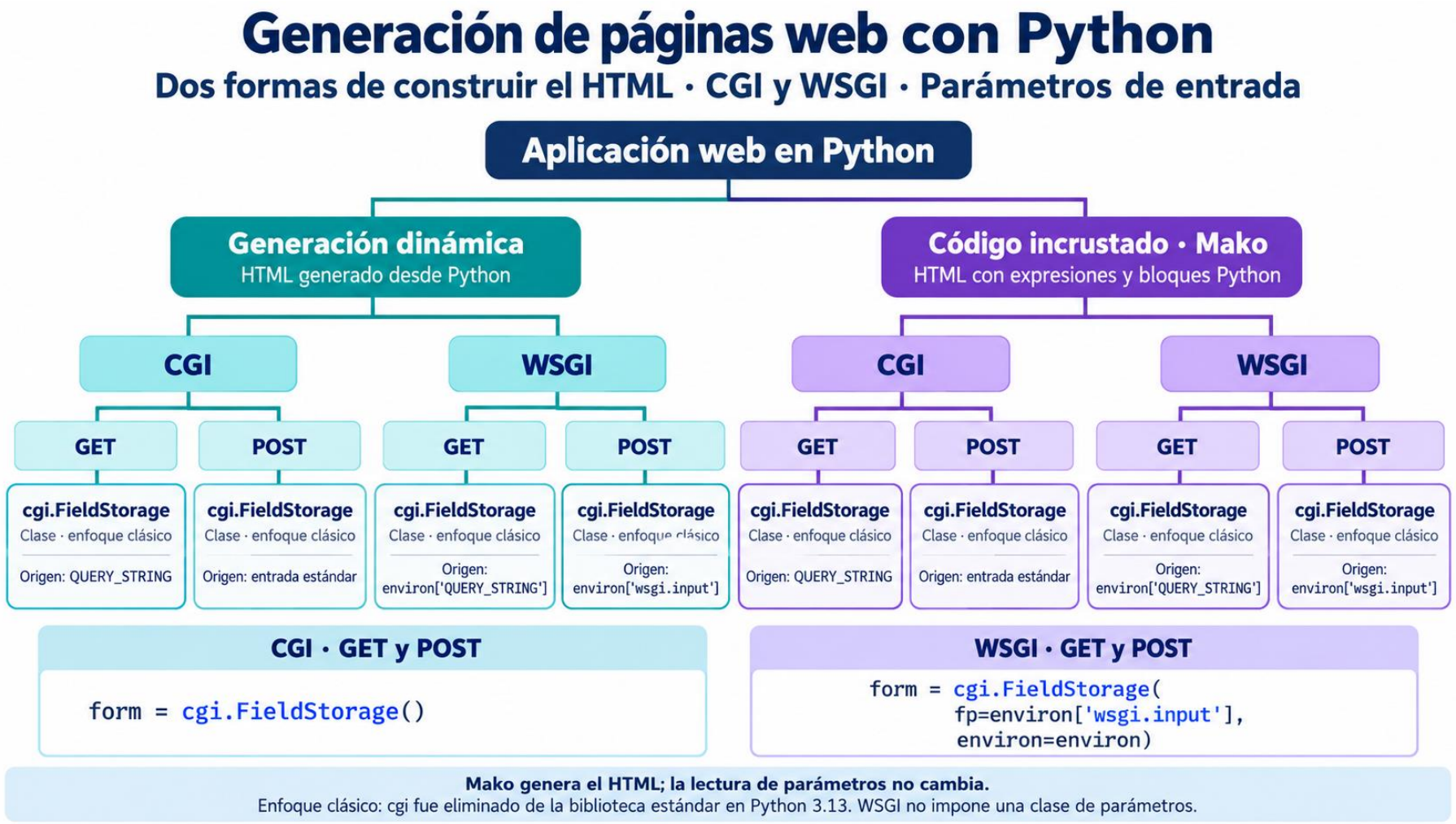
nombre=Felipe
```

RESUMEN FORMULARIOS



•FIELDSTORAGE

- Antiguamente se utilizaba el módulo cgi y dentro de él la clase FieldStorage para todas las modalidades de lectura de parámetros tanto get como post
- No obstante, el modulo cgi ha sido deprecado (eliminado por poco uso y obsolescencia) en Python 3..1



CGI

- Generación dinámica usando CGI
- Código incrustado Mako usando CGI

¿CÓMO EJECUTA NGINX UN PROGRAMA CGI?



CGI
Interfaz entre el ejecutor y el programa.

FastCGI
Protocolo de comunicación entre Nginx y fcgiwrap.

fcgiwrap
Servicio externo que debe estar instalado y disponible.

Nginx no ejecuta directamente el CGI: delega su ejecución en fcgiwrap.

Apache puede ejecutar CGI mediante **mod_cgi** o **mod_cgid**, según su configuración.

CGI Y NGINX

- **Apache:** incorpora una implementación directa de cgi en el modulo mod_cgi o mod_cgid
- **Nginx** no dispone de soporte CGI directo integrado en la aplicación. Para utilizarlo, necesita un servicio externo, como **fcgiwrap. En este contexto, fcgiwrap va a hacer el rol servidor de aplicaciones**.
  - **Nginx se comunican entre si fcgiwrap siguiendo las reglas de intercambio del protocolo de comunicaciones FastCGI**, para el que sí disponen ambos de soporte integrado.
  - Cuando Nginx recibe una petición de ejecución de un programa CGI, **envía a fcgiwrap los datos de la petición (siguiendo reglas FastCGI).** Esto incluye: el método HTTP, los parámetros, el cuerpo de la petición si lo hay y la información necesaria para localizar el programa.
  - **fcgiwrap ejecuta el programa CGI** y le proporciona los datos de entrada mediante el mecanismo CGI: variables de entorno (get) y entrada estándar (post). En nuestro caso, el programa se ejecuta con el intérprete de **Python en un proceso independiente**.
  - El programa genera su respuesta por la **salida estándar (stdout)**. **fcgiwrap** la recoge por una tubería y la entrega a Nginx (con FastCGI).
  - **Nginx envía la respuesta al navegador**, que muestra el resultado al usuario.
  - Para que funcione, **fcgiwrap debe estar instalado y disponible para atender las peticiones de Nginx**.

FASTCGI

- Es importante entender que es FastCGI.. Es un protocolo de comunicación que estandariza las reglas que deben seguir para intercambiar datos un servidor web (como Nginx) y un servicio externo a él que ejecuta programas por delegación (por ejemplo fcgiwrap).
- Por eso decimos que Nginx envia a fcgiwrap datos mediante FastCgi o que fcgiwarp devuelve datos a Nginx mediante FastCGI. Lo que se está indicando que El intercambio de datos sigue las reglas de comunicación establecidas por FastCGI



RECORRIDO DE UNA PETICIÓN CGI GET

- Por ejemplo: imaginemos que el servidor recibe esta petición

```
http://localhost/cgi-bin/saludo.py?nombre=Ana
```

- La petición implica ejecutar un recurso en la carpeta cgi-bin. El servidor tiene configurada esta carpeta como fuente de programas CGI por tanto sabe que tiene que ejecutar un programa y que tiene que buscarlo ahí
- El recurso es el programa saludo.py. El programa puede estar en cualquier lenguaje de programación, pero una línea al principio del fichero (**el shebang) le indica que es un programa Python**, y donde está el interprete que debe ejecutar
- El programa debe recibir como parámetro de entrada nombre= Ana

- **El modulo CGI crea un nuevo proceso del interprete de Python para que ejecute el programa** y le pasa el parámetro nombre=Ana mediante **la variable de entorno QUERY_STRING**. Esta es una variable de entorno predefinida por CGI

```
QUERY_STRING=nombre=Ana
```

CGI Y ATAQUES DE DENEGACIÓN DE SERVICIO (DOS)

- Antes se ha citado un hecho fundamental en CGI. **Cada petición es un proceso** que incorpora la **ejecución del interprete o máquina virtual** de un lenguaje de programación. Esto conlleva el uso de **recursos de memoria y CPU**
- **Si llegan muchas peticiones simultáneas, pueden acumularse procesos** y **saturar el servidor**. Históricamente, esta forma de funcionar de CGI ha sido usada para implementar **ataques de Denegación de servicio (DOS)**
- **WSGI reduce el coste del procesamiento de la petición al no crear un proceso para cada petición.** Aunque eso **no significa que sea invulnerable** a los ataques de DOS, ya que igualmente necesita ejecutar un programa, pero al menos no necesita poner en marcha un proceso independiente y cargar en él interprete, disminuyendo mucho el coste de las peticiones

SHEBANGS O HASHBANG

- **Linux** a diferencia de windows, **no usa la extensión de un archivo para decidir como ejecutarlo**, es decir, no interpreta automáticamente que un archivo terminado en .py debe ejecutarse utilizando Python.
- **El shebang (también se le puede llamar hashbang) es una línea que se coloca al principio de un script** en sistemas Unix/Linux para **indicar al sistema operativo qué intérprete debe utilizar para ejecutarlo.**
- Un shebang como el mostrado abajo en la imagen es necesario para usar Python con Nginx + fcgiwrap + CGI
- En nuestro contexto, lo que hace el shebang es indicar a Linux que debe ejecutar el programa usando el interprete de Python
- Fcgiwrap, no está pensado para ejecutar programas Python en exclusiva, en realidad, también puede ejecutar código en Perl o PHP. Por esto **no se puede dar por implicito el lenguaje** y **es necesario poner el shebang**
- Como antes hemos indicado, fcgiwrap crea un subproceso y le indica al sop que ejecute el fichero .py. Si Linux encuentra como primera línea en saludos.py un shebang , sabrá que interprete tiene que abrir para ejecutarlo. Pero si no hay shebang dará un error

```
#!/usr/bin/env python3

print("Content-Type: text/html")
print()
print("<h1>Hola mundo</h1>")
```

El nombre shebang procede de la forma en que se pronuncian los dos primeros caracteres de la línea
#! SH (sharp/hash) + BANG (!) = SHEBANG/HASHBANG (#!)

SHEBANGS

- La opción indicada de shebang #!/usr/bin/python3: exige que el intérprete esté instalado exactamente en esa ruta.
- #!/usr/bin/env python3: busca el ejecutable python3 utilizando las rutas definidas en la variable de entorno PATH. Es mejor que usemos este shebang

```
#!/usr/bin/python3

print("Content-Type: text/html")
print()
print("<h1>Hola mundo</h1>")
```

```
#!/usr/bin/env python3

print("Content-Type: text/html")
print()
print("<h1>Hola mundo</h1>")
```

- IMPORTANTE: El shebang tiene que ser siempre la primera línea de un fichero que contiene código Python. Ni siguiera puede haber comentarios antes que el shebang

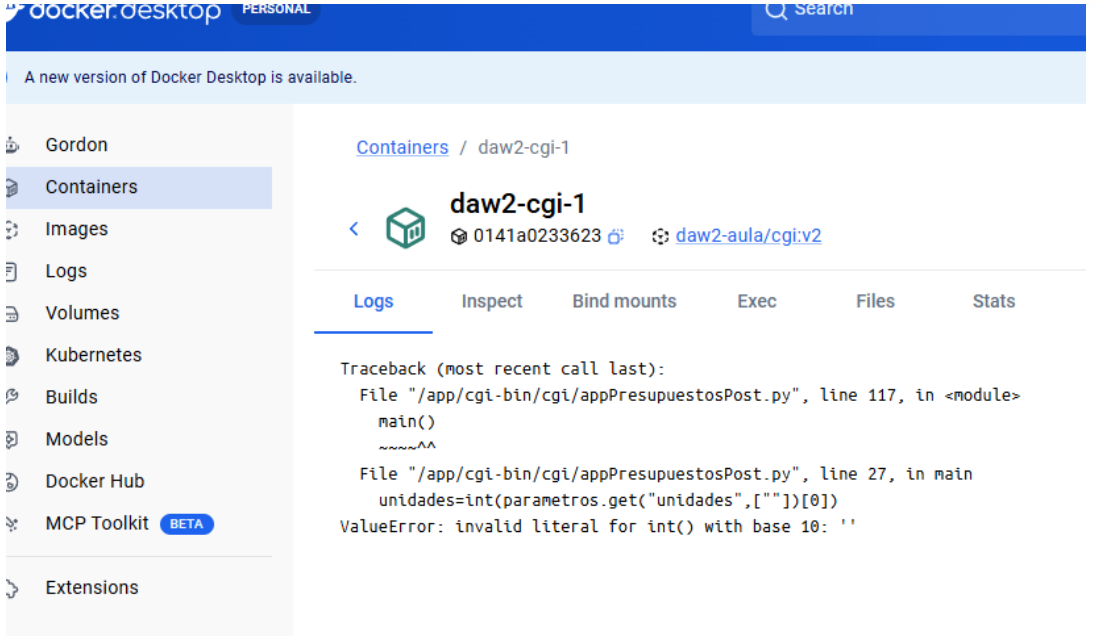
DERECHOS DE EJECUCIÓN

- Para que un archivo CGI pueda ejecutarse directamente en Linux, también debe disponer de permisos de ejecución. Por ejemplo:

```
chmod +x saludo.py
```

TRAZADO DE PROGRAMAS Y SEGUIMIENTO DE ERRORES

- Uno de los problemas que supone programar en contenedores es que para VS Code resulta imposible acceder a los errores que se producen en nuestros programas haciendo muy difícil la programación.
- No obstante, si un programa falla debemos ejecutar en el terminal una de estas instrucciones, que nos mostrarán los últimos errores producidos en la maquina. Por ejemplo para el contenedor cgi que contiene enginx+gcgiwrap+mako
- También es posible ver estos errores en Docker desktop seleccionando el contenedor cgi




```
docker compose logs --tail=50 cgi # ultimos 50 mensajes de error
docker compose logs --since=1m cgi #mensajes en el ultimo minuto
```

```
>> C:\2DAWdockers> docker compose logs --tail=50 cgi # ultimos 50 mensajes de error
cgi-1  | Traceback (most recent call last):
cgi-1  |   File "/app/cgi-bin/cgi/appPresupuestosPost.py", line 117, in <module>
cgi-1  |     main()
cgi-1  |     ~~~~^^
cgi-1  |   File "/app/cgi-bin/cgi/appPresupuestosPost.py", line 27, in main
cgi-1  |     unidades=int(parametros.get("unidades",[""])[0])
cgi-1  | ValueError: invalid literal for int() with base 10: ''
PS C:\2DAWdockers>
```

GENERACION DINÁMICA USANDO CGI

ENTRADA CGI

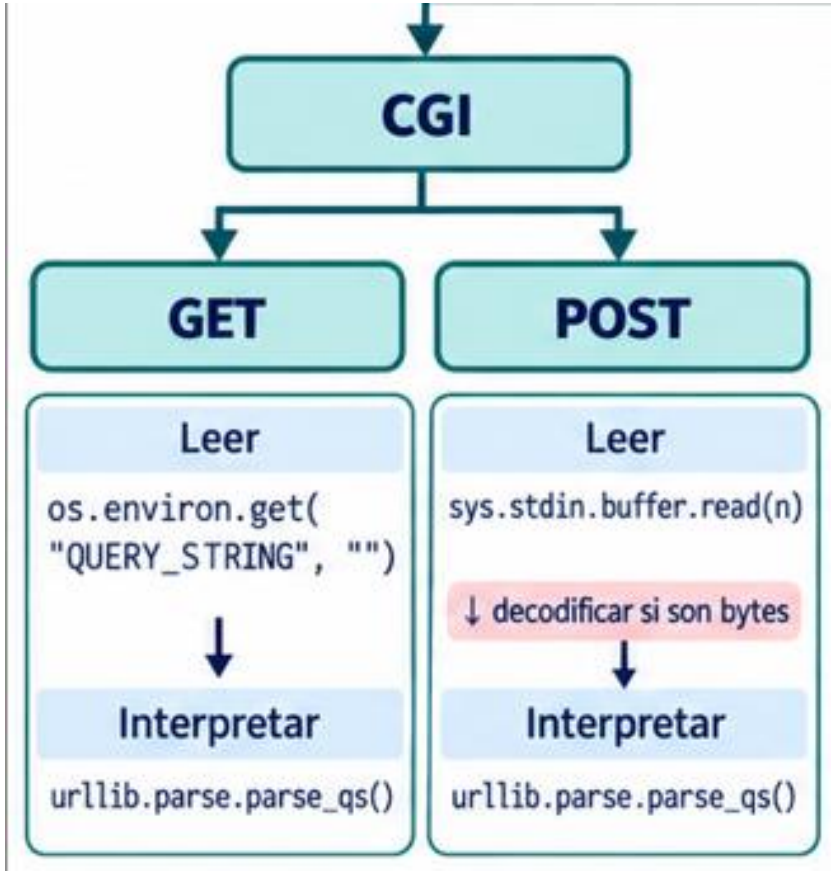
- Como se ha indicado, el módulo o servicio externo que implementa CGI, lanza un proceso por cada petición de una aplicación web que recibe.
- Los parámetros se pasan a la aplicación mediante **variables de entorno o por medio de la entrada estándar stdin**. Las VE son variables gestionadas por el sistema operativo. Cada proceso (mod CGI) que crea otro proceso (aplicación web), genera para el segundo un entorno de procesamiento que incluye un conjunto de variables de entorno con valores
- Se reciben al menos cuatro VE que reflejan el contenido de los cuatro campos del mensaje HTTP:
    - REQUEST_METHOD: es el campo método indicado en la cabecera del mensaje HTTP
    - CONTENT_LENGTH: longitud indicada en la cabecera del mensaje HTTP, dica cuántos **bytes** de longitud tiene el cuerpo . Se usa para saber cuantos bytes tiene que leer stdin para leer el cuerpo
    - CONTENT_TYPE: indicada en la cabecera del mensaje HTTP, es el tipo de datos contenido en el cuerpo del mensaje HTTP, Ver tabla de mensajes http. También indica la codificación de los elementos del cuerpo del mensaje
    - QUERY_STRING: es una cadena de texto con pares nombre-valor que indca los parámetros de entrada que puedan venir en la URL. **Por tanto sólo es útil cuando usamos el método get**

ENTRADA CGI

- QUERY_STRING proporciona los parámetros que se hayan incrustado en la URL (**método get)** Pero en el método post, no se incrusta nada en la URL, sino que los parámetros se introducen en el cuerpo de un mensaje HTTP.
- Es importante entender que **NO** SE PROPORCIONA EL CUERPO DEL MENSAJE HTTP POR VARIABLES DE AMBIENTE, ya que este podría información no textual, como imágenes por ejemplo.
- Por eso EN EL MÉTODO POST, además de leer las VE es necesario leer el cuerpo del mensaje HTTP usando la entrada estándar sys.stdin

ENTRADA CGI

- El tipo de entrada depende del método utilizado para hacer la petición:
  - Metodo get: se usan variables de ambiente que se pasan al proceso cuando este se crea, y que se leen en la aplicación con **os.environ.get(“QUERY STRING”)**
  - Metodo post: se usa la entrada estándar del proceso, es decir stdin. Para hacer la lectura se debe usar **sys.stdin.buffer,read(n)**
  - En ambos casos el resultado es una cadena de texto que se parsea para obtener un diccionario usando urlib.parse.parse_qs



UN PROGRAMA CGI

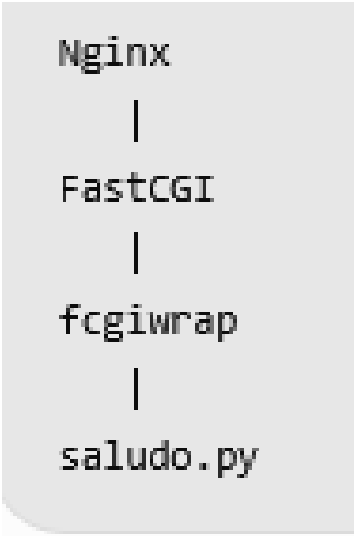
- Esto es una versión de saludo.py para CGI
  - Existe un Shebang (#!/usr/bin/python3)que permite identificar que interprete tiene que ser llamado
  - Obtiene información de la petición mediante variables de entorno (QUERY_STRING, etc.).
  - Genera la respuesta escribiendo en la salida estándar (stdout).
  - Escribe unas cabeceras CGI (como Content-Type), seguidas de una línea vacía y del contenido.
  - finaliza una vez generada la respuesta.

```python
#!/usr/bin/python3

import os
nombre = os.environ.get("QUERY_STRING", "")
print("Content-Type: text/html")
print()
print(f"<h1>Hola {nombre}</h1>")
```

- Podemos usar este programa en las dos arquitecturas CGI estudiadas en esta unidad (Apache y Nginx)





- Si la petición es GET (no hay cuerpo de la petición)
  - Usar os.environ.get("QUERY_STRING","") para obtener la entrada de datos desde variables de ambiente
- Si la petición es POST (los datos vienen en el cuerpo de la petición)
  - En cgi usar sys.stdin.buffer.read()

Si se usa el método get, los parámetros de entrada van a venir en la URL después de ?, por lo que nos va a bastar con leer el QUERY_STRING que se nos pase en la forma de una variable de entorno

```
URL: /cgi-bin/plan.py?asignatura=Python&minutos=100

QUERY_STRING: asignatura=Python&minutos=100
```

- En el caso de Post los parámetros se van a pasar en el cuerpo del mensaje HTTP
- CGI no pasa el cuerpo del mensaje en la forma de variables de entorno, por ello se debe usar la entrada estándar del proceso (en Python, sys.stdin,)
  - Primero se lee la longitud n del cuerpo del mensaje http a leer mediante la variable de ambiente CONTENT_LENGTH
  - A continuación se lee el buffer usando el método read de sys.stdin.read(n)

```
POST /cgi-bin/saludo.py HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 13


nombre=Felipe
```

CGI: dos vías de entrada



Petición HTTP

Servidor web / servicio CGI

Inicia un proceso por petición

Proceso Python (p. ej. plan.py)

CGI clásico

1 Variables de entorno

REQUEST_METHOD	Método: GET, POST…
QUERY_STRING	Parámetros de la URL, después de ?
CONTENT_LENGTH	Longitud del cuerpo en bytes
CONTENT_TYPE	Formato de los datos del cuerpo

Pares nombre–valor preparados para el proceso.

```
os.environ.get(...)
```

2 Entrada estándar · stdin

```
asignatura=Python&
minutos=100
```

stdin

El cuerpo de la petición llega por este canal.

```
sys.stdin.buffer.read(longitud)
```

CONTENT_LENGTH indica cuántos bytes leer.



El cuerpo puede contener datos de texto o binarios (por ejemplo, archivos).

Formulario GET

Datos en la URL → **QUERY_STRING**

```
/plan.py?minutos=100
```

Formulario POST

Datos en el cuerpo → **stdin**

```
minutos=100
```

POST también puede incluir parámetros en la URL: se reciben en **QUERY_STRING**.

COMO LEER VARIABLES DE AMBIENTE OS.ENVIRON

- Es un diccionario de Python, en el que se cargan las variables de ambiente asociadas por el sistema operativo a un proceso
- Es necesario importarla mediante una línea import os
- Es un objeto no invocable por tanto no requiere parámetros
- Devuelve un diccionario en el que las claves son los nombres de las variables de ambiente
- En el ejemplo env_var es una variable de tipo dict[str,[str]]. El cast dict(os.environ) es necesario porque aunque os.environ se comporte como un diccionario no es un objeto diccionario

```
import os
env_var: dict[str,[str]] = dict(os.environ)
print("User's Environment variable:")
print(env_var)
```

```
{'GPG_KEY': '7169605F62C751356D054A26A821E680E5FA6305',
 'HOME': '/home/guest',
 'HOSTNAME': 'dc2a4a3b0fee',
 'LC_CTYPE': 'C.UTF-8',
 'PATH': '/usr/local/bin:/usr/local/sb
}
```

OS.ENVIRON.GET (CLAVE, VALOR_POR_DEFECTO) OS.ENVIRON.[CLAVE]

- Obtener una variable de entorno especifica: se puede hacer usando get o seleccionándola entre corchetes.
- La diferencia es que **si se selecciona con corchetes dará una excepción KEYError** si la clave no existe.
- **Con get se puede establecer un valor por defecto para evitar la excepción** si la clave no se encuentra

```
import os
home: str = os.environ['HOME']
print("HOME:", home)
```

```
try:
    print(os.environ["MY_HOME"])
except KeyError:
    print("Environment variable does not exist")
```

```
import os
home: str =  os.environ.get('JAVA_HOME',"")
print("JAVA_HOME:", home)
```

También podemos cambiar el valor de una variable de ambiente usando os.environ

```
import os
os.environ['HOME'] = 'C:\TEMP'
```

OS.ENVIRON.GET (QUERY_STRING,””) (MÉTODO GET)

En el contexto de CGI usaremos os.environ,get(VARIABLE, VALOR DEFECTO) para obtener:

- REQUEST_METHOD: metodo = os.environ.get("REQUEST_METHOD", "")
- CONTENT_TYPE: tipo = os.environ.get("CONTENT_TYPE", "")
- CONTENT_LENGTH: tipo = ongitud = int(os.environ.get("CONTENT_LENGTH", "0") or "0")
  CONTENT_TYPE: QUERY_STRING: os.environ.get("QUERY_STRING", "")

- El primer parámetro es una variable de ambiente entre comillas
- El segundo parámetro es un valor por defecto si dicha variable no existe
- Valor devuelto: **Siempre una cadena si existe. Si no existe, devuelve el valor por defecto.**

```
Import os
# Si QUERY_STRING existe:
os.environ["QUERY_STRING"] = "nombre=Fulano“ # si no existe query string da excepcion
os.environ.get("QUERY_STRING", "")  # "nombre=Felipe“ # modo Seguro, Evita una excepción si query_string no existe

# Si no existe:
os.environ.get("QUERY_STRING", "")  # ""
os.environ.get("QUERY_STRING", None) # None
```

PARSING DE QUERY_STRING

- Un get de query string nos devuelve una cadena de texto con los parámetros de entrada separados por &
- Necesitamos separarlos y crear un diccionario con ellos. Para ello usamos el objeto parse_qs

```
parametros = "nombre=Fulano&edad=40&curso=DAW"
```

```
from urllib.parse import parse_qs
parametros = parse_qs(query)
```

Texto recibido de os.environ.get(Query string)

"nombre=Fulano&edad=40&curso=DAW"

Resultado de aplicar valor es una lista

```
{
  "nombre": ["Fulano"],
  "edad": ["40"],
  "curso": ["DAW"]
}
```

```
nombre = parametros.get("nombre", ["Invitado"])[0]

# 1) parametros.get("nombre", ["Invitado"]) devuelve una lista
# 2) [0] coge el primer valor
# 3) resultado: "Fulano"
```

- Muy importante: Parse devuelve un diccionario en el que la clave es el nombre del parámetro, pero el valor **ES UNA LISTA**
- **Esto es necesario, porque la cadena de parámetros de entrada podría ser: "nombre=Fulano& nombre=Mengano "**

URLPARSE.PARSE.PARSE_QS(STR)

- Parsea una cadena de texto que contiene pares claves=valor separados por el carácter & y devuelve un diccionario cuyos valores son listas
- Fíjate como la imagen muestra entre corchetes los valores. Esto es porque son listas. Pasrse_qs devuelve listas en lugar de valores indiviuales ya que una URL puede contener varias veces el mismo parámetro

```
GET /cgi-bin/saludo.py?nombre=Federico&edad=25&curso=DAW
import os
from urllib.parse import parse_qs

query = os.environ.get("QUERY_STRING", "")
#"nombre=Fulano&edad=25"
Parametros: dict[str,list[str]] = parse_qs(query)
```

```
parametros=
{
  "nombre": [“Federico"],
  "edad": ["40"],
  "curso": ["DAW"]
}
```

- Imagina que la petición http fuera

```
GET /cgi-bin/saludo.py?nombre=Federico&nombre=Ana&nombre=Luis &edad=25&curso=DAW
```

```
parametros=
{
  "nombre": ["Federico“,”Ana,”Luis””],
  "edad": ["40"],
  "curso": ["DAW"]
}
```

- En este caso parámetros contendría varios valores para la clave nombre, lo que hace necesario implementarlo como una lista

USO DE GET CON EL DICCIONARIO

- Para acceder a cada valor de la clave nombre tenemos que utilizar un indice

```
nombres: list[str]=parametros[nombre] #["Federico“,”Ana,”Luis”]
nombre0: str = parametros["nombre"][0]#Federico
nombre1: str = parametros["nombre"][1]#Ana
nombre2: str = parametros["nombre"][2]#Luis
```

- Para entender como funciona, es importante aclarar como se aplican los índices en Python

nombre0: str = parametros["nombre"][0].

- Primero se obtiene parametros["nombre"] que da como resultado ["Federico“,”Ana,”Luis”] Y a este resultado se le aplica el índice [0] que nos permite obtener “Federico
- Pero en general, es preferible usar el método get de la clase dict, porque permite evitar la excepción KeyError, que se daría si la clave no se encuentra nombre no se encuentra.
- Para ello es necesario introducir un valor por defecto, en este caso [“invitado”]
- Recuerda, parametros[nombre] devuelve una lista ["Federico“,”Ana,”Luis”], por eso al get es necesario pasarle el valor por defecto también como lista entre corchetes, [“invitado”]

```
nombres : list[str] = parametros.get("nombre", ["Invitado"]) #["Federico“,”Ana,”Luis”]

nombre0: str =parametros.get("nombre", ["Invitado"])[0] #Federico
nombre1: str =parametros.get("nombre", ["Invitado"])[1] #Ana
nombre2: str =parametros.get("nombre", ["Invitado"])[2] #Luis
```

URLPARSE.PARSE.PARSE_QSL(STR)

- Imagina que tenemos una petición con la query

query = "nombre=Federico&edad=25&nombre=Ana&ciudad=Valencia“

- El resultado de aplicar pase_qs es un diccionario en el que los valores se entregan en la forma de listas

```
from urllib.parse import parse_qs

parametros: dict[str, list[str]] = parse_qs(query)

print(parametros)
```

```
{
    "nombre": ["Federico", "Ana"],
    "edad": ["25"],
    "ciudad": ["Valencia"]
}
```

- Pero el uso del diccionario supone un problema, fíjate en el resultado: nombre se asocia con Federico y Ana, pero la edad, ¿es la de Federico o es la de Ana? Y la ciudad ¿es la de Federico o es la de Ana? Este es el problema de parse_qs, se obtienen todos los parámetros pero se pierde el orden en el que se han pasado y este orden es muy importante porque establece asociaciones entre los parámetros: en este caso, la edad se corresponde con federico y la ciudad con Ana

"nombre=Federico&edad=25&nombre=Ana&ciudad=Valencia“

URLPARSE.PARSE.PARSE_QSL(STR)

- Por eso a veces es útil usar parse_qsl en lugar de parse_qs, ya que lo que entrega es una lista de tuplas, es decir

list[tuple(str, str)]

```python
from urllib.parse import parse_qsl
parametros: list[tuple[str, str]] = parse_qsl(query)
print(parametros)
```

```
[
    ("nombre", "Federico"),
    ("edad", "25"),
    ("nombre", "Ana"),
    ("ciudad", "Valencia")
]
```

- Una tupla en Python, no contiene claves, contiene posiciones (a diferencia del diccionario en el que si dos claves están repetidas la segunda sobreescribe a la primera). Esto nos permite acceder a las filas de la tupla por posición en lugar de por el valor de la clave como se muestra en las dos imágenes abajo
- A tener en cuenta que print si imprime la líea completa la pones con comillas y paréntesis pero si imprime un elemento no pone comillas

```python
print(parametros[0])     # ("nombre", "Federico")
print(parametros[0][1])  # Federico
print(parametros[1])  # ("edad", "25"),
print(parametros[1][1])  # 25
print(parametros[2][1])  # Ana
```

```python
for clave, valor in parametros:
    if clave == "nombre":
        print(valor)
```

- Otro ejemplo dela utilidad de Parse_qsl en este caso mucho más claro

```
producto=libro&cantidad=2&producto=lapiz&cantidad=5
```

Con parse_qs() obtendríamos:

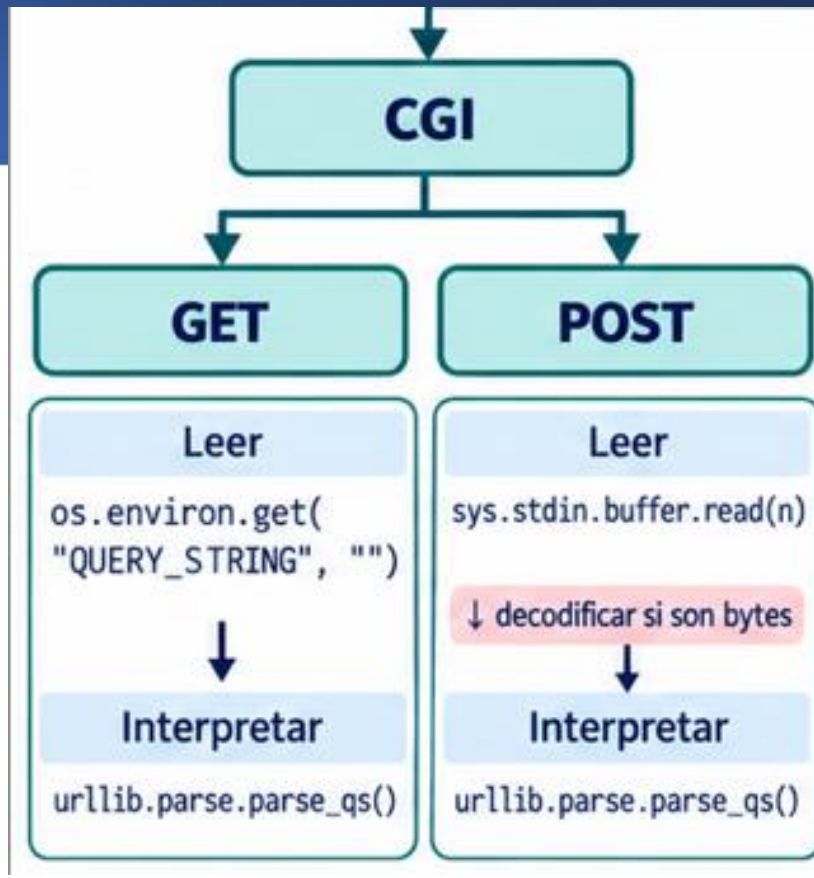
```
{
    "producto": ["libro", "lapiz"],
    "cantidad": ["2", "5"]
}
```

Con parse_qsl() conservamos exactamente esta secuencia:

```
[
    ("producto", "libro"),
    ("cantidad", "2"),
    ("producto", "lapiz"),
    ("cantidad", "5")
]
```

SYS.STDIN.BUFFER.READ()

- Si la petición viene en un POST los parámetros de entrada se ponen en el cuerpo del mensaje HTTP, tras una línea en blanco que los separa de la cabecera, y por tanto no podemos obtenerlos de la variable de ambiente QUERY_STRING
- Para leer el cuerpo del mensaje HTTP se usa la entrada estándar SYS.STDIN del proceso python.
- Fijate en la imagen de la izquierda, los datos de entrada están en el cuerpo del Post. Estos solo se pueden recuperar con sys.stdin.buffer.read(n) donde n es el número de bytes a leer
- En la imagen de la derecha en cambio, hay datos que están en la URL de la petición que se podrían recuperar con os.environ.get("QUERY STRING","") aunque esto no sería lo normal en un post




```
POST /cgi-bin/registro.py HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 13


nombre=Fulano
```

```
POST /cgi-bin/registro.py?nombre=Felipe&edad=25 HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 11


curso=2DAW
```

SYS.STDIN.BUFFER.READ() (MÉTODO POST)

- Antes de usar sys.stdin.buffer.read(), es necesario calcular cuantos datos hay que leer en el cuerpo, cosa que se logra leyendo la variable de ambiente CONTENT_LENGHT con os.environ.get

longitud = int(os.environ.get("CONTENT_LENGTH", "0") or "0")

El or "0") se debe a que si Content-length no existe get coge el valor por defecto '0', pero en ocasiones content-length si existe en la cabecera, pero con el valor cadena nula "" , y en esa caso, como existe, en lugar del valor por defecto se toma la cadena nula generando una excepción en el cast int()

Una vez tenemos la longitud usamos sys.stdin.buffer.read(longitud) para leer cadena de datos

- El resultado es una secuencia de bytes (un objeto BYTES). Por eso para poder usarlo en el program y convertirlo en una cadena de texto se le aplica el método **decode("utf-8")** con esto conseguimos que cuerpo sea una variable de tipo str

```python
#!/usr/bin/env python3
import os
import sys
from urllib.parse import parse_qs
from html import escape


# Obtenemos el tamaño del cuerpo en bytes
longitud = int(os.environ.get("CONTENT_LENGTH", "0") or "0")
# Leemos el cuerpo de la petición POST
cuerpo = sys.stdin.buffer.read(longitud).decode("utf-8")

# Convertimos los parámetros en un diccionario
parametros = parse_qs(cuerpo)

# Extraemos los valores, que inicialmente son cadenas
nombre = parametros.get("nombre", ["Sin nombre"])[0]
edad = parametros.get("edad", ["No indicada"])[0]

# Enviamos la respuesta CGI
print("Content-Type: text/html; charset=utf-8")
print()
print("<!DOCTYPE html>")
print('<html lang="es">')
print('<head><meta charset="utf-8"><title>Datos recibidos</title></head>')
print("<body>")
print(f"<p>Nombre: {escape(nombre)}</p>")
print(f"<p>Edad: {escape(edad)}</p>")
print("</body></html>")
```

SALIDA DE LA CABECERA DEL MENSAJE HTTP DE RESPUESTA

- En cgi la salida del programa (típicamente una cadena de texto html) se va a guardar en el cuerpo de un mensaje http de respuesta
- Pero previamente **es obligatorio generar la cabecera**
- Para ello es necesario como mínimo escribir usando print el parámetro content-type
- Y ha continuaciones obligatorio imprimir una línea en blanco, que es el elemento que hace de separador entre cabecera y cuerpo del mensaje http
- Lo dicho anteriormente es el mínimo imprescindible. Pero se pueden imprimir mas elementos de la cabecera como el content-lenght
- Ademas, antes de la cabecera se puede escribir una línea de estado indicando el éxito o fallo de la operación emplleando la clave status
- si no se escribe status, se entiende que el estado devuelto es 200ok. Por eso en general esta línea no hay que ponerla

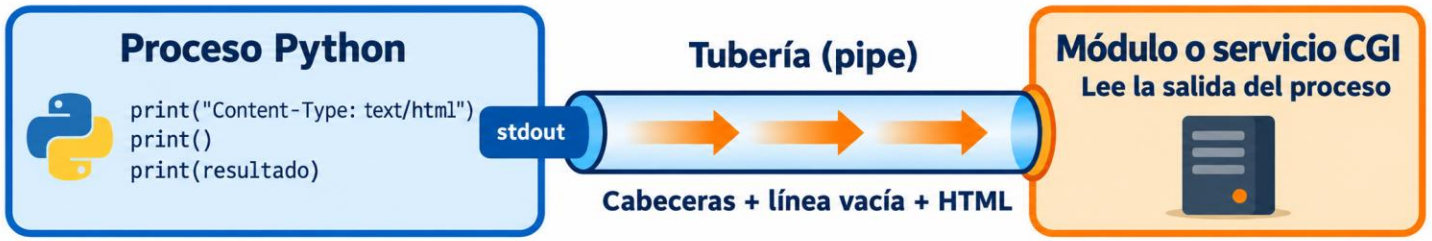
```
#MINIMO OBLIGATORIO PARA LA CABECERA
# cabecera
print("Content-Type: text/html; charset=utf-8")
print() # generar una línea de separación
Print(html) # generación del cuerpo
```

```
html = "<h1>¡Buenos días!</h1>"
cuerpo = html.encode("utf-8")
# generar cabecera
print("Content-Type: text/html; charset=utf-8")
print(f"Content-Length: {len(cuerpo)}")
print() # línea de separación con el cuerpo
print(html) #cuerpo
```

```
html = "<h1>¡Buenos días!</h1>"
cuerpo = html.encode("utf-8")
#generar estado respueta
print("Status: 200 OK")
# generar cabecera
print("Content-Type: text/html; charset=utf-8")
print(f"Content-Length: {len(cuerpo)}")
print() # línea de separación con el cuerpo
print(html) #cuerpo
```

SALIDA CGI PIPES - TUBERÍAS

- En sentido contrario (la salida del proceso que tiene que ser recogida por el el modulo CGI (o servicio externo) el procedimiento es el siguiente
- El modulo CGI establece, cuando crea el proceso en el que se ejecuta la aplicación Python, una tubería (pipe), en la que a un lado está el modulo CGI y al otro el stdout del proceso
- Como print() escribe en stdout pot defecto , cualquier cosa que se imprima por medio de un print dentro del proceso en el que se ejecuta la aplicación web, se lee en el modulo cgi por medio de la tuberia
- Nota: print no escribe necesariamente en el terminal, sino que, como en el caso de Java y Python, escribe en un flujo de salida standard llamado stdout que por defecto esta direccionado con el terminal, pero podría estarlo a un fichero o a una tuberia.
- Una tuberia es un canal que permite a dos procesos comunicarse. En este caso, a un lado de la tubería está el modulo CGI (Fcgiwarp) y por el otro al interp

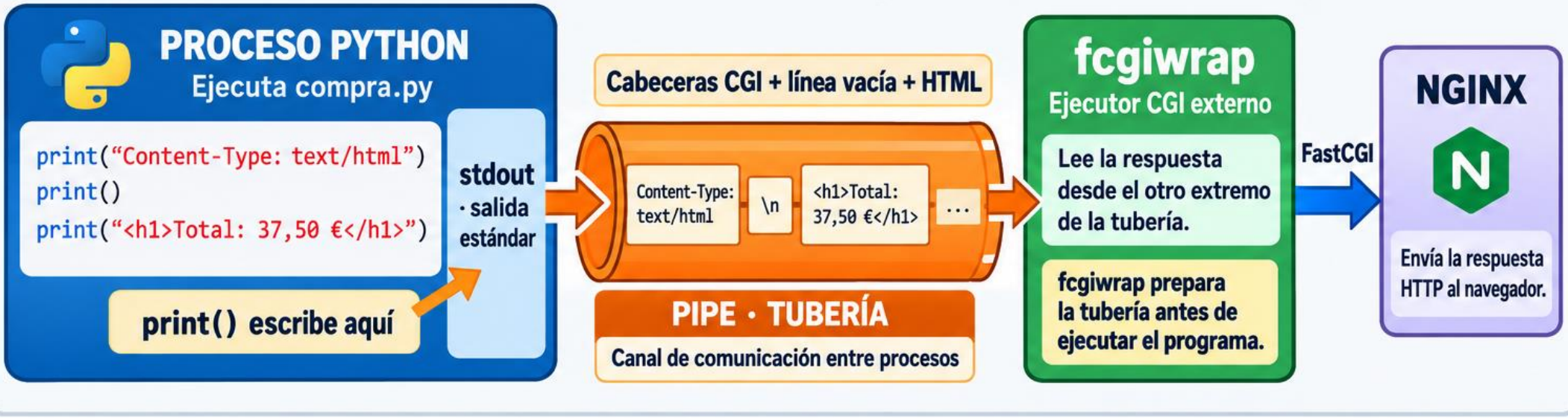


print() escribe por defecto en **stdout**

¿A DÓNDE VA print() EN UN PROGRAMA CGI?

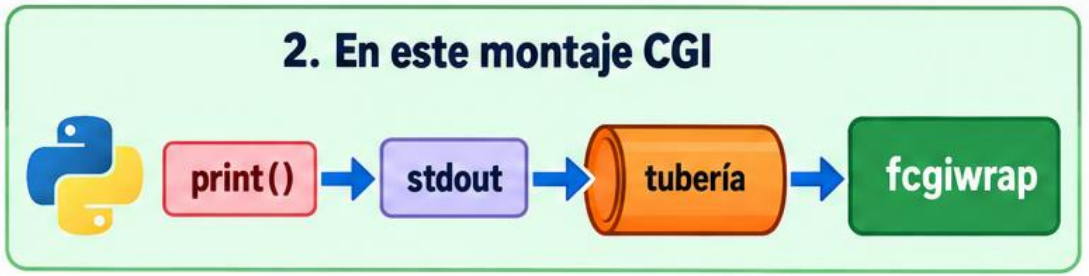
print() escribe en la salida estándar; no necesariamente en la terminal.

Comunicación entre procesos



EL DESTINO DE stdout DEPENDE DE CÓMO SE EJECUTA EL PROGRAMA

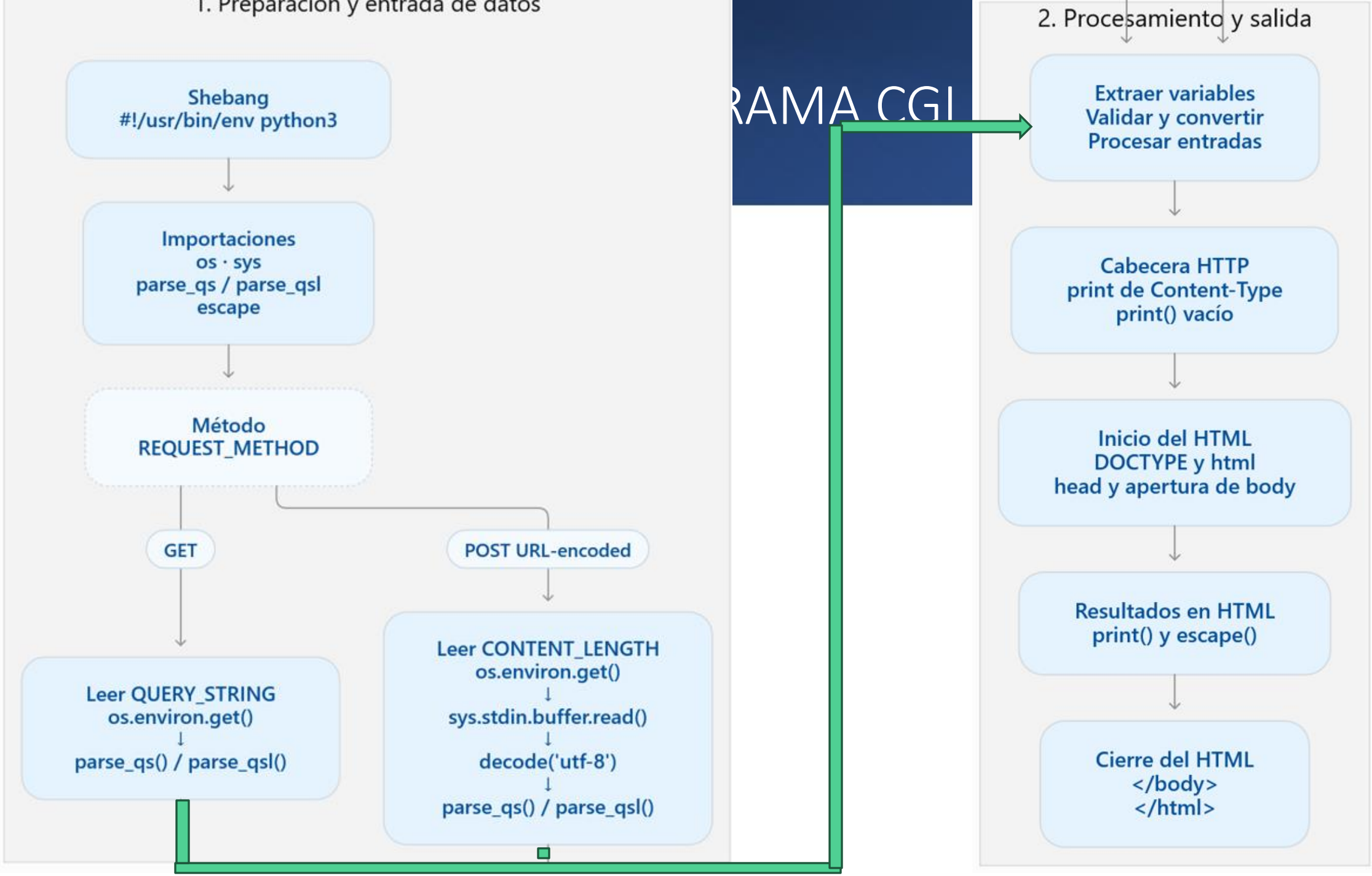




stdout es un flujo de salida estándar, no un dispositivo físico.

En Java, System.out.println() utiliza System.out: la idea es equivalente.

RAMA CGI


1. Preparación y entrada de datos
Shebang
#!/usr/bin/env python3
Importaciones
os · sys
parse_qs / parse_qsl
escape
Método
REQUEST_METHOD
GET
POST URL-encoded
Leer QUERY_STRING
os.environ.get()
↓
parse_qs() / parse_qsl()
Leer CONTENT_LENGTH
os.environ.get()
↓
sys.stdin.buffer.read()
↓
decode('utf-8')
↓
parse_qs() / parse_qsl()
2. Procesamiento y salida
Extraer variables
Validar y convertir
Procesar entradas
Cabecera HTTP
print de Content-Type
print() vacío
Inicio del HTML
DOCTYPE y html
head y apertura de body
Resultados en HTML
print() y escape()
Cierre del HTML
</body>
</html>

FORMULARIOS CON CAMPOS VACÍOS

- Un campo de texto o numérico de un formulario que el usuario no rellena, puede llegar como nota= o edad= en la cadena de entrada. Otros campos, como una casilla sin marcar, pueden no aparecer en la petición.
- Supongamos que recibimos esta cadena de entrada:

  Query=“nota=&edad=&%nombre=Fulano”

- Si ejecutamos parámetros=parse_qs(query), nota y edad van a quedar descartados y el resultado va a ser un diccionario en el que sólo quedará el parámetro nombre
- Esto en principio sólo va a ser un problema, generando una excepción en dos contextos:
  - Si el campo es numérico y se va a realizar un cast int(parametros.get("nota")[0] [""])
  - Si el campo se va a acceder usandolo como índice:

```
from urllib.parse import parse_qs
query="nota=&edad=&%nombre=Fulano"
parametros=parse_qs(query)
print(parametros)
#resultado
# {
#     '%nombre': ['Fulano']
# }
#parametros[“nota”] genera excepción
#int(parametros.get("nota", [""])[0])
genera excepcion
```

FORMULARIOS CON CAMPOS VACÍOS

```
from urllib.parse import parse_qs
query="nota=&edad=&%nombre=Fulano"
parametros=parse_qs(query,
keep_blank_values=True)
print(parametros)
#resultado
# {
#    'nota': [‘’],
#    'edad': [‘’],
#    '%nombre': ['Fulano']
# }
```

- Para evitar que los campos vacíos del formulario sean descartados de la cadena de entrada lanzar parse_qs con el parámetro opcional keep_blank_values=True
- No obstante, si se usa parametros.get("nota", [“0"])[0]) con un valor por defecto [“0"], en lugar de parametros[“nota”] [0] o parametros.get("nota", [“"])[0]) , no es necesario usar keep_blank_values=True, porque get no da excepción si nota no existe, y si existe y tiene valor nulo le asigna un valor por defecto [“0”] que el cast puede convertir a 0
- Otra opción es poner en el cast or “0”.
  - `int(parametros.get("nota",)[0])) genera excepción porque evalua int(“”)`
  - `int(parametros.get("nota", [“"])[0])) genera excepción porque evalua int(“”)`
  - `int(parametros.get("nota", [“0”])[0])) NO genera excepción porque evalua int(“0”)`
  - `int(parametros.get("nota")[0]) or “0”) NO genera excepción porque evalua int(“0”)`
- `int(parametros.get("nota" [“"])[0]) or “0”) NO genera excepción porque evalua int(“0”)`

- Desarrolla un programa CGI denominado registro.py que permita registrar los datos de un alumno. Cuando el usuario acceda inicialmente a la página, el programa mostrará un formulario HTML que permita introducir:
  - Nombre del alumno.
  - Edad.
  - Ciclo formativo (DAW, DAM o ASIR).
- Cuando el usuario pulse el botón Enviar, el formulario realizará una petición POST al mismo programa CGI.
- El programa recuperará los datos enviados y generará dinámicamente una nueva página HTML mostrando la información del alumno.
- No es necesario utilizar bases de datos ni conservar los registros.

```python
#!/usr/bin/env python3
import os
import sys
from urllib.parse import parse_qs
from html import escape
# 1. Consultamos el método de la petición HTTP
metodo: str = os.environ.get("REQUEST_METHOD", "GET")
# 2. Generamos las cabeceras de la respuesta CGI
print("Content-Type: text/html; charset=utf-8")
print()
print("<!DOCTYPE html>")
print("<html lang='es'>")
print("<head>")
print("<meta charset='UTF-8'>")
print("<title>Registro de alumnos</title>")
print("</head>")
print("<body>")
# 3. Si la petición es GET, mostramos el formulario
if metodo == "GET":
    print("<h1>Registro de alumnos</h1>")
    print("""
    <form action="/cgi-bin/registro.py" method="post">
        <label for="nombre">Nombre:</label>
        <input type="text" id="nombre" name="nombre" required>
        <br><br>
        <label for="edad">Edad:</label>
        <input type="number" id="edad" name="edad" min="1" required>
        <br><br>
        <label for="ciclo">Ciclo formativo:</label>
        <select id="ciclo" name="ciclo">
            <option value="DAW">DAW</option>
            <option value="DAM">DAM</option>
            <option value="ASIR">ASIR</option>
        </select>
        <br><br>
        <button type="submit">Registrar alumno</button>
    </form>
    """)
# 4. Si la petición es POST, procesamos sus datos
elif metodo == "POST":
    # Comprobamos el formato del cuerpo HTTP
    tipo: str = os.environ.get("CONTENT_TYPE", "")
    if tipo.split(";")[0].strip().lower() != "application/x-www-form-urlencoded":
        print("<h1>Error: formato de datos no admitido</h1>")
    else:
        # Recuperamos el tamaño del cuerpo HTTP
        longitud: int = int(os.environ.get("CONTENT_LENGTH", "0") or "0")
        if longitud <= 0 or longitud > 4096:
            print("<h1>Error: tamaño de petición no válido</h1>")
        else:
            # Leemos los bytes del cuerpo HTTP
            datos: bytes = sys.stdin.buffer.read(longitud)
            # Convertimos los bytes en texto
            texto: str = datos.decode("utf-8")
            # Interpretamos los parámetros del formulario
            parametros: dict[str, list[str]] = parse_qs(texto)
            # Recuperamos sus valores
            nombre: str = parametros.get("nombre", [""])[0]
            edad_texto: str = parametros.get("edad", [""])[0]
            ciclo: str = parametros.get("ciclo", [""])[0]
            # Validación básica de los datos
            if (not nombre.strip()
                or not edad_texto.isdecimal()
                or int(edad_texto) < 1
                or ciclo not in ("DAW", "DAM", "ASIR")):
                print("<h1>Error: datos del formulario no válidos</h1>")
            else:
                edad: int = int(edad_texto)
                # Generamos dinámicamente la respuesta HTML
                print("<h1>Alumno registrado correctamente</h1>")
                print("<h2>Datos recibidos:</h2>")
                print("<ul>")
                print(f"<li>Nombre: {escape(nombre)}</li>")
                print(f"<li>Edad: {edad}</li>")
                print(f"<li>Ciclo: {escape(ciclo)}</li>")
                print("</ul>")
                print('<a href="/cgi-bin/registro.py">Nuevo registro</a>')
else:
    print("<h1>Método HTTP no admitido</h1>")
print("</body>")
print("</html>")
```

ENTRADA CGI DE DATOS CON SELECCIÓN DE METODO

- En el programa mostrado a continuación se selecciona el método de entrada de manera que es posible capturar la entrada independientemente del método

```
import os
import sys
from urllib.parse import parse_qs


metodo = os.environ.get("REQUEST_METHOD", "GET")


if metodo == "POST":
    longitud = int(os.environ.get("CONTENT_LENGTH") or "0")
    cuerpo = sys.stdin.buffer.read(longitud).decode("utf-8")
    parametros = parse_qs(cuerpo)
else:
    parametros = parse_qs(os.environ.get("QUERY_STRING", ""))


nombre = parametros.get("nombre", ["Invitado"])[0]
html = plantilla.render(nombre=nombre)
```

URLLIB.PARSE.URLSPLIT

- urllib.parse.urlsplit(*urlstring*): devuelve un objeto named Tuple, que es una tupla que puede ser accedida tanto por nombre como por índice. Los nombres de los elementos de la tupla coinciden con los de una URL
- Una url típica tiene estos componentes "scheme://netloc/path?query#fragment“, por ejemplo: http://docs.python.org:80/3/library/urllib.parse.html?highlight=params#url-parsing
- Ejempolo de uso:

scheme	0	URL scheme specifier
netloc	1	Network location part
path	2	Hierarchical path
query	3	Query component
fragment	4	Fragment identifier
username		User name
password		Password
hostname		Host name (lower case)
port		Port number as integer, if present

```
from urllib.parse import urlsplit
urlsplit("scheme://netloc/path?query#fragment") # SplitResult(scheme='scheme', netloc='netloc', path='/path',query='query',
fragment='fragment')
o = urlsplit("http://docs.python.org:80/3/library/urllib.parse.html?highlight=params#url-parsing")
print(o.scheme) #'http'
print(o.netloc) #'docs.python.org:80'
print(o.hostname)#'docs.python.org'
print(o.port) #80
```

ESCAPE() Y EL PROBLEMA DEL XSS

```python
from urllib.parse import parse_qs
from html import escape


# 1. Recuperamos la variable de entorno proporcionada por CGI
query = os.environ.get("QUERY_STRING", "")


# 2. Interpretamos sus parámetros
parametros = parse_qs(query)


# 3. Obtenemos el valor del parámetro nombre
nombre = parametros.get("nombre", ["Invitado"])[0]


# 4. Generamos el documento HTML
print("Content-Type: text/html; charset=utf-8")
print()


print("<html>")
print("<body>")
print(f"<h1>Hola, {escape(nombre)}</h1>")
print("</body>")
print("</html>")
```

- La funcion escape() impide que los caracteres especiales introducidos por el usuario sean interpretados como HTML.
- EScape es una función del módulo estándar *html* de Python que permite convertir determinados caracteres especiales en sus entidades HTML equivalentes.
- Por ejemplo

  nombre = "Felipe“

- se transformaría en

  Felipe

- Gracias a escape, el navegador no interpreta como una etiqueta HTML, sino como texto normal en negrita y pone los caracteres de formato equivalentes
- La utilizamos por motivos de seguridad: estamos incorporando a un documento HTML un dato que procede del usuario y, por tanto, no sabemos qué contiene.
- Es decir la petición GET /cgi-bin/saludo.py?nombre=Fulano la hace el usuario. En este caso ha puesto nombre=Fulano, pero podría haber puesto elementos dañinos generando un ataque por XSS

XSS (CROSS-SITE SCRIPTING).

- Imagina que el usuario lanza esta petición y no estamos usando escape({nombre})

```
GET /cgi-bin/saludo.py?nombre=<script>alert('Hola')</script>

print(f"<h1>Hola, {nombre}</h1>")
```

- Ahora la variable nombre se va a cargar con

```
nombre=<script>alert('Hola')</script>
```

- Y por tanto el html resultante incluirá un script en JavaScript, que en este caso no es dañino ya que es un simple alert (una ventana emergente) pero que podría ser algo mucho peor

```
print(f"<h1>Hola, <script>alert('Hola')</script> </h1>")
```

- El atacante podría pasar esta URL a la victima, camuflada como un enlace a una foto o a un fichero compartido, de manera que cuando la victima la use en su navegador se ejecute el script malicioso en su equipo,

XSS (CROSS-SITE SCRIPTING).

- En cambio, si usamos escape({nombre})

```
print(f"<h1>Hola, {escape(nombre)}</h1>")
```

- <script>alert('Hola')</script> se transformará en

```
<h1>Hola, &lt;script&gt;alert(&#x27;Hola&#x27;)&lt;/script&gt;</h1>
```

- Y el script en lugar de ejecutase se mostrará como texto en la pantalla de la víctima

```
print(f"<h1>Hola, <script>alert('Hola')</script> </h1>")
```

URLENCODE

- Urlencode() es una funcion del modulo urllib.parse, que viene a hacer lo contrario de parse_qs(), es decir construir correctamente una cadena de parámetros URL.

```
from urllib.parse import urlencode

parametros = {
    "nombre": "José García",
    "edad": 25
}

query = urlencode(parametros)
url = "/cgi-bin/saludo.py?" + query
print(query)
# imprimirá nombre=Jos%C3%A9+Garc%C3%ADa&edad=25
```

- Fíjate como concatenando la url con el resultado de Urlencode() a partir de un diccionario podemos generar la url que necesitamos para lanzar la petición

QUOTE /UNQUOTE Y PERCENT ENCODING

- Las funciones quote() y unquote() del módulo estándar urllib.parse de Python sirven para codificar y decodificar caracteres que pueden aparecer en una URL mediante un mecanismo denominado percent-encoding (codificación porcentual).
- Por ejemplo imagina que la respuesta incluye el & y el ?, la URL quedaría:

  https://localhost/cgi-bin/saludo.py?pelicula:=Bonny&Clade

- El parámetro introducido en el formulario incluye un & que es a su vez el elemento separador de campos del fomulario
- Otro ejemplo de uso Imagina que necesitamos enviar al servidor un nombre contiene acentos

  https://localhost/cgi-bin/saludo.py?nombre=José García

- Cuando el formulario genera la cadena sabe como resolver este tipo de situaciones, y parse_qs, es capaz de parsear la cadena teniendo encuenta la sintaxis generada por el diccionario. Por tanto este problema NO LO VAMOS A TENER EN LA ENTRADA DE DATOS DESDE FORMULARI
- El problema se producirá si mi aplicación necesita generar una URL. En ese caso es necesario usar quote() unquote() para representar la cadena por medio del percent Encoding

PERCENT ENCODING

Carácter	Codificación en URL
Espacio	%20
á	%C3%A1
é	%C3%A9
ñ	%C3%B1
&	%26
=	%3D
?	%3F
/	%2F

- La función quote() transforma una cadena con caracteres inaceptables en una Url en su equivalente Percent encoding que sí puede ponerse en una cadena url

```
from urllib.parse import quote

nombre = "José García"
nombre_codificado = quote(nombre)
print(nombre_codificado) #Jos%C3%A9+Garc%C3%ADa&edad=25
url = "/cgi-bin/saludo.py?nombre=" + quote(nombre) # /cgi-bin/saludo.py?nombre=Jos%C3%A9%20Garc%C3%ADa
```

- La función unquote() permite recuperar el texto original a partir de una cadena codificada mediante percent-encoding.

```
from urllib.parse import quote, unquote

original = "España & Portugal"

codificado = quote(original9)#spa%C3%B1a%20%26%20Portugal

decodificado = unquote(codificado)#España & Portugal
```

DONDE DEJAR LOS FORMULARIOS

- En la configuración actual de nuestro setup, el servidor de paginas web abre ficheros estáticos en:
- C:\2DAWdockers\src\public
- Por tanto para los formularios crear una carpeta forms dentro de esta carpeta y dejar los formularios ahí . Por ejemplo:(C:\2DAWdockers\src\public\forms\form1.html
- Para acceder al formulario poner en el navegador la url http://localhost:8080/form/form1.html

Codigo incrustado Mako usando CGI

- El código incrustado plantea una estrategia diferente respecto a la generación dinámica: partimos de un documento fundamentalmente HTML y colocamos determinadas instrucciones de servidor en los puntos donde necesitamos introducir contenido dinámico.
- **No es posible incrustar código Python**, pero si PHP y Java

Ejemplo PHP incrustado

```
<h1>Catálogo de productos</h1>


<ul>
    <?php
        $productos = ["Libro", "Ratón", "Teclado"];
       foreach ($productos as $producto) {
            echo "<li>" . htmlspecialchars($producto,
ENT_QUOTES, 'UTF-8') . "</li>";
        }
    ?>
</ul>
```

PORQUÉ CODIGO INCRUSTADO

- **La ventaja del código incrustado frente a la generación de HTML desde Python:** El código incrustado permite aprovechar el diseño de una página HTML ya existente, por compleja que sea, e incorporar instrucciones de programación únicamente en aquellos lugares donde se necesite generar contenido dinámico.
- De esta manera, **no es necesario construir toda la estructura HTML mediante instrucciones print(),** como ocurre cuando generamos directamente la página desde un programa CGI.
- **Esto simplifica considerablemente el desarrollo de páginas web complejas**, facilita su mantenimiento y permite separar mejor el diseño de la página de la lógica

LENGUAJES CON CÓDIGO INCRUSTADO

- Se puede emplear código incrustado con diferentes lenguajes

```
</>PHP
<h1>
    <?php echo "Hola mundo"; ?>
</h1>
```

```
</> JAVA -JSP

<h1>
    <% out.print("Hola mundo"); %>
</h1>
```

```
</>  C# (ASP.NET Web Forms)

<h1>
    <% Response.Write("Hola mundo"); %>
</h1>
```

- Desgraciadamente **Python no es uno de ellos**. No al menos de forma nativa. Pero instalando herramientas como Mako, se puede lograr algo parecido.
- En general **para que el servidor web acepte código incrustado necesita tener instalado un componente o extensión** de ese lenguaje, que haga la función de servidor de aplicaciones, y por supuesto el interprete, compilador o máquina virtual de ese lenguaje

```
</> RUBI

<h1>
    <%= "Hola mundo" %>
</h1>
```

MARCADORES DELIMITADORES DE CÓDIGO INCRUSTADO

- **El código incrustado empieza con el marcador, *<%* , y finaliza con *%>***. La excepción es PHP que usa <?php ... ?> y JavaScript que usa <script> ... </script>

Lenguaje / tecnología	Marcador de apertura	Marcador de cierre	Extensión habitual
PHP	<?php	?>	.php
JavaScript	<script>	</script>	.js
Java (JSP)	<%	%>	.jsp
C# (ASP.NET Web Forms)	<%	%>	.aspx
VBScript (ASP clásico)	<%	%>	.asp
Ruby (ERB)	<%	%>	.erb
Python (Mako)	<%	%>	.mako

MARCADORES DE EXPRESIÓN

- Muchas tecnologías tienen un segundo **marcador** que sirve específicamente **para evaluar una expresión** e insertar su resultado en el documento.

ecnología	Marcador	Ejemplo
PHP	<?= ... ?>	<?= $nombre ?>
Java (JSP)	<%= ... %>	<%= nombre %>
ASP.NET Web Forms	<%= ... %>	<%= nombre %>
Ruby (ERB)	<%= ... %>	<%= nombre %>
Python (Mako)	${ ... }	${nombre}

```
</> JAVA -JSP
<%
    String nombre = "Fulano";
    int edad = 25;
%>

<h1>Hola, <%= nombre %></h1>
<p>El año que viene tendrás <%= edad
+ 1 %> años.</p>
```

```
</> C# (ASP.NET Web Forms)
<%
    nombre = "Fulano"
    edad = 25
%>

<h1>Hola, <%= nombre %></h1>
<p>El año que viene tendrás <%= edad + 1 %> años.</p>
```

```
</> RUBI -ERB
<%
    nombre = "Fulano"
    edad = 25
%>

<h1>Hola, <%= nombre %></h1>
<p>El año que viene tendrás <%= edad + 1 %> años.</p>
```

CODIGO INCRUSTADO PYTHON CON MOTORES DE PLANTILLAS MAKO

- Mako es **un motor de plantillas** para Python que **permite incrustar instrucciones Python directamente dentro de un documento HTML,** de una forma bastante parecida a JSP con Java o ERB con Ruby.
- **Mako permite utilizar código Python dentro de sus bloques <% %>.**
- **Mako no forma parte de Python.** Es una biblioteca externa que debemos instalar. En nuestro setup esta instalado en el contenedor cgi

```
from mako.template import Template

plantilla = Template(filename="saludo.html")

resultado = plantilla.render()

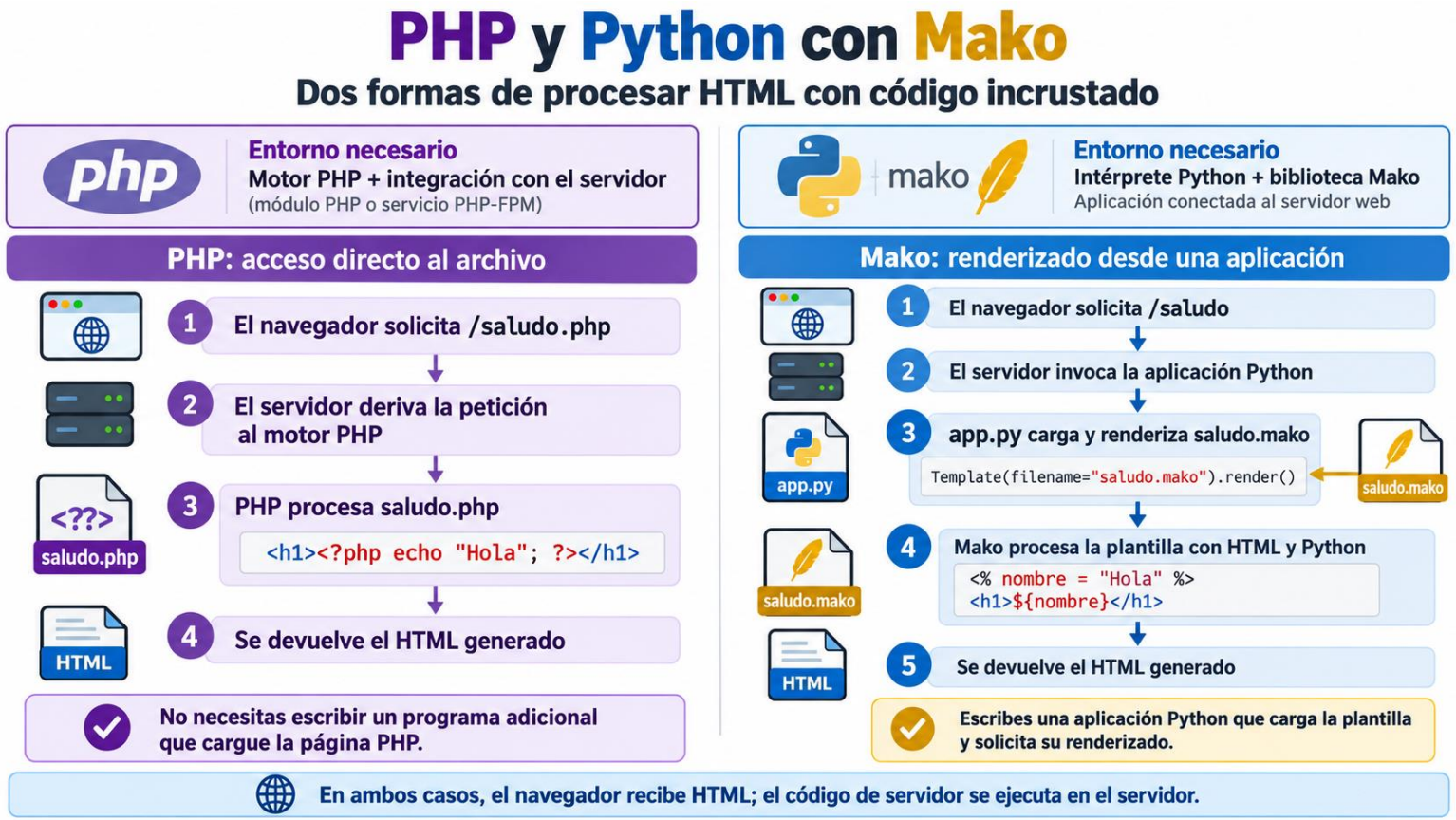
print(resultado)
```

Para procesar una plantilla desde Python mediante:

```
pip install mako
```

MODELO MAKO

- Mako permite crear páginas html con código Python incrustado, **pero dichas páginas no pueden ejecutarse directamente como ocurre con otros lenguajes como PHP**, sino que tiene que ser renderizada por otro programa Python antes de enviarse al servidor



- Técnicamente, **Mako no genera código incrustado ejecutable**, ya que necesita un programa Python que renderiza la plantilla Mako. Es este proceso de renderización lo que genera el HTML que se envia al usuario
- Pero Mako, no es simplemente un mecanismo para **incrustar variables en un template**, Mako permite bloques de código con funciones predefinidas, instrucciones condicionales e iterativas usando un lenguaje Python y un pseudolenguaje derivado de Python.
- No obstante, siempre hará falta un programa Python que lance la renderización de la plantilla, porque esta no se ejecuta por si sola al abrirla el navegador. El programa Python procesa la plantilla y genera el HTML
  - carga la plantilla con extensión mako
  - Identifica el fichero HTML con el código incrustado;
  - genera el HTML final. Mediante el método render

```
from mako.template import Template
plantilla = Template(filename="productos.mako")
html = plantilla.render()
print(html)
```

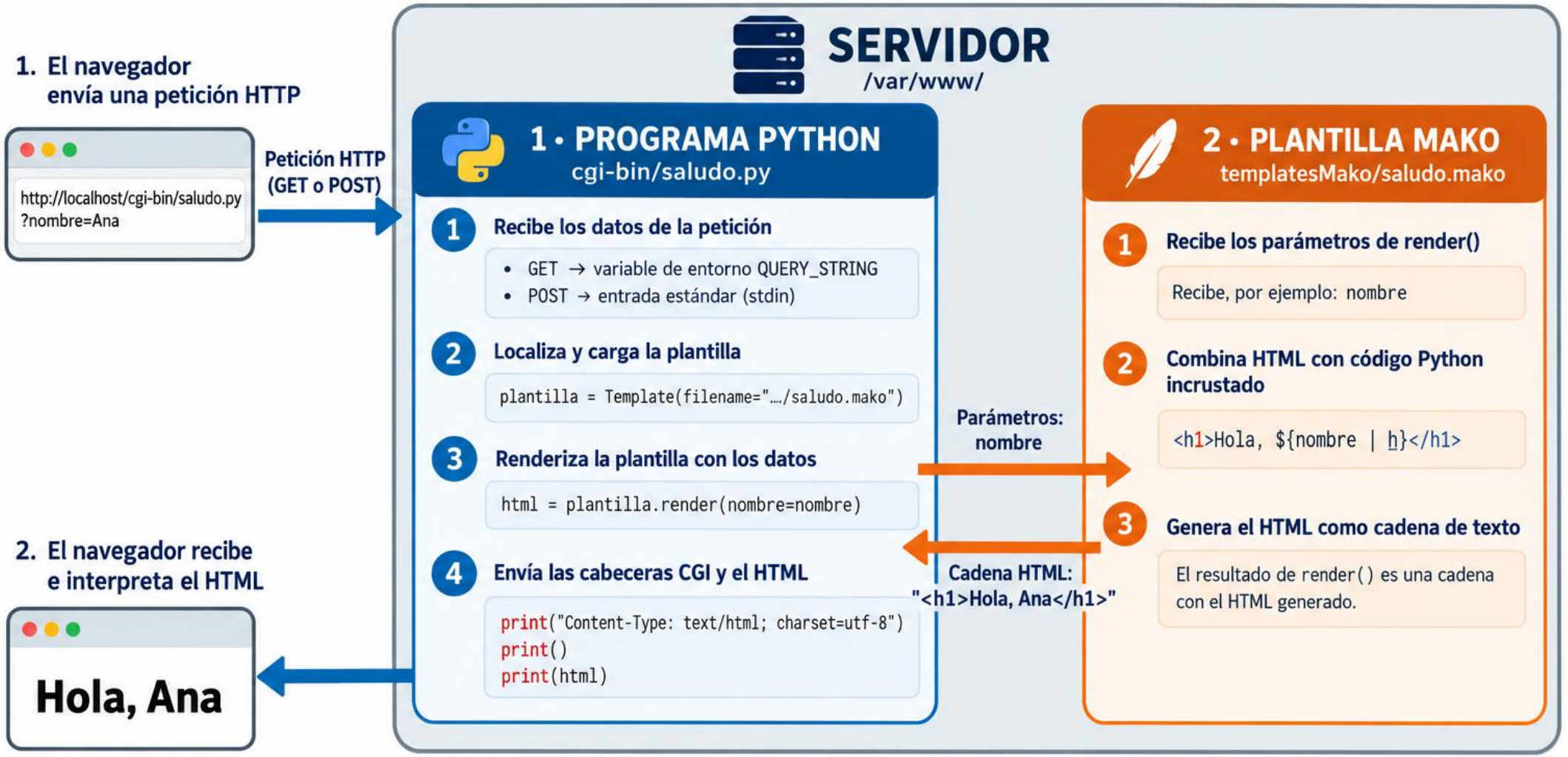
MAKO ES CÓDIGO INCRUSTADO POR PLANTILLAS

- Recuerda, en el uso de Mako hacen falta dos programas. Lo que se cuenta a continuación explica las funciones de los dos elementos usando CGI como elemento de procesamiento
  - Un programa lanzador Python en ./cgi-bin que:
    - Es lo que el usuario abre por medio de la petición url, por ejemplo http://localhost:8080/cgi-bin/compra.py
    - En CGI Lee las variables entorno de entrada que se hayan podido transmitir mediante un get o un post. En WSGI lee el objeto environ
    - Define la ubicación de la plantilla
    - Lanza un método render, con el que es posible pasar los parámetros de entrada, y que da como resultado una cadaen con el html resultante
    - Hace un print de la cadena “html” que genera la ejecución de la plantilla
  - Una plantilla Mako, en ./TemplatesMako
    - Es html, con código incrustado
    - Genera el fichero HTML virtual en la forma de una cadena de texto que se devuelve al método render



MAKO CON CGI: DE LA PETICIÓN AL HTML

Dos archivos que trabajan juntos: un programa Python y una plantilla Mako.



El HTML se genera en memoria: no es necesario guardar un archivo .html.

MARCADORES USADOS POR MAKO

Ejecutar un bloque de código Python	<% ... %>	<% edad = 25 %>
Evaluar una expresión	${ ... }	${nombre}
Declarar funciones o miembros Python del módulo generado	<%! ... %>	<%! import math %>
Estructuras de control	%if..%endif, %for %endfor, %while %endwhile etc.	% for p in productos:

```
<html>
<body>
    <%
        nombre = "Fulano"
        edad = 25
    %>

    <h1>Hola, ${nombre}</h1>
    <p>El año que viene tendrás ${edad + 1} años.</p>

</body>
</html>
```

El motor Mako procesa el documento, ejecuta las instrucciones Python y sustituye las expresiones ${...} por sus resultados.

MAKO –ESCAPADO HTML

- Otro ejemplo:

El modificador | h aplica escapado HTML, (sustituir determinados caracteres especiales por sus correspondientes entidades HTML, para que el navegador los interprete como texto y no como parte del código HTML.) algo importante cuando mostramos datos externos.
En Mako, el escapado HTML no viene activado por defecto en las expresiones.

```
<h1>Productos disponibles</h1>


<%
    productos = ["Libro", "Ratón", "Teclado"]
%>


<ul>
% for producto in productos:
    <li>${producto | h}</li>
% endfor
</ul>
```

Ojo! En la definición de la plantilla se puede establecer un parámetro default_filter que establece el escapado para todo el documento haciendo innecesario el modifidor | h

```
from mako.template import Template


plantilla = Template(
    filename="/app/templatesMako/saludo.mako",
    default_filters=["h"]
)
```

ESQUEMA DE UN PROGRAMA LANZADOR DE PLANTILLAS MAKO CON CGI




```python
#!/usr/bin/env python3    #1 shwbNC
import os
import sys
from urllib.parse import parse_qs
from mako.template import Template

# Valores iniciales
precio = ""
cantidad = ""
total = None
error = ""

# Recibir y procesar los datos del formulario
if os.environ.get("REQUEST_METHOD", "GET") == "POST":
    longitud = int(os.environ.get("CONTENT_LENGTH") or "0")
    cuerpo = sys.stdin.buffer.read(longitud).decode("utf-8")
    parametros = parse_qs(cuerpo)
    precio = parametros.get("precio", [""])[0]
    cantidad = parametros.get("cantidad", [""])[0]
    try:
        precio_numero = float(precio)
        cantidad_numero = int(cantidad)
        if precio_numero <= 0 or cantidad_numero <= 0:
            error = "El precio y la cantidad deben ser mayores que cero."
        else:
            total = precio_numero * cantidad_numero
    except ValueError:
        error = "Introduce un precio válido y una cantidad entera."

# Cargar la plantilla
plantilla = Template(
    filename="/app/templatesMako/compra.mako",
    input_encoding="utf-8",
    default_filters=["h"]
)

# Pasar los datos a la plantilla y generar el HTML
html = plantilla.render(
    precio=precio,
    cantidad=cantidad,
    total=total,
    error=error
)

# Crear cabecera mensaje http
print("Content-Type: text/html; charset=utf-8")
print()

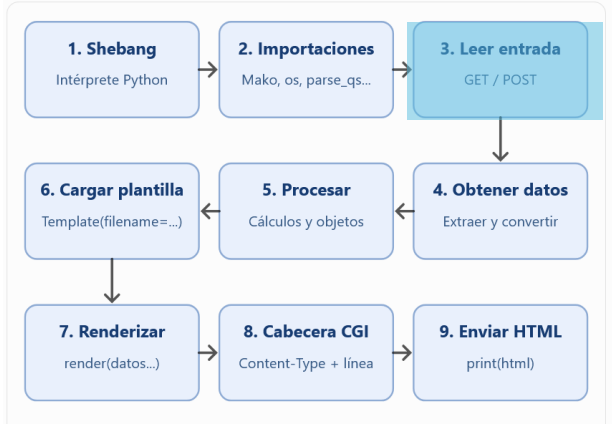
print(html)
```

1. Shebang
2. Importaciones
3. Leer entrada
4. Obtener datos
5. Procesar
6. Cargar plantilla
7. Renderizar
8. Cabecera CGI
9. Enviar HTML

MAKO CON CGI - ENTRADA VARIABLES AMBIENTE

- El programa Python es el encargado de recoger los parámetros de entrada
- El procedimiento es el usado genéricamente con CGI. Los datos de entrada se pasan por medio de variables de ambiente que se recogen con os.environ.get(os.environ.get("QUERY_STRING", "")
  - METODO GET

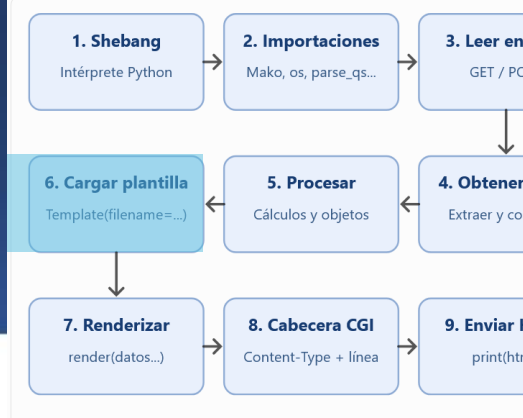
```
parametros = parse_qs(os.environ.get("QUERY_STRING", ""))
```



  - METODO POST (con codificación application/x-www-form-urlencoded)

```
longitud = int(os.environ.get("CONTENT_LENGTH") or "0")
cuerpo = sys.stdin.buffer.read(longitud).decode("utf-8")
parametros = parse_qs(cuerpo)
```

LOCALIZACIÓN DE LA PLANTILLA CON TEMPLATE



- **Template es una clase de Mako que permite crear un objeto que representa una plantilla.** Cuando escribimos Template( ), estamos llamando a su constructor.
- Es neceario importarla

  from mako.template import Template
- Tiene tres campos: filename, input_coding, y default_filters. Es imprescindible indicar el campo filename, el resto son opcionales

Parámetro	Función
filename	Ruta del archivo de la plantilla.
input_encoding	Codificación utilizada para leer ese archivo.
default_filters=["h"]	Aplica escapado HTML por defecto a las expresiones. Importante destacar que si ponemos este filtro, el modificador h se vuelve redundante y es inneecsarioen expresiones como `<li>${producto | h}</li>`

```
plantilla = Template(
    filename="/app/templatesMako/saludo.mako",
    input_encoding="utf-8",
    default_filters=["h"]
)
```

MAKO CON CGI PASO DE PARÁMETROS CON RENDER

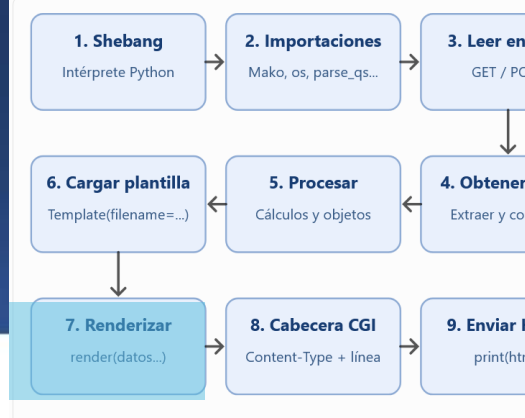
- El programa Python, pasa a la plantilla Mako los parámetros como argumentos dentro del método render del objeto Template. Se pueden pasar múltiples parámetros, tantos como sea necesario. Es imprescindible que cada parámetro tenga un nombre, o sea tienen que ser pares clave=valor. Por ejemplo:

```
html = plantilla.render(nombre="Ana", edad=25)
```

- En la plantilla Mako deben existir plantillas para cada uno de los argumentos. Los parámetros pasados por render() están disponibles en la plantilla sin que sea necesario declarar nada. Por ejemplo:

```
<h1>Hola, ${nombre}</h1>
<p>Tienes ${edad} años.</p>
```

MAKO CON CGI PASO DE PARÁMETROS CON RENDER



- Se pueden pasar además colecciones de argumentos, usando listas y diccionarios

```
html = plantilla.render(
    nombre="Ana",
    productos=["Libro", "Ratón", "Teclado"]
)
```

- En este caso la plantilla tiene que usar un bucle for para recorrer la colección y visualizar los datos

```
<h1>Productos de ${nombre}</h1>
<ul>
% for producto in productos:
    <li>${producto}</li>
% endfor
</ul>
```

MAKO CON CGI PASO DE PARÁMETROS CON RENDER

- Incluso se pueden pasar objetos de una clase y listas de objetos de una clase

```
class Libro:
    def __init__(self, titulo:str, autor:str, precio:float, stock: int):
        self.titulo=titulo
        self.autor=autor
        self.precio=precio
        self.stock=stock

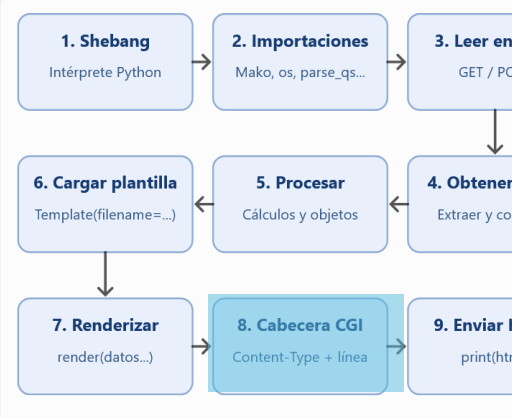
def main():
    lon=int(os.environ.get("CONTENT_LENGTH","0") or "0")
    query=sys.stdin.buffer.read(lon).decode("utf-8")
    parametros=parse_qs(query)

    libros: list[Libro]=[]
    #procesar parametros en esta caso van a ser listas de valores
    libros.append(Libro("Don Quijote","Cervantes",100.0,16"))
    libros.append(Libro("Tirant","Cervantes",1200.0,11"))

    plantilla=Template(filename="/app/templatesMako/libros.mako")

    html=plantilla.render(listaLibros=libros)
```

RESPUESTA Y CABECERA



- Como en el caso de la generación dinámica es necesario establecer la cabecera del mensaje http. Opcionalmente también se puede establecer un estado de la respuesta

```
# generar cabecera
print("Content-Type: text/html; charset=utf-8")
print(f"Content-Length: {len(cuerpo)}")
print() # línea de separación con el cuerpo
```

```
#MINIMO OBLIGATORIO PARA LA CABECERA
# cabecera
print("Content-Type: text/html; charset=utf-8")
print() # generar una línea de separación
```

```
#generar estado respueta
print("Status: 200 OK")
# generar cabecera
print("Content-Type: text/html; charset=utf-8")
print(f"Content-Length: {len(cuerpo)}")
print() # línea de separación con el cuerpo
```

SALIDA DE DATOS CON MAKO

- La plantilla Mako no genera directamente un fichero html que se envie al usuario.
  - Recuerda que estamos usando CGI. Necesitamos una cadena de texto que podamos imprimir por print(cadena) .
  - En CGI este print esta conectado a una tubería con el servidor web, el cual lo convierte en respuestas HTTP para el usuario
  - En Mako el render de la plantilla devuelve una cadena de texto, que el programa lanzador en Python debe imprimir con print

```
html = plantilla.render(
    nombre="Ana",
    productos=["Libro", "Ratón", "Teclado"]
)
print(html)
```

```
<h1>Productos de ${nombre}</h1>
<ul>
% for producto in productos:
    <li>${producto}</li>
% endfor
</ul>
```

PLANTILLAS MAKO

- Hay cuatro formas de integrar código en Mako

  Incrustar una variable o expresión en el html con ${...}

  Insertar un bloque completo de código Python (usando <% ..%>

  Poner bloques if o for mako de una sola sentencia individual usando %.

  - Ojo es un único % al principio de la instrucción, no hay que cerrarlo con un segundo %
  - Estos %if y %for usan sintaxis Mako, no Python
  - Else también lleva %

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>${titulo}</title>
</head>
<body>
    <h1>${titulo}</h1>
    <%
        total_productos = len(productos)
        disponibles = 0
        for producto in productos:
            if producto["stock"] > 0:
                disponibles += 1
    %>
    <table border="1">
        <tr>
            <th>Producto</th>
            <th>Precio</th>
            <th>Stock</th>
            <th>Estado</th>
        </tr>
        % for producto in productos:
            <tr>
                <td>${producto["nombre"]}</td>
                <td>${producto["precio"]} €</td>
                <td>${producto["stock"]}</td>
                <td>
                    % if producto["stock"] > 0:
                        Disponible
                    % else:
                        Agotado
                    % endif
                </td>
            </tr>
        % endfor
    </table>
```

PLANTILLAS MAKO

- Hay una cuarta forma: generando dinámicamente html, usando un bloque <% ..%>, y el método content.write(“”)
- content.write(“”) genera cadenas de texto que se convierten en html de la páginaweb

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>${titulo}</title>
</head>
<body>
    <h1>${titulo}</h1>
   <%
        total_productos = len(productos)
        disponibles = 0
        for producto in productos:
            if producto["stock"] > 0:
                disponibles += 1
    %>
    <table border="1">
        <tr>
            <th>Producto</th>
            <th>Precio</th>
            <th>Stock</th>
            <th>Estado</th>
        </tr>
        <%
             for producto in productos:
                context.write("<tr>")
                context.write(f'<td>{str(producto["nombre"]))</td>')
                context.write(f'<td>{str(producto["precio"])} €</td>')
                context.write(f'<td>{str(producto["stock"])}</td>')
                if producto["stock"] > 0:
                    context.write("<td>Disponible</td>")
                else:
                    context.write("<td>Agotado</td>")


                context.write("</tr>")
        %>
    </table>
</body>
```

ITERATIVAS Y CONDICIONALES % CON SINTAXIS MAKO

- Dentro de los bloques <%..%> podemos introducir código Python como el que ya conoces. Esta solución es mucho más comoda y legible que los bloques content.write()º
- Pero los bloques % introducen una única sentencia condicional o iterativa de sintaxis mako
- La sintaxis mako es casi idéntica a la sintaxis Python salvo que es necesario poner un %endif, o %endfor para finalizar la expresión

%if / %elif / %else: - %endif

```
% if stock > 5:
    <p>Stock suficiente.</p>
% elif stock > 0:
    <p>Quedan pocas unidades.</p>
% else:
    <p>Agotado.</p>
% endif
```

- %elif y % else se puede omitir
- Como en Python usar dos puntos tras la sentencia
- a diferencia de Python hay un %endif

%For %endfor

```
<ul>
% for numero in range(1, 11):
    <li>${numero}</li>
% endfor
</ul>


<ul>
% for producto in productos:
    <li>${producto["nombre"]}</li>
% endfor
</ul>
```

- Salvo por el uso de %endfor la sintaxis es como en Python

%While %endwhile

```
<% contador = 1 %>


% while contador <= 3:
    <p>Repetición ${contador}</p>
    <% contador += 1 %>
% endwhile
```

- Salvo por el uso de %endwhile la sintaxis es como en Python

- **Las directivas de Mako deben comenzar en una línea propia**. Pueden tener espacios delante, pero no una etiqueta como `<td>`. Por ejemplo:

<td>%if l.estaDisponible(): #incorrecto

- No será reconocido como un directiva %if,

<td>

%if l.estaDisponible(): #correcto

DEFINICIONES DE DATOS

- **Las listas o diccionarios** que sea necesario usar en un template **se definen dentro de un bloque <% ... %>,** porque su definición es una asignación de Python.
- **Todas las variables definidas** dentro de un bloque <% ... %>, **pueden ser utilizarlos en el resto de la plantilla**.
- Las líneas con % se utilizan para las estructuras de control. **No podemos definir en ellas listas o diccionarios**

% numeros = [1, 2, 3, 4, 5] da error

<% numeros = [1, 2, 3, 4, 5] %> correcto

```
<%
    frutas = ["Manzana", "Pera", "Naranja"]

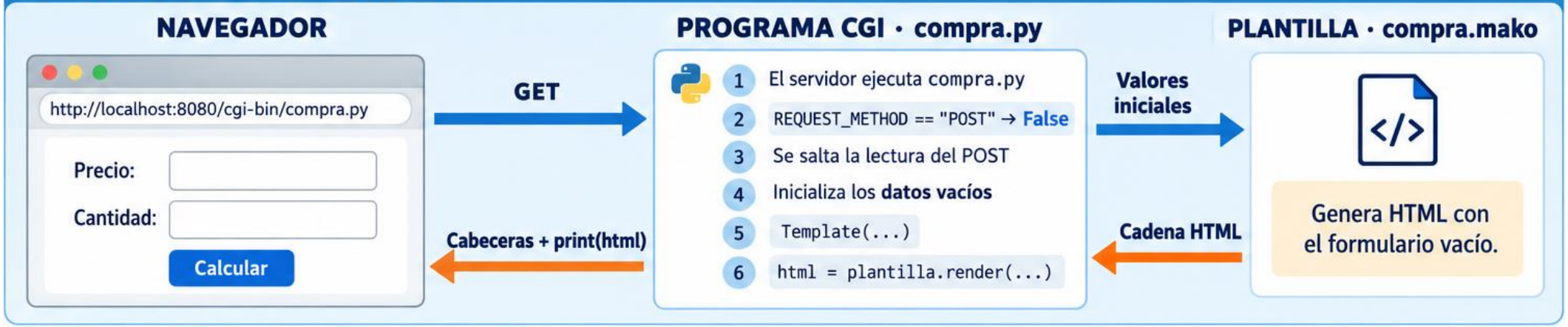

    producto = {
        "nombre": "Teclado",
        "precio": 35.50,
        "stock": 12
    }
%>
<ul>
% for fruta in frutas:
    <li>${fruta}</li>
% endfor
</ul>


<p>Producto: ${producto["nombre"]}</p>
<p>Precio: ${producto["precio"]} €</p>
```

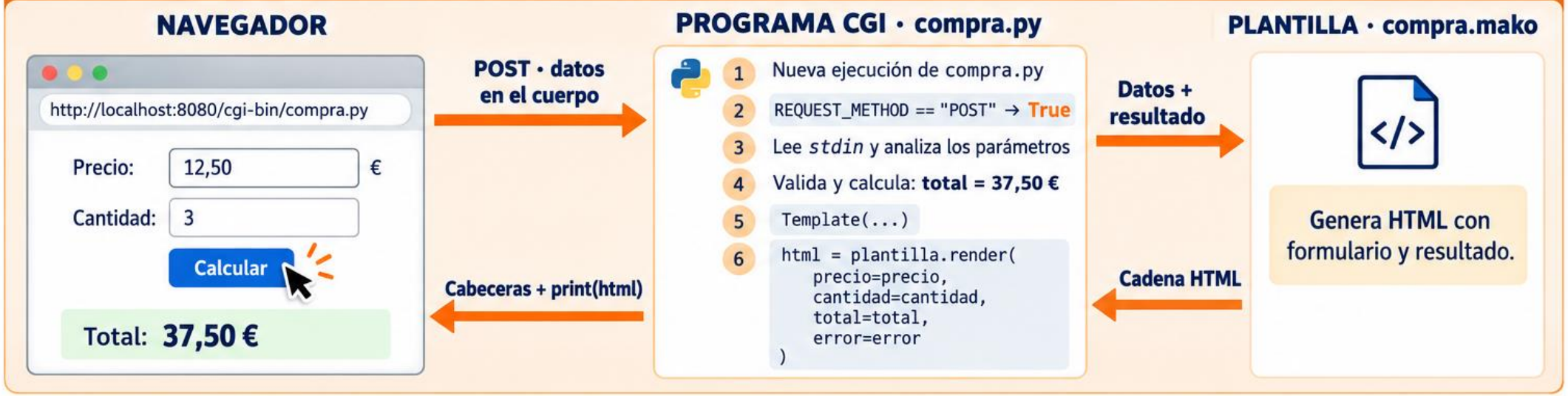
MAKO + CGI: UN FORMULARIO, DOS PETICIONES

El usuario accede siempre a `compra.py`; Python ejecuta la plantilla Mako.

1 · PRIMERA VISITA: GET



2 · ENVÍO DEL FORMULARIO: POST



GET y POST son peticiones independientes.

render() genera texto; el navegador muestra el HTML.

La plantilla .mako se ejecuta en el servidor.

CATALOGO DE PRODUCTOS CON MAKO

- Vamos a utilizar dos archivos:
  - tienda.py: programa Python que prepara los datos y ejecuta el motor Mako.
  - catalogo.html: Plantilla Mako. Es un documento HTML que incorpora instrucciones incrustadas Python mediante los marcadores de Mako.
- El programa primero define los datos de entrada a la plantilla (el diccionario productos)NOTA: EN ESTE EJEMPLO LOS DATOS NO SE RECIBEN NI POR POST NI POR GET, ESTÁN PREDEFINIDOS
- Luego define la ubicación de la plantilla, codificación y filtros. El filtro h establece el escapado HTML predeterminado de las expresiones ${...}.
- render() lanza la plantilla, se le pasan dos parámetros de entrada, el diccionario de productos y un título
- Render devuelte una cadena que es el html generado y que al imprimirse en CGI genera la respuesta al usuario

```python
#!/usr/bin/env python3
from mako.template import Template
# Datos que proporcionaremos a la plantilla
productos = [
    {"nombre": "Teclado", "precio": 35.50, "stock": 12},
    {"nombre": "Ratón", "precio": 18.90, "stock": 0},
    {"nombre": "Monitor", "precio": 159.00, "stock": 5},
    {"nombre": "Auriculares", "precio": 42.00, "stock": 8}
]


# Cargamos el documento que contiene HTML y código incrustado
plantilla = Template(
    filename="catalogo.html",
    input_encoding="utf-8",
    default_filters=["h"]
)


# Procesamos la plantilla proporcionándole los productos
resultado = plantilla.render(
    productos=productos,
    titulo="Catálogo de informática"
)


# Mostramos el documento HTML generado
print(resultado)
```

CATALOGO DE PRODUCTOS PLANTILLA MAKO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>${titulo}</title>
</head>
<body>
    <h1>${titulo}</h1>
    <%
        total_productos = len(productos)
        disponibles = 0
        for producto in productos:
            if producto["stock"] > 0:
                disponibles += 1
    %>
    <table border="1">
        <tr>
            <th>Producto</th>
            <th>Precio</th>
            <th>Stock</th>
            <th>Estado</th>
        </tr>
        % for producto in productos:
            <tr>
                <td>${producto["nombre"]}</td>
                <td>${producto["precio"]} €</td>
                <td>${producto["stock"]}</td>
                <td>
                    % if producto["stock"] > 0:
                        Disponible
                    % else:
                        Agotado
                    % endif
                </td>
            </tr>
        % endfor
    </table>
```

- La plantilla Mako tiene código incrustado Python marcado con marcadores Mako
- En concreto tenemos un bucle for para mostrar en una tabla html los productos
- Y una serie de condicionales para mostrar un texto adicional

```
    <h2>Información adicional</h2>

    % if disponibles == total_productos:
        <p>Todos los productos están disponibles.</p>
    % elif disponibles == 0:
        <p>No hay productos disponibles.</p>
    % else:
        <p>Algunos productos están agotados.</p>
    % endif
</body>
</html>
```

MAKO+CGI+MÉTODO GET

- Programa Planificador: calcula cuantas sesiones de estudio de 30minutos pueden planificarse a partir de los minutos disponibles
- Entradas:
  - Nombre asignatura:
  - Minutos disponibles
- Salida:
  - Número de sesiones de 30m
  - Tiempo sobrante (para repasar)
- Ejemplo: con **100 minutos disponibles y bloques de 30 minutos**, propone tres bloques y deja 10 minutos para repasar.

- En esta versión la entrada es por medio de get

EJEMPLO MAKO-CGI CON FORMULAIRO CALCULADORA PRECIO DE COMPRA

- Se utilizan dos archivos:
  - cgi-bin/compra.py
  - templatesMako/compra.mako
- La clave para entender este programa es entender la secuencia de sucesos:

1. El usuario hace una petición get desde el navegador para abrir http://localhost:8080/cgi-bin/compra.py
2. La línea es muy importante, porque hace que el programa trate de leer el formulario cuando este se recibe por medio de una petición post, algo que no ha ocurrido en el paso anterior, por tanto este bloque se salta y se pasa al bloque cargar plantilla, y luego al render:

```
if os.environ.get("REQUEST_METHOD", "GET") == "POST":
```

3. 3. El render devuelve una cadena con el html que es el formulario en si y compras.py lo imprime, de manera que se muestra en pantalla
4. 4. Cuando el usuario pone valores en el formulario, y se hace clic en el botón, se lanza de nuevo compra.py pero en este caso, la petición sigue el método post
5. 5 ahora si que se leen los parámetros de entrada que vienen en el cuerpo del post porque if os.environ.get("REQUEST_METHOD", "GET") == "POST": se evalua a true
6. 6 se vuelve a cargar el template, incluyendo los valores elegidos por el usuario, se procesa y se devuelve una cadena
7. 7 compras.py imprime la cadena y el servidor web hace que el usuario la visualice

```python
#!/usr/bin/env python3
import os
import sys
from urllib.parse import parse_qs
from mako.template import Template

# Valores iniciales
precio = ""
cantidad = ""
total = None
error = ""

# Recibir y procesar los datos del formulario
if os.environ.get("REQUEST_METHOD", "GET") == "POST":
    longitud = int(os.environ.get("CONTENT_LENGTH") or "0")
    cuerpo = sys.stdin.buffer.read(longitud).decode("utf-8")
    parametros = parse_qs(cuerpo, keep_blank_values=True)
    precio = parametros.get("precio", [""])[0]
    cantidad = parametros.get("cantidad", [""])[0]
    try:
        precio_numero = float(precio)
        cantidad_numero = int(cantidad)
        if precio_numero <= 0 or cantidad_numero <= 0:
            error = "El precio y la cantidad deben ser mayores que cero."
        else:
            total = precio_numero * cantidad_numero
    except ValueError:
        error = "Introduce un precio válido y una cantidad entera."

# Cargar la plantilla
plantilla = Template(
    filename="/app/templatesMako/compra.mako",
    input_encoding="utf-8",
    default_filters=["h"]
)

# Pasar los datos a la plantilla y generar el HTML
html = plantilla.render(
    precio=precio,
    cantidad=cantidad,
    total=total,
    error=error
)

# Enviar la respuesta al navegador
print("Content-Type: text/html; charset=utf-8")
print()
print(html)
```

EJEMPLO PLANIFICADOR LANZADOR PYTHON GET

```python
#!/usr/bin/env python3
import os
from urllib.parse import parse_qs
from mako.template import Template

# Recogemos los parámetros enviados mediante GET
parametros = parse_qs(os.environ.get("QUERY_STRING", ""))

asignatura = parametros.get("asignatura", ["Programación"])[0]
minutos = parametros.get("minutos", ["100"])[0]
duracion = parametros.get("duracion", ["30"])[0]

# Validamos los valores recibidos
error = ""
bloques = 0
resto = 0

try:
    minutos = int(minutos)
    duracion = int(duracion)
    if not (1 <= minutos <= 480 and 1 <= duracion <= 120):
        error = “Minutos entre 1 y 480 y bloques entre 1 y 120."
    else:
        bloques = minutos // duracion
        resto = minutos % duracion
except ValueError:
    error = "Los minutos y la duración deben ser números enteros."
```

Lanzador /app/cgi-bin/plan.py:

```python
# Generamos el HTML con Mako
plantilla = Template(
    filename="/app/templatesMako/plan.mako",
    input_encoding="utf-8",
    default_filters=["h"]
)
resultado = plantilla.render(
    asignatura=asignatura,
    minutos=minutos,
    duracion=duracion,
    bloques=bloques,
    resto=resto,
    error=error
)

print("Content-Type: text/html; charset=utf-8")
print()
print(resultado)
```

EJEMPLO PLANIFICADOR TEMPLATE MAKO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Mi sesión de estudio</title>
</head>
<body>
    <h1>Planificador de estudio</h1>
    <form action="/cgi-bin/plan.py" method="get">
        <p>
            <label>Asignatura:
                <input type="text" name="asignatura"
                       value="${asignatura}" required>
            </label>
        </p>
        <p>
            <label>Minutos disponibles:
                <input type="number" name="minutos"
                       value="${minutos}" min="1" max="480" required>
            </label>
        </p>
        <p>
            <label>Duración de cada bloque:
                <input type="number" name="duracion"
                       value="${duracion}" min="1" max="120" required>
            </label>
        </p>
        <button type="submit">Crear plan</button>
    </form>
```

- Template /app/templatesMako/plan.mako:

```
    % if error:
        <p>${error}</p>
    % else:
        <h2>Tu plan para ${asignatura}</h2>
        % if bloques > 0:
            <ol>
            % for numero in range(1, bloques + 1):
                <li>
                    Bloque ${numero}: estudiar desde el minuto
                    ${(numero - 1) * duracion} hasta el ${numero * duracion}.
                </li>
            % endfor
            </ol>
        % else:
            <p>No tienes tiempo suficiente para un bloque completo.</p>
        % endif
        % if resto > 0:
            <p>Reserva los ${resto} minutos restantes para repasar.</p>
        % endif
    % endif
</body>
</html>
```

```python
#!/usr/bin/env python3
import os
import sys
from urllib.parse import parse_qs
from mako.template import Template


# Leemos los datos enviados en el cuerpo de la petición POST
longitud = int(os.environ.get("CONTENT_LENGTH", "0") or "0")
cuerpo = sys.stdin.buffer.read(longitud).decode("utf-8")
parametros = parse_qs(cuerpo)


# Si no recibimos algún dato, utilizamos un valor predeterminado
asignatura = parametros.get("asignatura", ["Programación"])[0]
minutos = parametros.get("minutos", ["100"])[0]
duracion = parametros.get("duracion", ["30"])[0]


# Validamos los datos y calculamos el plan
error = ""
bloques = 0
resto = 0


try:
    minutos = int(minutos)
    duracion = int(duracion)


    if not (1 <= minutos <= 480 and 1 <= duracion <= 120):
        error = "Introduce minutos entre 1 y 480 y bloques entre 1 y 120."
    else:
        bloques = minutos // duracion
        resto = minutos % duracion
```

MISMO PROGRAMA CON MÉTODO POST

- Sí. **La plantilla cambia a method="post" y Python lee los datos desde la entrada estándar**, en lugar de obtenerlos de QUERY_STRING.
- En **plan.py**, añade sys a los imports y sustituye la lectura de parámetros por:

```python
except ValueError:
    error = "Los minutos y la duración deben ser números enteros."
# Cargamos la plantilla
plantilla = Template(
    filename="/app/templatesMako/plan.mako",
    input_encoding="utf-8",
    default_filters=["h"]
)
# Pasamos los datos a la plantilla para generar el HTML
resultado = plantilla.render(
    asignatura=asignatura,
    minutos=minutos,
    duracion=duracion,
    bloques=bloques,
    resto=resto,
    error=error
)

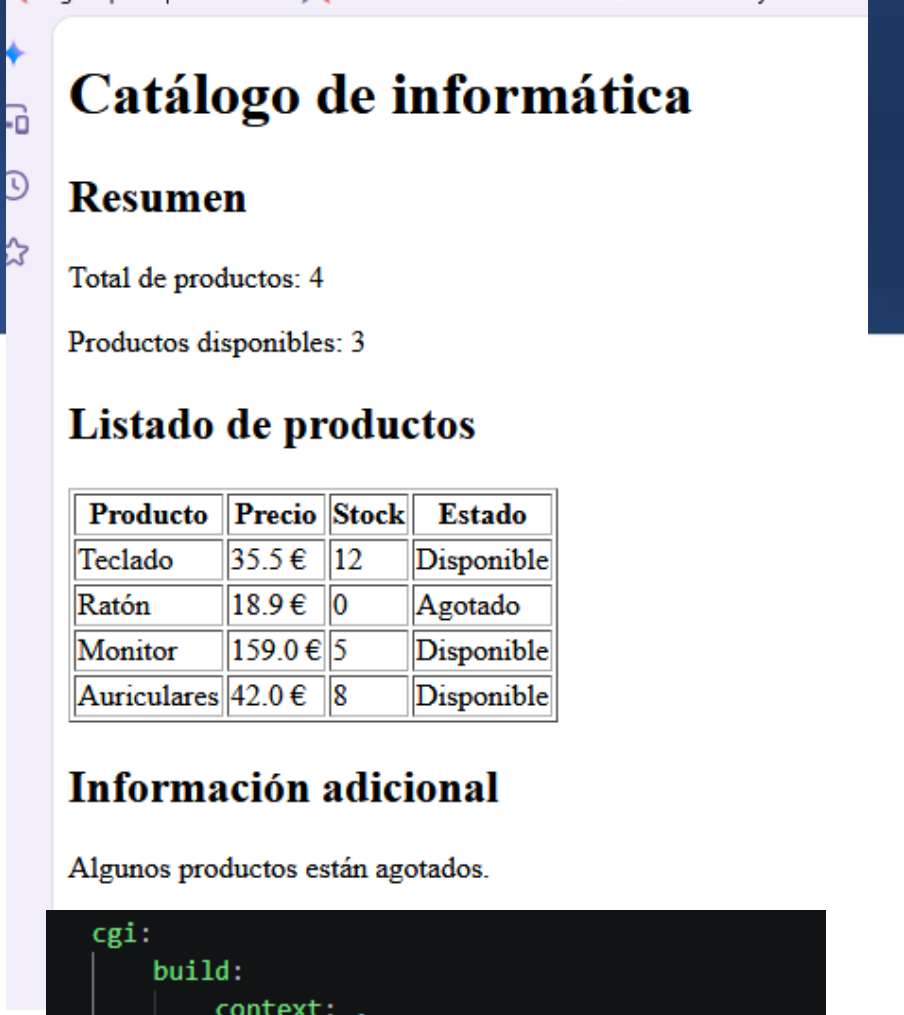
# Enviamos la respuesta CGI
print("Content-Type: text/html; charset=utf-8")
print()
print(resultado)
```

MAKO EN NUESTRO ENTORNO DE DESARROLLO

- Para ejecutar el programa Mako catalogo.py

  http://localhost:8080/cgi-bin/catalogo.py

- Un programa con Mako tiene tres partes
  - Un formulario de entrada de datos en /nginx/public/forms
  - Un fichero con extensión py,
    - en 2DAWdockers/cgi-bin/ aunque el servidor los monta el volumen en /app/cgi-bin
    - Escrito en Python
    - Proporciona datos de entrada
    - Establece la ubicación y codificación de la plantilla
    - Lanza el render de la plantilla
  - Una plantilla Mako
    - Escrita en HTML con código Mako (pseudo Python) incrustado
    - En 2DAWdockers/templatesMako/ (Aunque el servidor monta el volumen en /app/templatesMako



Catálogo de informática

Resumen

Total de productos: 4

Productos disponibles: 3

Listado de productos

Producto	Precio	Stock	Estado
Teclado	35.5 €	12	Disponible
Ratón	18.9 €	0	Agotado
Monitor	159.0 €	5	Disponible
Auriculares	42.0 €	8	Disponible

Información adicional

Algunos productos están agotados.

```
cgi:
    build:
        context: .
        dockerfile: docker/cgi/Dockerfile
    volumes:
    - ./cgi-bin:/app/cgi-bin
    - ./templatesMako:/app/templatesMako

    expose:
        # No necesita puerto externo
        - "9000"
    restart: unless-stopped
```

WSGI

- Generación dinámica usando WSGI
- Codigo incrustado Mako usando WSGI

ARQUITECTURA NGINX+GUNICORN+PYTHON

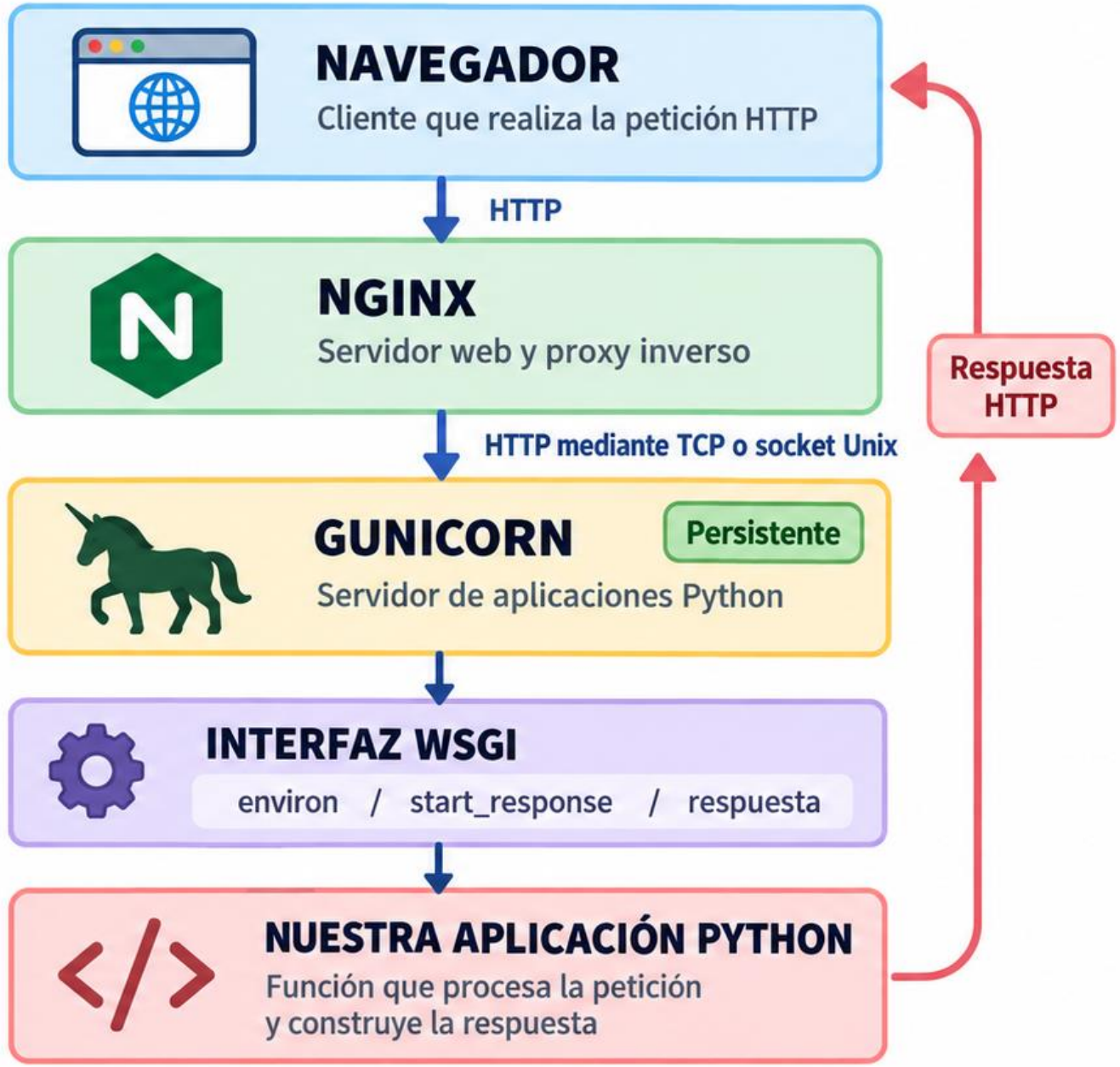
- La arquitectura para utilizar WSGI es Nginx + Gunicorn + Python
- Recuerda en WSGI, Gunicorn no ejecuta un programa Python como saludos.py creando un proceso independiente por programa con el interprete
- En su lugar mantiene un proceso que ejecuta el interprete de Python **de forma persistente** y que ejecuta las funciones en Python que se invoquen en la petición Post o Get.

UN PROGRAMA WSGI

- El programa **sigue siendo un programa Python pera ahora cumple las convenciones de WSGI**
  - La aplicación es un **objeto invocable**; en GENERAL UNA FUNCIÓN.
  - Recibe las entradas de la petición mediante un diccionario **environ** pasado como parámetro de entrada al objeto invocable
  - **Establece el estado y las cabeceras** de la respuesta **invocando start_response().**
  - **Devuelve un iterable de bytes** con el contenido generado.
  - No necesita **finalizar el proceso** para terminar de atender una petición.

```
def saludo(environ, start_response):
    nombre = environ.get("QUERY_STRING", "")
    contenido = f"<h1>Hola {nombre}</h1>"
    start_response("200 OK",[("Content-Type", "text/html; charset=utf-8")])
    return [contenido.encode("utf-8")]
```

Nuestra arquitectura de desarrollo



Generación dinámica usando WSGI

Es importante, para entender WSGI el cambio de paradigma con respecto a CGI:

En WSGI **no se puede proporcionar al programa parámetros de entrada por medio de variables de ambiente.**

Recuerda que en WSGI el único proceso es el proceso persistente. Este puede tener en su entorno variables de ambiente, pero la invocación de una función como respuesta a una petición Get o Post no va a tener variables de ambiente propias, porque la ejecución de esa función se da en el ámbito del mismo proceso persistente

El mecanismo de paso de parámetros de entrada es mucho más sencillo en realidad. Se trata de un programa python (el proceso persistente) invocando una función dentro del mismo proceso (también Python). Esto está inventado hace tiempo: se llama paso de parámetros.

Un programa llama Python, (el proceso persistente), llama a una función de la aplicación web (también Python) pasandole un objeto **environ** (también Python) con los parámetros de entrada

WSGI: entrada de datos de la petición

El servidor llama a una función Python y le pasa los datos



Navegador

Petición HTTP · GET o POST

Proceso Python persistente · servidor WSGI
(por ejemplo, Gunicorn)

Servidor WSGI
Prepara los datos de la petición

Llamada a función:
application(environ, start_response)

Aplicación web · función Python

environ: diccionario Python

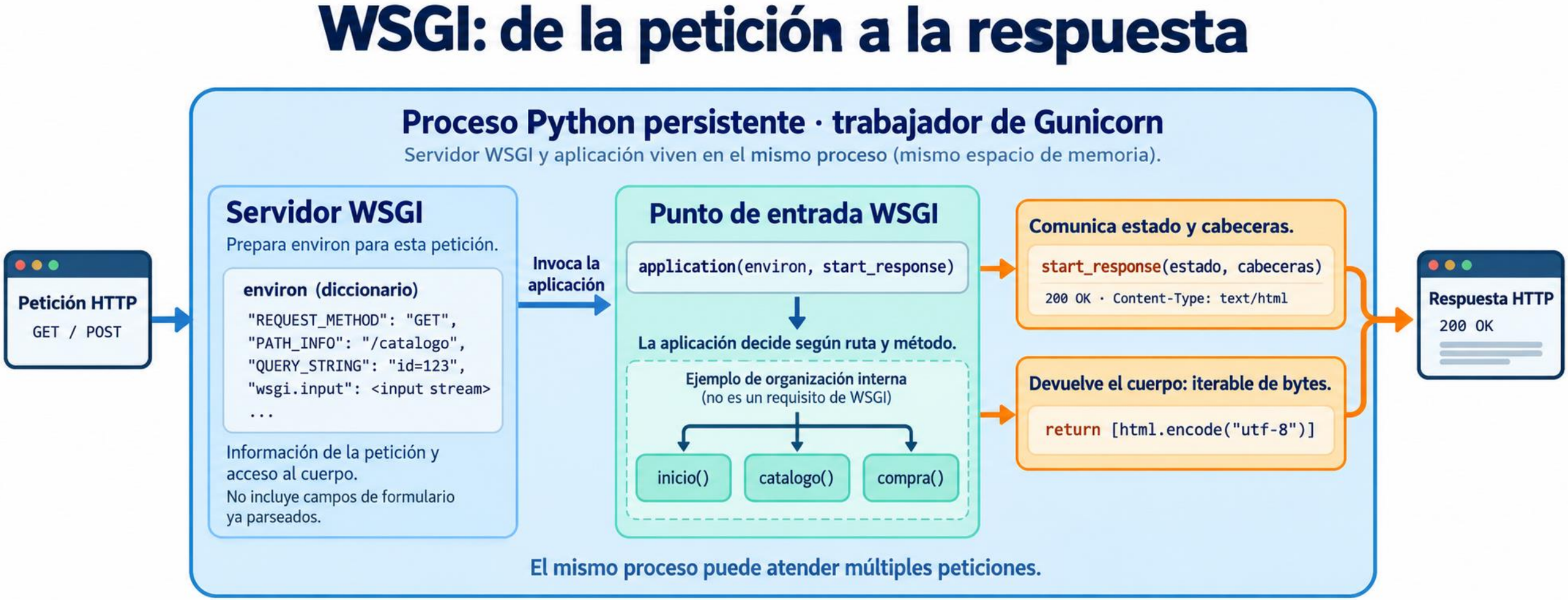
REQUEST_METHOD	→	método GET o POST
QUERY_STRING	→	parámetros de la URL
wsgi.input	→	flujo para leer el cuerpo de la petición

En POST, el cuerpo se lee y se interpreta según Content-Type.

start_response permite iniciar la respuesta HTTP.

La petición llega por HTTP. Dentro del proceso, los datos se entregan mediante parámetros de una función Python.

FUNCIONAMIENTO DE WSGI



En WSGI :

- La aplicación web **es un conjunto de funciones,** a las que se accede por una función raíz que hace de punto de entrada WSGI de la aplicación. A aplicación después decide que función atiende la URL
- Una petición get o post solicita la ejecución del punto de entrada, con parámetros adicionales que permiten discriminar que función debe atender la petición
- El proceso persistente, extrae de la petición los parámetros de entrada y los empaqueta en un **objeto environ** y llama a la función start_response(environ)
- La aplicación utiliza start_response para indicar el estado HTTP y las cabeceras de la respuesta.
- El proceso persistente invoca la función pasandole comopo parámetros de entrada el objeto environ, y el objeto start_response(estado, cabeceras)
- La salida es el valor de retorno de la función, en general el cuerpo de la respuesta

El objeto Environ contiene claves como REQUEST_METHOD, QUERY_STRING, CONTENT_TYPE y CONTENT_LENGTH, que se consultan mediante environ.get(). El cuerpo de la petición se lee a través de environ["wsgi.input"].



WSGI – START RESPONSE COMO FUNCIÓN CALLBACK

- Si te fijas el código del punto de entrada wsgi, application, tiene algo muy extraño. La función **start_response es a la vez, un parámetro de entrada** de la función, y **una función que se invoca** en el cuerpo de application
- Esto es otra característica de Python, y se llama **función CALLBACK**. A una función se le puede pasar como parámetro otra función. Fíjate en el ejemplo abajo:
  - **Ejecutar** recibe como parámetro, una funcion indeterminada, a la que llamamos **funcion**. Es la función **callback**
  - Al ejecutar(saludar, "Fulano"), **funcion** toma el valor de **saludar** y en su cuerpo se ejecuta **saludar(nombre):**
  - Al ejecutar(no_saludar, “Mengano"), f**uncion** toma el valor de **no_saludar** y se ejecuta la funcion **no_saludar(nombre)**
- Es decir, al pasar como parámetro una función X a la funcion Y, hacemos que el código de Y sea más versátil ya que puede ejecutar una entre varias posibles funciones
- Y la pregunta es porqué es necesario pasar **start_response** como parámetro callback?: Teniendo en cuenta lo dicho, si pasamos una funcion callback es porque esa función podría ser una de varias posibles opciones.
- Pues efectivamente esto es lo que ocurre en WSGI. **Diversas implementaciones del protocolo en diversos servidores web, pasaran diversas versiones de start_response** y application debe ser capaz de ejecutarla cualquiera que sea su implementación, siempre que se respeten los dos parámetros de entrada

```
def application(environ, start_response):
    contenido = "<h1>Hola mundo desde Python</h1>"
    start_response(
        "200 OK",
        [("Content-Type", "text/html; charset=utf-8")]
    )
    return [contenido.encode("utf-8")]
```

```
def saludar(nombre):
    print("Hola,", nombre)
def no_saludar(nombre):
    print("No pienso saludarte, ", nombre)
def ejecutar(funcion, nombre):
    funcion(nombre)
ejecutar(saludar, "Fulano")
ejecutar(no_saludar, "Mengano")
```

PROGRAMAR UNA APLICACIÓN WSGI SIN FRAMEWORKS

- Una aplicación WSGI debe ser un objeto invocable que acepte dos argumentos (environ y start_response) y devuelva un iterable que produzca secuencias de bytes
- En la imagen tienes una versión WSGI del programa saludo.py

```
def application(environ, start_response):


    contenido = "<h1>Hola mundo desde Python</h1>"


    start_response(
        "200 OK",
        [("Content-Type", "text/html; charset=utf-8")]
    )


    return [contenido.encode("utf-8")]
```

WSGI

- **Un programa wsgi tiene los siguientes elementos**
  - **Application:** Punto de entrada. Es una función que puede invocarse.
  - **Environ:** objeto de tipo dict que contiene información de HTTP y variables definidas por WSGI
  - **Start_response**: Función proporcionada por Gunicorn para que nuestra aplicación establezca el estado HTTP y las cabeceras de la respuesta.
  - **Contenid**o: Variable que contiene el HTML generado dinámicamente.
  - **Return [contenido.encode("utf-8")]** Devuelve el contenido de la respuesta mediante un iterable de objetos tipo byte.

```
def application(environ, start_response):

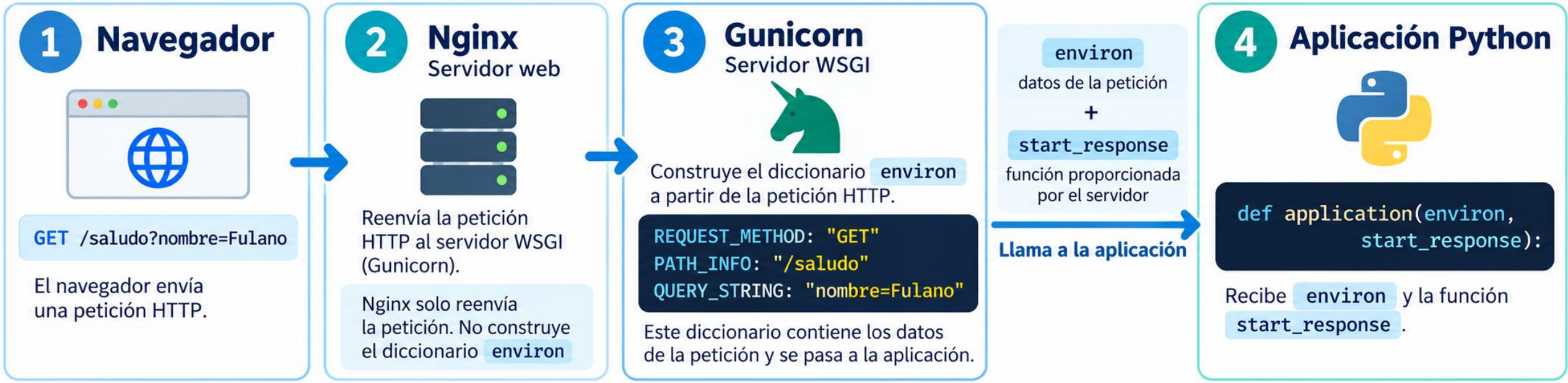

    contenido = "<h1>Hola mundo desde Python</h1>"


    start_response(
        "200 OK",
        [("Content-Type", "text/html; charset=utf-8")]
    )


    return [contenido.encode("utf-8")]
```

ENVIRON

- Gunicorn invoca la función punto de entrada pasándole los datos de la petición en unpbjeto diccionario python llamado **environ .** El tipo de datos de environ es **dict[str, object]**



•ENVIRON: DICT[STR, OBJECT]

```
{
    "REQUEST_METHOD": "GET", # Object es un texto
    "SCRIPT_NAME": "",
    "PATH_INFO": "/saludo",
    "QUERY_STRING": "nombre=Felipe",
    "SERVER_NAME": "localhost",
    "SERVER_PORT": "8000",
    "SERVER_PROTOCOL": "HTTP/1.1",


    "wsgi.version": (1, 0),
    "wsgi.url_scheme": "http", # Object es un texto
    "wsgi.multithread": True, # Object es un boolean
    "wsgi.multiprocess": False,
    "wsgi.run_once": False,


    "wsgi.input": objeto_de_entrada, # Object es un objeto de tipo indeterminado
    "wsgi.errors": objeto_de_errores
}
```

- El objeto environ es de tipo environ: dict[str, object]
- Las **claves** del diccionario son de tipo **str**
- **Pero los valores**, **pueden ser de tipos de datos muy diversos**, en algunos casos son texto, en otros números, en otros booleans, e incluso objetos cualquier tipo. Por eso los valores, **son de tipo Object**, que en Python es la clase madre de todos los objetos. De esta forma el valor puede ser cualquier cosa

•ENVIRON: DICT[STR, OBJECT]

```
{
    "REQUEST_METHOD": "GET", # Object es un texto
    "SCRIPT_NAME": "",
    "PATH_INFO": "/saludo",
    "QUERY_STRING": "nombre=Felipe",
    "SERVER_NAME": "localhost",
    "SERVER_PORT": "8000",
    "SERVER_PROTOCOL": "HTTP/1.1",


    "wsgi.version": (1, 0),
    "wsgi.url_scheme": "http", # Object es un texto
    "wsgi.multithread": True, # Object es un boolean
    "wsgi.multiprocess": False,
    "wsgi.run_once": False,


    "wsgi.input": objeto_de_entrada, # Object es un objeto de tipo indeterminado
    "wsgi.errors": objeto_de_errores
}
```

- La primera parte, en rojo, es información sobre la petición y sobre el servidor. Son todo datos que se pueden obtener del mensaje HTTP por el que ha llegado la petición
- La segunda parte, en verde, se refiere a **Información sobre el entorno de ejecución WSGI.** Son datos que el servidor proporciona para que la aplicación conozca cómo está ejecutándose; no son opciones que la aplicación configure al modificar el diccionario.
- La tercera parte, en morado, son objetos **que permiten leer datos y escribir diagnósticos como wsgi.input y wsgi.errors . Estos no contienen simplemente texto; contienen objetos que se utilizan de forma parecida a un archivo abierto.**
    - **Wsgi.input** permite leer el cuerpo de la petición
    - **wsgi.errors**: permite escribir mensajes de diagnóstico

COMO SE USA HACE LA ENTRADA EN WSGI

- **En una petición get** los parámetros se embeden en la URL, y por tanto se pueden obtener del objeto con clave **QUERY_STRING**

```
from urllib.parse import parse_qs
# URL: /saludo?nombre=Felipe
texto = environ.get("QUERY_STRING", "")
parametros = parse_qs(texto)
nombre = parametros.get("nombre", ["Invitado"])[0]
```

  - En una **petición post** los parámetros llegan en **wsgi.input** que contiene el cuerpo de la petición. Para obtener los parámetros de entrada hay que leerlo de una manera muy parecida a como se hacia en CGI para los métodos post

```
entrada = environ["wsgi.input"]
longitud = int(environ.get("CONTENT_LENGTH") or "0")
datos = entrada.read(longitud)  # Obtiene bytes
texto = datos.decode("utf-8")   # Obtiene str
```

PRINCIPALES CLAVES DE ENVIRON

Clave	Qué contiene	Ejemplo
REQUEST_METHOD	Método HTTP empleado.	"GET"
QUERY_STRING	Parámetros de la URL, sin el signo ?.	"nombre=Fulano&edad=25"
PATH_INFO	Ruta solicitada dentro de la aplicación.	"/saludo"
SCRIPT_NAME	Prefijo de la ruta donde está montada la aplicación.	""
SERVER_PROTOCOL	Versión del protocolo HTTP recibida.	"HTTP/1.1"
SERVER_NAME	Nombre del servidor.	"localhost"
SERVER_PORT	Puerto del servidor.	"8000"
CONTENT_TYPE	Tipo de contenido del cuerpo, si se ha indicado.	"application/json"
CONTENT_LENGTH	Longitud del cuerpo en bytes, cuando se conoce.	"27“ ojo no es un numero sino un string

RECUPERACIÓN DE DATOS DE ENVIRON

- Environ es un diccionario. Para obtener los valores sólo hay que seleccionar las claves.
- Esto se puede hacer pasando la clave entre corchetes **o preferiblemente usando el método get**, para evitar que un error en el nombre de la clave genere una excepción KeyError
- **Items() devuelve un iterable** que se puede usar en una instrucción for para recorrer todos los parámetros

```
metodo: str = environ["REQUEST_METHOD"]

query: str = environ.get("QUERY_STRING", "")

ruta: str = environ.get("PATH_INFO", "")


#acceso mediante items
for clave, valor in environ.items():
    print(clave, valor)
```

PROGRAMA WSGI PARA PETICIÓN GET

```
GET /saludo?nombre=Fulano HTTP/1.1 Host: localhost:8000
```

- Observa que hemos utilizado exactamente las mismas funciones que conocíamos de CGI: parse_qs() y escape().
- El principal cambio es la forma de recibir los datos y de devolver la respuesta

```python
def application(environ, start_response):
    # 1. Recuperamos QUERY_STRING del diccionario environ
    query: str = environ.get("QUERY_STRING", "")
    # 2. Convertimos los parámetros en un diccionario
    parametros: dict[str, list[str]] = parse_qs(query)
    # 3. Recuperamos el nombre
    nombre: str = parametros.get("nombre", ["Invitado"])[0]
    # 4. Construimos el documento HTML
    contenido = f"""
    <html>
    <body>
        <h1>Hola, {escape(nombre)}</h1>
    </body>
    </html>
    """
    # 5. Establecemos el estado y las cabeceras HTTP
    start_response(
        "200 OK",
        [("Content-Type", "text/html; charset=utf-8")]
    )
    # 6. Devolvemos el contenido codificado como bytes
    return [contenido.encode("utf-8")]
if __name__ == "__main__":
    with make_server("localhost", 8000, application) as servidor:
        print("Servidor disponible en http://localhost:8000")
        servidor.serve_forever()
```

ENVIRON CONTIENE TIPOS DE DATOS HETEROGENEOS

- Los elementos del diccionario environ no son todos de tipo str, hay algunos elementos que son boolean, otros son un flujo de bytes.
- Si desconocemos el tipo de daots puede ayudar la función type

```
print(type(environ["QUERY_STRING"]))
```

Clave	Tipo de su valor	Ejemplo
"REQUEST_METHOD"	str	"GET"
"QUERY_STRING"	str	"nombre=Ana&edad=20"
"PATH_INFO"	str	"/saludo"
"wsgi.multithread"	bool	True
"wsgi.input"	Objeto que permite leer el cuerpo de la petición	Se utiliza con .read() devuelve un objeto bytes

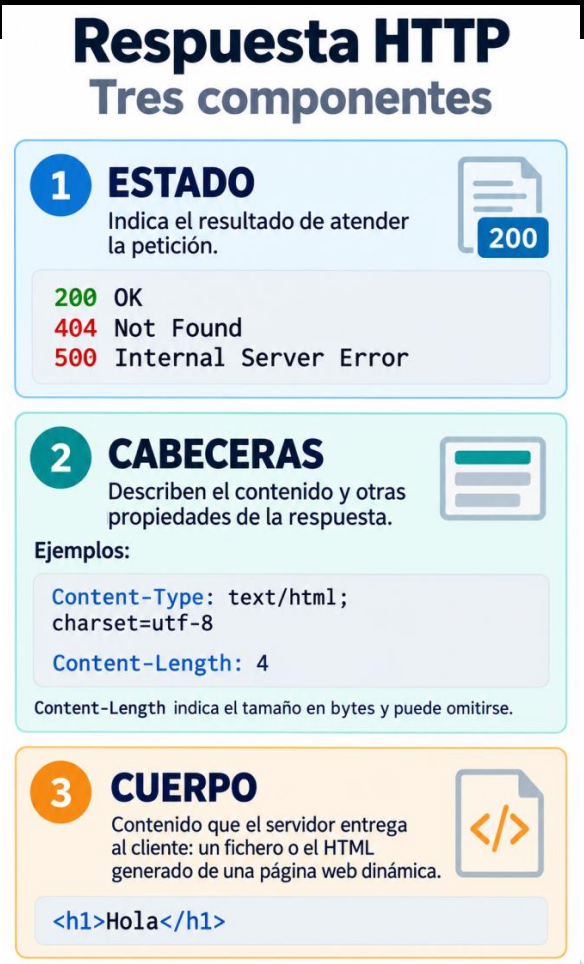
WSGI START_RESPONSE

```
start_response(
    "200 OK",                                   # str
    [                                           # lista
        ("Content-Type", "text/html; charset=utf-8"),  # tupla de 1 obligatoria
        ("Content-Length", "4") ,          # tupla de 2 str opcional
        ("Set-Cookie", "idioma=es"),    # tupla de 3 str opcional
        ("Cache-Control", "no-store")    # tupla de 4 str opcional
    ]
)
```

- **Start_response es una función con esta firma**

Start_response(str, list[tuple[str,str]]->write()

- En realidad, start_response() es una función buffer_salida.write(), que puede tener diversas implementaciones dependiendo del servidor de aplicaciones y por eso se pasa en la forma de una función callback, con ese nombre, start_response
- Para entender los parámetros de entrada de start_response, es necesario recordar que el navegador cliente recibirá la respuesta del servidor en la forma de mensajes HTTP, y que dichos mensajes tienen tres componentes , **estado, cabecera, y cuerpo**. Ver imagen a la derecha
- El primer parámetro de start_response es un str que indica el código de estado de la respuesta. En el ejemplo, el código entregado es “200 OK”. Pero es importante entender que start_response() debe lanzarse, al final, cuando la petición ya se ha procesado, de manera que sepamos si todo ha ido bien y se debe devolver el estado 200 OK o si ha habido un error y es necesario devolver 500 Internal server error
- El segundo argumento es mas complejo: se trata de una lista de tuplas.
  - Cada tupla contiene dos cadenas: el nombre de una cabecera y su valor.
  - Es necesario que el argumento sea una lista de tuplas, porque la cabecera puede tener uno, dos o más campos. Los campos más comunes, y que nosotros gastaremos son "Content-Type", y “Content-lenght”.
  - Pero podría haber mas como: set-cookie, Location, Cache-Control



START_RESPONSE – DETERMINACIÓN DEL ESTADO

- Como hemos indicado antes, start_response debe devolver un estado.
- Por tanto, antes de su ejecución, se tiene que haber resuelto todo el procesamiento, para que sea posible determinar si ha tenido éxito o ha habido alguna incidencia

```
def application(environ, start_response):
    try:
        # Primero procesamos la petición y generamos el
contenido
        contenido = generar_pagina()
        cuerpo = contenido.encode("utf-8")

        # Si hemos llegado aquí, el procesamiento ha terminado
bien
        estado = "200 OK"

    except Exception:
        # Si el procesamiento falla, preparamos una respuesta
de error
        cuerpo = b"<h1>Error interno del servidor</h1>"
        estado = "500 Internal Server Error"

    start_response(
        estado,
        [("Content-Type", "text/html; charset=utf-8")]
    )

    return [cuerpo]
```

CUERPO DE LA RESPUESTA

- En CGI utilizábamos `print()` para escribir la respuesta en stdout. En WSGI, la respuesta es el valor devuelto por la función punto de arranque, es decir, lo que se devuelve en el return.
- WSGI exige que nuestra aplicación devuelva un iterable cuyos elementos sean objetos bytes, no objetos str ni int.
- **Return podría devolver una pagina generada dinámicamente, o cualquier otro resultado del procesamiento de la aplicación WSGI, por ejemplo una imagen o un pdf**. Por eso , **cualquiera que sea el resultado devuelto se empaqueta en una secuencia de bytes** en el cuerpo del mensaje de respuesta http
- Así que es necesario devolver una secuencia de bytes. Pero en Python ni siquiera **existe** el tipo de datos byte. Para solucionar esa deficiencia en Python existe la clase Bytes. Un objeto Bytes es un iterable que representa una secuencia de bytes. En WSGI, el return de la función punto de entrada es un objeto de tipo Bytes
- Nota: En WSGI podemos usar print() para depuración. Pero su salida no constintuye la respuesta http

```python
# Nuestra aplicación WSGI
def application(environ, start_response):
    contenido = "<h1>Hola, mundo</h1>"
    start_response(
        "200 OK",
        [("Content-Type", "text/html; charset=utf-8")]
    )
    return [contenido.encode("utf-8")]

# Función que ejecuta el servidor para llamar a nuestro programa wsgi
def start_response_simulado(estado, cabeceras):
    print("Estado:", estado)
    print("Cabeceras:", cabeceras)
# Simulamos la llamada del servidor a la aplicación
respuesta = application({}, start_response_simulado)
# Recogemos los fragmentos del cuerpo en un único objeto bytes
cuerpo = b"".join(respuesta)
print("Como bytes:", cuerpo)
# Convertimos los bytes a texto para generar una salida por el terminal con el fin de depurar el programa
texto = cuerpo.decode("utf-8")
print("Como texto:", texto)
```

De texto a respuesta WSGI

1. Partimos de un texto (str)

```
contenido = "Hola"
```

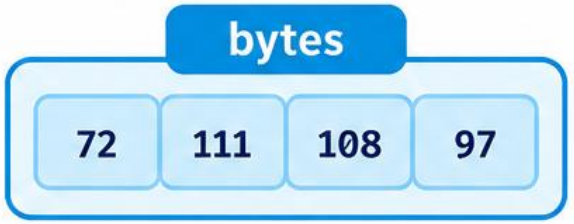
Un objeto de tipo `str` que contiene texto.

"Hola"

2. Codificamos el texto en UTF-8

```
datos = contenido.encode("utf-8")
```

Resultado: `b"Hola"`



Un único objeto `bytes`.
Al recorrerlo, obtenemos enteros.

3. Lo introducimos en una lista

```
respuesta = [contenido.encode("utf-8")]
```

Resultado: `[b"Hola"]`



Una lista cuyo único elemento es un objeto `bytes`.

Respuesta en WSGI

```
return [contenido.encode("utf-8")]
```

Gunicorn recorre el iterable y obtiene fragmentos de tipo `bytes`.

En resumen: al recorrer

Recorrer `b"Hola"` → 72, 111, 108, 97

Recorrer `[b"Hola"]` → `b"Hola"` (un único elemento)

- Contenido es una cadena codificada en Unicode (2 bytes por carácter)
- Pero la codificación Unicode es inadecuada para visualizar en el navegador un documento html. Por eso necesitamos aplicar el método encode y recodificar a UTF-8
- En UTF-8, los caracteres tienen 8 bits, son por tanto bytes. Como no existe el tipo byte en Python, encode sólo puede devolver un objeto de tipo Bytes
- El objeto Bytes devuelto por return se pone entre corchetes.
  - Los corchetes actúan como casts, convirtiendo en una lista el elemento que contienen. WSGI establece que el servidor debe recibir **un iterable de** objetos bytes. Por eso es necesario convertir la respuesta en una lista
- Ojo, es necesario que en start_response se indique en el campo content-type de la cabecera que el texto esta codificado en uft 8

```
def application(environ, start_response):


    contenido = "<h1>Hola mundo desde Python</h1>"


    start_response(
        "200 OK",
        [("Content-Type", "text/html; charset=utf-8")]
    )


    return [contenido.encode("utf-8")]
```

contenido.encode("utf-8") → devuelve un objeto bytes, que también es iterable, pero cuyos elementos individuales son enteros.
[contenido.encode("utf-8")] → devuelve una lista cuyo único elemento es un objeto bytes.

SHEBANGS EN WSGI

- En wsgi NO HACE FALTA PONER EL SHEBANG. La razón es que WSGI es un estándar que usa un único lenguaje de programación Python. Por tanto el servidor de aplicaciones Gunicorn sabe perfectamente que lenguaje se va a encontrar en los ficheros del program

Ejecutar una aplicación WSGI en nuestro setup

PROGRAMAS WSGI EN NUESTRO SETUP

- Los programas CGI que hemos hecho hasta ahora se ejecutaban en el contendor cgi, que contiene Python, Fciwrap, y Mako, y al que accedíamos por el puerto local 8080
- En cambio, los programas wsgi los vamos a ejecutar en el contenedor que contiene gunicorn+DJango, y al que se accede por el puerto local 8002. Por tanto, dejaremos los programas wsgi en C:\2DAWdockers\wsgi-app\
- El formulario para invocarlos tiene que llamar a:
  - Get: http://localhost:8002/?nombre=Ana
  - Post: http://localhost:8002/

EL MODELO WSGI NECESITA DE UN ENRUTADOR

Los servidores de aplicaciones gunicorn, enlazan un puerto local (el 8002) con un servicio que arranca un único programa, mediante una instrucción como esta:

```
docker compose exec wsgi gunicorn --bind 0.0.0.0:8000 enrutador:application
```

Es decir, el servidor, de facto SÓLO PUEDE SERVIR UNA ÚNICA APLICACIÓN. En este caso la aplicación enrutador, que hemos añadido y dejado en wsgi-app.

El mapeado dentro de dockers es 8002:8000, donde 8002 es el puerto del ordenador host y 8000 el puerto dentro del contenedor Gunicorn

0.0.0.0, en este caso, las del contenedor. Todo lo que llegue por ellas se reenvía al puerto del contenedor 8000 el cual se vincula a la aplicación enrutador:application

- Si queremos ejecutar otra aplicación distinta, por ejemplo notas.py, tendríamos que ejecutar (siempre que notas.py sea un programa wsgi)

```
docker compose exec wsgi gunicorn --bind 0.0.0.0:8000 notas:application
```

- La opción dos seria modificar enrutador.py para que dependiendo del camino de entrada de la url lance una aplicación u otra

HACER UNA APLICACIÓN WSGI DESDE UN ENRUTADOR

PASO 1: HACER FORMULARIO

- Vamos a hacer un programa sencillo: el usuario introduce un nombre y una nota en un formulario y la aplicación indica si el usuario esta aprobado o suspendido
- El formulario se llama notas-wsgi.html
- Importante: ahora el formulario no llama a un fichero Python con extensión py, sino a una ruta asociada al puerto 8002

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Consultar resultado</title>
</head>
<body>
    <h1>Consultar resultado</h1>
    <form action="http://localhost:8002/notas" method="post"
          accept-charset="UTF-8">
        <label for="nombre">Nombre:</label>
        <input type="text" id="nombre" name="nombre" required>
       <br><br>
        <label for="nota">Nota:</label>
        <input type="number" id="nota" name="nota"
               min="0" max="10" step="0.1" required>
        <br><br>
        <button type="submit">Consultar</button>
    </form>
</body>
</html>
```

- Ejemplo de aplicación basada en el lanzador

```python
from urllib.parse import parse_qs
from html import escape

def contenido(environ) -> str:
    longitud: int = int(environ.get("CONTENT_LENGTH", "") or "0")
    cuerpo: str = environ["wsgi.input"].read(longitud).decode("utf-8")
    parametros: dict[str, list[str]] = parse_qs(cuerpo)

    nombre: str = parametros.get("nombre", [""])[0]
    nota: float = float(parametros.get("nota", ["0"])[0])

    resultado: str = "Suspende"
    if nota >= 5:
        resultado = "Aprueba"
    return f"""
    <h1>Resultado de la evaluación</h1>
    <p>Nombre: {escape(nombre)}</p>
    <p>Nota: {nota}</p>
    <p>Resultado: {resultado}</p>
    """
```

CREO LA APLICACIÓN WSGI

- PASO 2: Hacemos el programa WSGI notas.py
- Tiene que ser un programa wsgi, PERO no tiene que tener una función application y tiene que realizar todos los pasos que hemos indicado en las diapositivas
- Cada función debe tener esta estructura

```
def contenido(environ) -> str:
# Leer los datos del formulario y realizar los cálculos.
# Devolver únicamente el HTML que irá dentro del body.
return "<h1>Resultado del ejercicio</h1>"
```

  - Guardar el programa en la carpeta 2DAWdockers\wsgi-app\
  - Sólo generamos las etiquetas entre <body> y </body> el resto de etiquetas las proporciona enrutador.py

ENRUTADOR.PY

```
from notas import contenido as contenido_notas
from calculadora import contenido as contenido_calculadora

def application(environ, start_response):
    # Rutas disponibles: añadir una entrada por cada ejercicio.
 rutas = {
    "/notas": contenido_notas,
    "/calculadora": contenido_calculadora
    }
    ruta: str = environ.get("PATH_INFO", "/")
    estado: str = "200 OK"
    contenido: str

    if ruta == "/":
        contenido = "<h1>El servicio WSGI funciona correctamente</h1>"
    elif ruta in rutas:
        contenido = rutas[ruta](environ)
    else:
        estado = "404 Not Found"
        contenido = "<h1>No existe esa ruta</h1>"

    html: str = f"""<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Pruebas WSGI</title>
</head>
<body>
    {contenido}
</body>
</html>"""
```

AÑADIR UN IMPORT Y UNA RUTA AL ENRUTADOR.PY

- Tenemos que añadir esta aplicación en el enrutador.py como un import y como una nueva ruta

```
from notas import contenido as contenido_notas
from calculadora import contenido as contenido_calculadora
```

```
 rutas = {
    "/notas": contenido_notas,
    "/calculadora": contenido_calculadora
}
```

- Para ejecutar una aplicación u otra el formulario tiene que invocar:

```
<form action="http://localhost:8002/notas" method="post">
<form action="http://localhost:8002/calculadora" method="post">
```

PROGRAMA WSGI PARA PETICIÓN POST

```
POST /saludo HTTP/1.1
Host: localhost:8000
Content-Type: application/x-www-form-urlencoded
Content-Length: 13


nombre=Fulano
```

Observa que hemos utilizado exactamente las mismas funciones que conocíamos de CGI: parse_qs() y escape().

El principal cambio es la forma de recibir los datos y de devolver la respuesta

```python
from urllib.parse import parse_qs
def application(environ, start_response):
    # 1. Recuperamos la longitud del cuerpo de la petición
    longitud: int = int(environ.get("CONTENT_LENGTH", "0") or "0")
    # 2. Obtenemos el flujo de entrada
    entrada = environ["wsgi.input"]
    # 3. Leemos los bytes del cuerpo de la petición
    datos: bytes = entrada.read(longitud)
    # 4. Convertimos los bytes en una cadena
    texto: str = datos.decode("utf-8")
    # 5. Interpretamos los parámetros
    parametros: dict[str, list[str]] = parse_qs(texto)
    # 6. Recuperamos el parámetro nombre
    nombre: str = parametros.get("nombre", ["Invitado"])[0]
    # Aquí continuaríamos construyendo la respuesta...
   contenido = f"""
 <html>
    <body>
        <h1>Hola, {escape(nombre)}</h1>
    </body>
    </html>
    """
    # 5. Establecemos el estado y las cabeceras HTTP
    start_response(
        "200 OK",
        [("Content-Type", "text/html; charset=utf-8")]
    )
    # 6. Devolvemos el contenido codificado como bytes
    return [contenido.encode("utf-8")]
if __name__ == "__main__":
    with make_server("localhost", 8000, application) as servidor:
        print("Servidor disponible en http://localhost:8000")
        servidor.serve_forever()
```

ERRORES DE EJECUCIÓN EN LOS PROGRAMAS WSGI

- Trazado de errores
- docker compose logs --tail=50 wsgi #lista 50 últimos errores
- docker compose logs --tail=1m wsgi #lista errores ultimo minuto

En WSGI podemos imprimir en pantalla para hacer trazas ¡, para visualizarlas ejecutar

docker compose logs -f wsgi

```
query = environ.get("STRING_QUERY","")
  print(query)
  parametros= parse_qs(query)
  print(parametros)
```

```
wsgi-1  | {'nombre': ['Paraguas', 'Bolso', 'Reloj', 'Movil', 'Cartera'], 'descripcion'
: ['Grande color rojo', 'Pequeño y color blanco', 'Marca acme carisimo', 'Xiamoi una p
atata', 'piel marron'], 'lugar': ['Autobus', 'Metro', 'Banco parque', 'No sabe', 'Cine
'], 'correo': ['correo1@example.com', 'correo2@example.com', 'correo3@example.com', 'c
orreo4@example.com', 'correo5@example.com'], 'tipoAviso': ['Destacado', 'Discreto', 'D
estacado', 'Discreto', 'Discreto']}
```

FLUJOS DE ENTRADA

- Hay que distinguir en WSGI parámetros textuales de objetos tipo flujo para bajar fichero
  - environ["QUERY_STRING"] permite consultar los parámetros de la URL, independientemente del método HTTP utilizado.
  - environ["wsgi.input"] proporciona un objeto de tipo flujo desde el que podemos leer los bytes del cuerpo de la petición.

```
POST /saludo HTTP/1.1
Host: localhost:8000
Content-Type: application/x-www-form-urlencoded
Content-Length: 13


nombre=Fulano
```

Clave	Tipo	Finalidad
wsgi.version	tuple	Versión del estándar WSGI.
wsgi.url_scheme	str	Protocolo de la URL: HTTP o HTTPS.
wsgi.input	Objeto de tipo flujo	Permite leer los bytes del cuerpo de la petición.
wsgi.errors	Objeto de tipo flujo	Permite escribir mensajes de error.
wsgi.multithread	bool	Indica si pueden existir llamadas simultáneas desde varios hilos.
wsgi.multiprocess	bool	Indica si la aplicación puede ejecutarse mediante varios procesos.
wsgi.run_once	bool	Indica si el servidor espera invocar la aplicación una sola vez.

EJEMPLO INFORMACIÓN PROVINCIAL – FORMULARIO GET

- El usuario usa este formulario para seleccionar una provincia y cuando pulsa enviar, se lanza la aplicación wsgi
- El formulario es una página web estática

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Consultar provincias</title>
</head>
<body>
    <h1>Información de una provincia</h1>


    <form action="http://localhost:8000/consulta" method="get">
        <label for="provincia">Selecciona una provincia:</label>


        <select name="provincia" id="provincia">
            <option value="alicante">Alicante</option>
            <option value="valencia">Valencia</option>
            <option value="madrid">Madrid</option>
            <option value="sevilla">Sevilla</option>
        </select>


        <button type="submit">Consultar</button>
    </form>
</body>
</html>
```

INFORMACIÓN PROVINCIAL MÉTODO GET - APP.PY

```python
from urllib.parse import parse_qs
# Poblaciones ficticias para este ejercicio
provincias = {
    "alicante": {
        "nombre": "Alicante",
        "prefijo_postal": "03",
        "comunidad": "Comunitat Valenciana",
        "poblacion": 2000000
    },
    "valencia": {
        "nombre": "Valencia",
        "prefijo_postal": "46",
        "comunidad": "Comunitat Valenciana",
        "poblacion": 2700000
    },
    "madrid": {
        "nombre": "Madrid",
        "prefijo_postal": "28",
        "comunidad": "Comunidad de Madrid",
        "poblacion": 7000000
    },
    "sevilla": {
        "nombre": "Sevilla",
        "prefijo_postal": "41",
        "comunidad": "Andalucía",
        "poblacion": 2000000
    }
}
```

```python
def application(environ, start_response):
    # 1. Leer los parámetros de la URL
    parametros = parse_qs(environ.get("QUERY_STRING", ""))
    provincia = parametros.get("provincia", [""])[0]

    # 2. Buscar la provincia y generar el HTML
    if provincia in provincias:
        datos = provincias[provincia]
        contenido = f"""
        <!DOCTYPE html>
        <html lang="es">
        <head>
            <meta charset="utf-8">
            <title>Resultado de la consulta</title>
        </head>
        <body>
            <h1>{datos["nombre"]}</h1>
            <p>Prefijo postal: {datos["prefijo_postal"]}</p>
            <p>Comunidad autónoma: {datos["comunidad"]}</p>
            <p>Población ficticia: {datos["poblacion"]} habitantes</p>
        </body>
        </html>
        """
        estado = "200 OK"
    else:
        contenido = "<h1>Provincia no encontrada</h1>"
        estado = "404 Not Found"

    # 3. Indicar el estado y las cabeceras
    start_response(
        estado,
        [("Content-Type", "text/html; charset=utf-8")]
    )
    # 4. Entregar el cuerpo de la respuesta
    return [contenido.encode("utf-8")]
```

EJEMPLO INFORMACIÓN PROVINCIAL – FORMULARIO POST

- En este formulario sólo cambia el método

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Consultar provincias</title>
</head>
<body>
    <h1>Información de una provincia</h1>


        <form action="http://localhost:8000/consulta"
          method="post" accept-charset="utf-8">
        <label for="provincia">Selecciona una provincia:</label>


        <select name="provincia" id="provincia">
            <option value="alicante">Alicante</option>
            <option value="valencia">Valencia</option>
            <option value="madrid">Madrid</option>
            <option value="sevilla">Sevilla</option>
        </select>


        <button type="submit">Consultar</button>
    </form>
</body>
</html>
```

INFORMACIÓN PROVINCIAL MÉTODO POST - APP.PY

```python
from urllib.parse import parse_qs
# Poblaciones ficticias para este ejercicio
provincias = {
    "alicante": {
        "nombre": "Alicante",
        "prefijo_postal": "03",
        "comunidad": "Comunitat Valenciana",
        "poblacion": 2000000
    },
    "valencia": {
        "nombre": "Valencia",
        "prefijo_postal": "46",
        "comunidad": "Comunitat Valenciana",
        "poblacion": 2700000
    },
    "madrid": {
        "nombre": "Madrid",
        "prefijo_postal": "28",
        "comunidad": "Comunidad de Madrid",
        "poblacion": 7000000
    },
    "sevilla": {
        "nombre": "Sevilla",
        "prefijo_postal": "41",
        "comunidad": "Andalucía",
        "poblacion": 2000000
    }
}
```

```python
def application(environ, start_response):

    # 2. Leer el cuerpo de la petición
    longitud = int(environ.get("CONTENT_LENGTH") or "0")
    cuerpo = environ["wsgi.input"].read(longitud)

    # 3. Convertir los bytes a texto e interpretar los parámetros
    texto = cuerpo.decode("utf-8")
    parametros = parse_qs(texto)
    provincia = parametros.get("provincia", [""])[0]

    # 4. Consultar el diccionario y generar el HTML
    if provincia in provincias:
        datos = provincias[provincia]
        contenido = f"""
        <!DOCTYPE html>
        <html lang="es">
        <head>
            <meta charset="utf-8">
            <title>Resultado de la consulta</title>
        </head>
        <body>
            <h1>{datos["nombre"]}</h1>
            <p>Prefijo postal: {datos["prefijo_postal"]}</p>
            <p>Comunidad autónoma: {datos["comunidad"]}</p>
            <p>Población ficticia: {datos["poblacion"]} habitantes</p>
        </body>
        </html>
        """
       estado = "200 OK"
    else:
        contenido = "<h1>Provincia no encontrada</h1>"
        estado = "404 Not Found"
    # 5. Preparar la respuesta
    start_response(
        estado,
        [("Content-Type", "text/html; charset=utf-8")]
    )
```

- En ocasiones puede ser necesario asegurar que el método utilizado para llamar al programa es post. Se puede lograr con el programa mostrado abajo

```
    # 1. Comprobar que se ha utilizado POST
    if environ.get("REQUEST_METHOD") != "POST":
        start_response(
            "405 Method Not Allowed",
            [
                ("Content-Type", "text/html; charset=utf-8"),
                ("Allow", "POST")
            ]
        )
        return [b"<h1>Utiliza el formulario para enviar los datos</h1>"]
```

CALCULADORA DE FIGURAS GEOMETRICAS

- En este ejemplo vamos a hacer una **calculadora de figuras geométricas**. El usuario elige una figura e introduce sus medidas en un formulario HTML estático. La función application() recibe los datos y decide qué función debe realizar el cálculo:
  - calcular_rectangulo()
  - calcular_triangulo()
- Ambas figuras utilizan una base y una altura, por lo que podemos compartir los campos del formulario.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Calculadora de áreas</title>
</head>
<body>
    <h1>Calculadora de áreas</h1>
    <form action="http://localhost:8000/calcular" method="post">
        <label for="figura">Figura:</label>
        <select name="figura" id="figura">
            <option value="rectangulo">Rectángulo</option>
            <option value="triangulo">Triángulo</option>
        </select>
        <p>
            <label for="base">Base en centímetros:</label>
            <input type="number" name="base" id="base"
                   min="0.01" step="any" required>
        </p>
        <p>
            <label for="altura">Altura en centímetros:</label>
            <input type="number" name="altura" id="altura"
                   min="0.01" step="any" required>
        </p>
        <button type="submit">Calcular</button>
    </form>
</body>
</html>
```

```python
from urllib.parse import parse_qs
def calcular_rectangulo(base, altura):
    return base * altura

def calcular_triangulo(base, altura):
    return base * altura / 2

def application(environ, start_response):
    # 1. Leer los datos enviados por el formulario
    longitud = int(environ.get("CONTENT_LENGTH") or "0")
    cuerpo = environ["wsgi.input"].read(longitud)
    parametros = parse_qs(cuerpo.decode("utf-8"))
    figura = parametros.get("figura", [""])[0]
    try:
        base = float(parametros.get("base", [""])[0])
        altura = float(parametros.get("altura", [""])[0])
        if base <= 0 or altura <= 0:
            raise ValueError

        # 2. Elegir la función que realiza el procesamiento
        if figura == "rectangulo":
            area = calcular_rectangulo(base, altura)
            nombre = "Rectángulo"
        elif figura == "triangulo":
            area = calcular_triangulo(base, altura)
            nombre = "Triángulo"
        else:
            raise ValueError
```

```python
        # 3. Generar el contenido de la respuesta
        contenido = f"""
        <!DOCTYPE html>
        <html lang="es">
        <head>
            <meta charset="utf-8">
            <title>Resultado</title>
        </head>
        <body>
            <h1>Área del {nombre.lower()}</h1>
            <p>Base: {base} cm</p>
            <p>Altura: {altura} cm</p>
            <p>Área: {area:.2f} cm²</p>
        </body>
        </html>
        """
        estado = "200 OK"
    except ValueError:
        contenido = """
        <h1>Datos incorrectos</h1>
        <p>Selecciona una figura e introduce medidas positivas.</p>
        """
        estado = "400 Bad Request"

    # 4. Entregar la respuesta al servidor
    start_response(
        estado,
        [("Content-Type", "text/html; charset=utf-8")]
    )
    return [contenido.encode("utf-8")]
```

Codigo incrustaco mako con wsgi

MAKO CON WSGI

- Con WSGI la plantilla Mako no va a cambiar en nada. Unicamente el programa lanzador Python va a sufrir algunos cambios
- Con **WSGI, algunas cosas no cambian en el programa lanzador Python**
  - Definir y Cargar plantilla
  - Lanzar render() con los parámetros de entrada
- Lo que cambia, con respecto a CGI es
  - **El programa Python debe definirse como una función punto de entrada**
  - **Cómo se recibe la petición (la consulta de entorno por medio de environ en lugar de consultar las variables de entorno)**
  - **como se devuelve la respuesta**. (por medio de un objeto **bytes** en lugar de hacer print de una cadena de texto)

EL PROGRAMA LANZADOR DEBE SER UNA FUNCION

CGI

```python
#!/usr/bin/env python3
from mako.template import Template
# Datos que proporcionaremos a la plantilla
productos = [
    {"nombre": "Teclado", "precio": 35.50, "stock": 12},
    {"nombre": "Ratón", "precio": 18.90, "stock": 0},
    {"nombre": "Monitor", "precio": 159.00, "stock": 5},
    {"nombre": "Auriculares", "precio": 42.00, "stock": 8}
]


# Cargamos el documento que contiene HTML y código incrustado
plantilla = Template(
    filename="catalogo.html",
    input_encoding="utf-8",
    default_filters=["h"]
)


# Procesamos la plantilla proporcionándole los productos
resultado = plantilla.render(
    productos=productos,
    titulo="Catálogo de informática"
)
```

WSGI

Se define una función a la que se le pasa como parámetros environ y Star_response y debe tener un return

```python
from mako.template import Template
def application(environ, start_response):
    # Datos que proporcionaremos a la plantilla
    productos = [
        {"nombre": "Teclado", "precio": 35.50, "stock": 12},
        {"nombre": "Ratón", "precio": 18.90, "stock": 0},
        {"nombre": "Monitor", "precio": 159.00, "stock": 5},
        {"nombre": "Auriculares", "precio": 42.00, "stock": 8}
    ]
    # Cargamos el documento que contiene HTML y código incrustado
    plantilla = Template(
        filename="catalogo.html",
        input_encoding="utf-8",
        default_filters=["h"]
    )
    # Procesamos la plantilla proporcionándole los productos
    resultado = plantilla.render(
        productos=productos,
        titulo="Catálogo de informática"
    )
    # Indicamos el estado HTTP y las cabeceras de respuesta
    start_response(
        "200 OK",
        [("Content-Type", "text/html; charset=utf-8")]
    )
```

ENTRADA DE DATOS DE LA PETICIÓN GET WSGI EN EL LANZADOR PYTHON

- Si la petición es get se debe leer el campo QUERY_STRING del objeto environ,
- Despues se parsea con urllib.parse.parse_qs

```python
from urllib.parse import parse_qs
from mako.template import Template

def application(environ, start_response):
    # 1. Obtener los parámetros GET
    parametros = parse_qs(environ.get("QUERY_STRING", ""))

    # parse_qs devuelve listas de valores para cada parámetro
    nombre = parametros.get("nombre", ["Invitado"])[0]

    # 2. Cargar la plantilla
    plantilla = Template(
        filename="saludo.html",
        input_encoding="utf-8",
        default_filters=["h"]
    )

    # 3. Pasar los datos a la plantilla
    resultado = plantilla.render(nombre=nombre)

    # 4. Enviar la respuesta
    start_response(
        "200 OK",
        [("Content-Type", "text/html; charset=utf-8")]
    )
    return [resultado.encode("utf-8")]
```

ENTRADA DE DATOS DE LA PETICIÓN POST WSGI EN EL LANZADOR PYTHON

- Con **POST**, leemos los datos del cuerpo de la petición mediante environ["wsgi.input"]
- Para ello primero leemos la longitud del cuerpo
- Despues se parsea con urllib.parse.parse_qs

```python
from urllib.parse import parse_qs
from mako.template import Template
def application(environ, start_response):
    # 1. Obtener los datos enviados por POST
    longitud = int(environ.get("CONTENT_LENGTH") or "0")
    cuerpo = environ["wsgi.input"].read(longitud).decode("utf-8")

    # parse_qs devuelve listas de valores para cada parámetro
    parametros = parse_qs(cuerpo)
    nombre = parametros.get("nombre", ["Invitado"])[0]

    # 2. Cargar la plantilla
    plantilla = Template(
        filename="saludo.html",
        input_encoding="utf-8",
        default_filters=["h"]
    )
    # 3. Pasar los datos a la plantilla
    resultado = plantilla.render(nombre=nombre)
    # 4. Enviar la respuesta
    start_response(
        "200 OK",
        [("Content-Type", "text/html; charset=utf-8")]
    )
    return [resultado.encode("utf-8")]
```

EL PROGRAMA LANZADOR DEBE SER UNA FUNCION

- Los datos de … ecibe la función.
- El cuerpo de …
- El estado y la …

```
#!/usr/bin/env python3
from mako.template import Template
# Datos que proporcionaremos a la plantilla
productos = [
    {"nombre": "Teclado", "precio": 35.50, "stock": 12},
    {"nombre": "Ratón", "precio": 18.90, "stock": 0},
    {"nombre": "Monitor", "precio": 159.00, "stock": 5},
    {"nombre": "Auriculares", "precio": 42.00, "stock": 8}
]


# Cargamos el documento que contiene HTML y código incrustado
plantilla = Template(
    filename="catalogo.html",
    input_encoding="utf-8",
    default_filters=["h"]
)


# Procesamos la plantilla proporcionándole los productos
resultado = plantilla.render(
    productos=productos,
    titulo="Catálogo de informática"
)


# Mostramos el documento HTML generado
print(resultado)
```

RESPUESTA

- El método render() procesa la plantilla con los datos proporcionados y, en este ejemplo, devuelve una cadena str:

```
resultado = plantilla.render(nombre=nombre)
```

- Ejecutar start_response para establecer estado y cabeceras de la respuesta HTTP:

```
start_response(
    "200 OK",
    [("Content-Type", "text/html; charset=utf-8")]
)
```

- Devolver el cuerpo como una lista de objetos bytes

```
return [resultado.encode("utf-8")]
```

ENVIRON Y START RESPONSE

- Si en CGI la entrada se realiza mediante objetos os.environ
- En wsgi el esquema cambia, no se llama a la ejecución de un modulo python, sino que se invoca una funcion
- Dicha función tiene siempre dos parámetros de entrad environ y start_response